. . . for Solving and Optimizing Engineering Models

Stirling-Cycle Model Class Reference Guide

Including Pulse-Tube and Low-T Cooler Supplements

David Gedeon

Sage v5 Edition

Gedeon Associates
16922 South Canaan Road
Athens, OH 45701

August 10, 2007

Copyright © 1999–2006 by Gedeon Associates
Cover clipart by Image Club Graphics Inc.

Typeset in TEX with pcTEX and LATEX software

Contents

Preface **vii**

I Stirling Cycle Model Class **1**

1 Overview **3**

2 How Do I Model... **5**
2.1 A Variable-Volume Space 5
2.2 A Regenerator 7
2.3 An Isothermal Heat Exchanger 8
2.4 An Internally-Finned Heat Exchanger 9
2.5 An Isothermal Matrix Heat Exchanger 10
2.6 A Conductive Matrix Heat Exchanger 11
2.7 An External Conduction Path or Busbar 13
2.8 Interconnected Heat-Exchanger Passages 14
2.8.1 Same Length Ducts 14
2.8.2 Different Length Ducts 15
2.9 A Displacer and Cylinder 16
2.10 A Regenerative Displacer and Cylinder 18
2.11 Displacer Appendix and Seal Leakage 18

3 Boundary Connections **21**
3.1 Force Connections 21
3.2 Pressure Connections 21
3.3 Heat Flow Connections 21
3.4 Gas Flow Connections 22
3.5 Density Connections 22

4 Entropy Generation **23**
4.1 Lost Available Energy 23
4.2 Second-Law Balance 24

5 Stirling Root Component **25**
5.1 Working Gas 26
5.1.1 Conductivity $k(T)$ and Viscosity $\mu(T)$ 27
5.1.2 Ideal Gas Objects 27
5.1.3 Redlich-Kwong Gas Objects 29
5.1.4 Tabular Gas Objects 35

6 Moving Parts **39**
6.1 Simple Crank Piston 39
6.2 Flexure Springs 40
6.2.1 Flexure Stack 40
6.2.2 Flexure-Stack Couple 41
6.2.3 Theory 41
6.3 Nonlinear Spring 45
6.4 Relative Nonlinear Spring 46
6.5 Linear Motors 46
6.5.1 Phasor Motor 46
6.5.2 Time-Ring Motor 47

7 Thermal Solids **49**
7.1 Attachment Child Components 49
7.2 Coordinate Conventions 50
7.3 Entropy Generation 51
7.4 Solid Material 52
7.5 Heat Sources 53
7.5.1 Point Heat Source 53
7.5.2 Line Heat Source 53
7.5.3 Isothermal Surface 53
7.6 Heat Conductors 54
7.6.1 Bar Conductor 54
7.6.2 Distributed Conductor 55
7.6.3 Conductive Surface 56
7.7 Quasi-Adiabatic Surfaces 60
7.7.1 Thick Surface 64
7.7.2 Thin Surface 64
7.7.3 Rigorous Surface 64
7.8 Floating Isothermal Surface 65

8 Gas Domains **67**
8.1 Attachment Child Components 67
8.2 Coordinate Conventions 68
8.3 Entropy Generation 69
8.4 Pressure Source Component 70
8.5 Gas Domain Components 70
8.5.1 Matrix Gas Domains 75
8.5.2 Duct Gas Domains 75
8.5.3 Variable-Volume Gas Domains 75
8.5.4 Combustion-Space Gas Domains 76
8.6 Gas Domain Theory 76
8.6.1 Viscous Pressure Gradient F 79
8.6.2 Film Heat Transfer Q_w 81
8.6.3 Combustion Heating Q_c 83
8.6.4 Gas Axial-Conduction Heat Flux q 83
8.6.5 Equations of State 83
8.6.6 Solution Method 84
8.7 Flow Connector Theory 86
8.7.1 Bernoulli's Law 87
8.7.2 Energy Continuity 87
8.8 Turbulence Models 92
8.8.1 Matrix Gas Domains 92
8.8.2 Duct Gas Domains 93
8.8.3 Variable Volume Gas Domains 101

9 Canisters **103**
9.1 Tubular Canister 104
9.2 Annular Canister 104
9.3 Tubular-Cone Canister 105
9.4 Annular-Cone Canister 106

10 Heat Exchangers **107**
10.0.1 Solid Tortuosity 109
10.0.2 Gas Axial-Conduction Enhancement 110
10.1 Woven Screen Matrix 111
10.1.1 Friction Factor 111
10.1.2 Nusselt Number 111
10.1.3 Axial-Conduction Enhancement 112
10.1.4 Tortuosity 112
10.2 Random Fiber Matrix 112
10.2.1 Friction Factor 113
10.2.2 Nusselt Number 113

10.2.3 Axial-Conduction Enhancement 113
10.2.4 Tortuosity 113
10.3 Packed Sphere Matrix 114
10.3.1 Friction Factor 114
10.3.2 Nusselt Number 114
10.3.3 Axial-Conduction Enhancement 114
10.3.4 Tortuosity 114
10.4 Wrapped Foil Matrix 114
10.4.1 Friction Factor 115
10.4.2 Nusselt Number 115
10.4.3 Axial-Conduction Enhancement 115
10.4.4 Tortuosity 115
10.5 Generic Matrix 116
10.5.1 Friction Factor 117
10.5.2 Nusselt Number 117
10.5.3 Axial-Conduction Enhancement 117
10.5.4 Tortuosity 117
10.6 Tube Bundle 117
10.6.1 Friction Factor 118
10.6.2 Nusselt Number 118
10.6.3 Axial-Conduction Enhancement 119
10.7 Rectangular Channels 119
10.7.1 Friction Factor 120
10.7.2 Nusselt Number 121
10.7.3 Axial-Conduction Enhancement 121
10.8 Rectangular Fins 121
10.9 Generic Cylinder 122
10.9.1 Length 123
10.9.2 Friction Factor 123
10.9.3 Nusselt Number 123
10.9.4 Axial-Conduction Enhancement 124

11 Miscellaneous Parasitics 125
11.1 Annulus (Shuttle/Seal/Appendix) 125
11.1.1 Theory 126
11.2 Radiation Transport Path 127
11.2.1 Theory 127

12 Composite Model Components 129
12.1 Piston-Cylinder Composites 129
12.1.1 Phasor Free-Piston and Cylinder 130
12.1.2 Time-Ring Free-Piston and Cylinder 130

12.1.3 Phasor Constrained Piston and Cylinder 130
12.1.4 Time-Ring Constrained Piston and Cylinder 131
12.1.5 Simple-Crank Piston and Cylinder 131
12.1.6 Phasor Free-Piston and Free-Cylinder 131
12.1.7 Time-Ring Free-Piston and Free-Cylinder 132

13 Submodels and Containers **133**
13.1 Submodels 133
13.2 Containers 133
13.2.1 Parallel Container 134
13.2.2 Multi-Length Container 135

14 Material Property Data Bases **137**
14.1 Changing Properties with PropBase 137
14.2 Log Splines 138
14.3 Solid Properties 138
14.4 Gas Properties 138
14.4.1 Redlich-Kwong Fitting 139
14.4.2 Entering Tables of State 141

II Pulse-Tube Model-Class Supplement **147**

15 Overview **149**

16 Compliance Duct Gas Domains **151**
16.1 Theory 152
16.1.1 Turbulent Conduction 152
16.1.2 Free Convection 152
16.1.3 Boundary Convection 154
16.1.4 Streaming Convection 155
16.2 Numerical Diffusion 157
16.2.1 Quantitative Estimates 157
16.2.2 Interpretation in terms of Refrigeration Power . . . 158

17 Compliance Duct Geometries **159**
17.1 Compliance Tube 159
17.2 Tapered Compliance Tube 159

18 Flow Restrictors **161**
18.1 Sintered Powder Plug 162
18.2 Sharp-Edged Orifice 163
18.3 Asymmetric Sharp-Edged Orifice 163

18.4 Check Valve 164
18.5 Time-Dependent Valve 165
18.6 DC Flow Blocking Restrictors 166
18.7 Mass-Flow Pump 167
18.8 Mass-Flow Driver 168
18.9 Flow Restrictor Theory 168

III Low-T Cooler Model-Class Supplement 173

19 Overview 175

20 Working Gas 177
20.1 TBSpline2Gas 178
20.1.1 Benefits of TBSpline2Gas 178
20.1.2 Entering Tables of State 179
20.2 TBSpline3Gas 180
20.2.1 Entering Tables of State 181

21 Low T Components 187
21.1 Adiabatic Compressor 187
21.1.1 Theory 189
21.2 Pressure-Regulated Compressor 190
21.3 Flow Reversers 190
21.3.1 Theory 191
21.4 Flow Separator 192
21.4.1 Theory 194

Bibliography 199

Preface

Combined Model Classes

This manual combines together three related model classes that were previously documented separately: the stirling-cycle model class the pulse-tube model class and the low-T cooler model class. Combining all three under one cover simplifies the printing process and also makes it easier for you to find things in one place. But the software for each model class is still licenced separately, which means that not all the model components documented in this manual may be available in your software distribution.

The first part of this manual documents the stirling-cycle model class, which forms the basis for the other two model classes. The stirling model class is designed to model most instances of what are generally known as stirling-cycle machines, whether free-piston or kinematicly driven, engines or coolers. The second part documents the pulse-tube model class, which is extends the stirling model class with certain components required for modeling pulse-tube coolers and other thermoacoustic devices containing thermal buffer tubes, orifi, valves, pumps, etc. All model components available to the stirling-cycle model class are also available to the pulse-tube model class. The third part documents the low-T cooler model class, which extends the pulse-tube model class with a special gas type having an equation of state designed for accuracy at extremely low temperatures (near the working gas critical temperature) and some model components designed for modeling Joule-Thomson coolers.

New in Version 4

On the software side, the Sage user interface and all three model classes have evolved significantly since the previous edition (third edition) of the written manuals. This manual brings the documentation up to date with the software, as of version 4.00. An exhaustive list of software improvements,

starting from the earliest days of Sage, is found in the file UpgradeHistory.pdf. This file is located in the Docs\UpgradeHistory subdirectory under your Sage installation directory (default c:\Program Files\Gedeon\Sage 4). Each software improvement documented there is generally associated with a revision in this reference manual.

Part I

Stirling Cycle Model Class

Chapter 1

Overview

The stirling model class allows you to model a wide variety of stirling-cycle coolers and engines within the Sage modeling framework. A stirling machine is built up from component parts of your choosing representing thermal solids, gas domains, canisters, heat exchangers, piston-cylinder pairs, and so forth. The parts function as a whole by virtue of their interconnections — heat and gas flows through appropriate boundaries, forces acting on appropriate attachment points and pressures on appropriate area faces. Add to this the possibility for user-defined variables and custom optimization specifications and you can see that the possible combinations are endless.

The distribution CD comes with a number of stirling-instance sample files to get you started (in the Apps\Stirling\Data sub-directory under the installation directory). These are accessible from Sage with the File|Open menu command. There may well be a sample file very close to the stirling geometry you are interested in. All you have to do is save it with another name (File|Save As) and edit the model to your liking. Each sample file also comes with documentation in an ASCII-format companion file with the same name but file extension doc. These are installed in the same directory as the sample files. Use your favorite word processor to read them.

The remainder of this guide will help you answer detailed questions you may have about what a particular model component does, its inner details and how it fits into the grand scheme of things.

Chapter 2

How Do I Model...

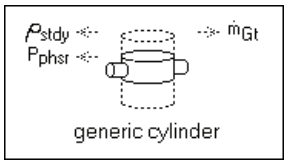
In case you are starting from scratch, these step-by-step instructions show you how to create some common elements of stirling machines from Sage model components and illustrate some of the finer points of the Sage interface. Some of the basic concepts introduced in the early examples are not repeated in later examples, so if you are new to Sage you might want to work through the first few examples in order. *Input files for all examples are found in the* Apps\Stirling\Data\Elements *sub-directory under the installation directory.*

You can use the copy-and-paste function of Sage's Edit menu (or the tool buttons) to copy model components from the sample files into your data file. You can only copy single, disconnected, model components this way (although any child components are copied too). After you copy a model component you have to first open the model you wish to paste it into. It is not possible to copy and paste between two separate Sage applications using the Windows clipboard.

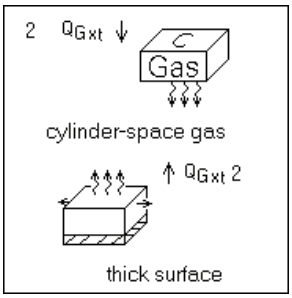
2.1 A Variable-Volume Space

Compression spaces or expansion spaces are examples of variable-volume spaces. From a modeling point of view both are essentially identical. *This example corresponds to sample file vvspace.stl*

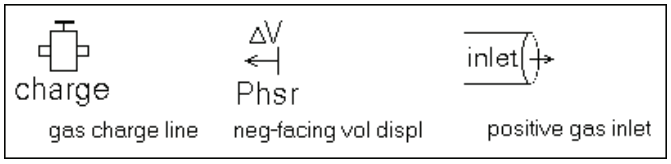
- From the *Basic* page of the component palette for the root-level edit form, click on the generic cylinder, position the mouse cursor in the edit form and click again to drop the generic cylinder into the edit window. The connector arrows will appear later.



- Double-click on the generic cylinder, to open its edit window, then drop in a cylinder-space gas from the *Gas Domain* palette page and a thick surface from the *Cylinder Walls* page. Establish a heat-flow connection between the gas and solid by dragging the mouse between connector arrows. Matching numbers indicate established connections. A thick surface models a relatively massive cylinder wall that is thermally isolated from a heat source or sink. **optional**: You could substitute an isothermal surface or conductive surface for the thick surface to model net heat transfer between the wall and gas.



- Double-click on the cylinder-space gas, to open its edit window, then drop in the appropriate sort of gas inlet from the *Charge/Inlet* page of the palette, in this case a positive gas inlet. In this case, also drop in a gas charge line, but see the tip below. Finally, drop in the appropriate sort of displacement component from *Volume Displacements* page of the palette, in this case a neg-facing phasor vol displ. Phasor vol displacements get connected to phasor moving parts (representing pistons and displacers) and Gt vol displacements to Gt moving parts with higher-harmonic content. Move the gas-inlet $\dot{m}_{Gt}$ connector arrow, the charge-line ρ_{stdy} arrow and the volume-displacement P_{phsr} arrow up to the root-component level by highlighting them with the mouse (hold down the shift key to highlight all three) then clicking the red *up-arrow* tool. Eventually, you will want to connect the gas-inlet $\dot{m}_{Gt}$ arrow to another gas domain, the charge-line ρ_{stdy} arrow to a pressure source and the volume-displacement P_{phsr} to a piston area attachment. **optional**: You may drop more than one gas inlet or volume displacement into a variable volume space. There are no restrictions. Even two or more in the same direction are possible.



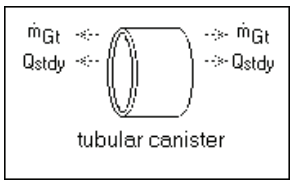
- Move up to the root level and your variable-volume space is now ready for connection to other model elements.

Tip: You may drop a charge line component into any gas domain for connection to a pressure source. But only one gas domain in an interconnected network of gas domains can have a charge line. Otherwise a solution singularity will result. Generally, the compression-space gas domain is a good place for the charge line.

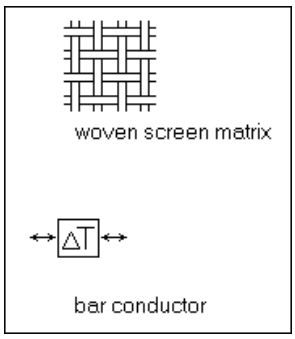
2.2 A Regenerator

This example corresponds to sample file regen.stl

- From the component palette of the Sage form, drop an appropriate canister component into the edit form, in this case a tubular canister. The connector arrows will appear later.



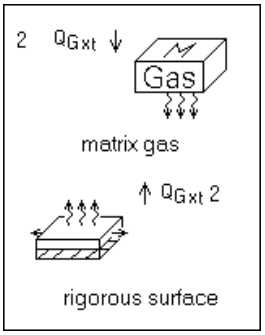
- Double-click on the canister to open its child-component edit-window then drop in an appropriate matrix component from the *Matrices* page of the palette and an appropriate wall-conduction component (optional) from the *Heat Flows* page. In this case the matrix is woven screen and the wall is modeled as a bar conductor.



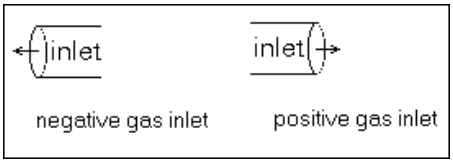
- Move the bar-conductor connector Q_{stdy} arrows up to the root-component level by highlighting them with the mouse (holding down the shift key to highlight both) then clicking the *red up-arrow* tool. Eventually, you will want to connect these arrows to temperature sources.

- Double-click on the canister, then matrix, to open the matrix edit window, then drop in a matrix gas from the *Gas Domain* palette page and a rigorous surface from the *Matrix Solids* page. Establish a heat-flow connection

between the gas and solid by dragging the mouse between connector arrows.



- Double-click on the matrix gas, to open its edit window, then drop in a negative and positive gas inlet from the palette. Highlight the $\dot{m}_{Gt}$ connector arrows and move them up to the root-component level by clicking the *red up-arrow* tool. Eventually, you will want to connect these arrows to other gas domains.



Optional: Within the rigorous surface, you could create negative and positive heat-flow ends for axial matrix conduction, then move the resulting Q_{stdy} connection arrows up to the root-component level for connection to temperature sources. This would make sense if the ends of your regenerator matrix were in good thermal contact with some sort of solid conduction paths. Otherwise gas heat transfer will couple internal matrix axial conduction to the enthalpy flow carried by the gas.

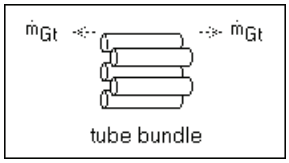
Tip: Change the names of the components displayed in the edit form to correspond with their function, using the Specify|Component Name menu command when the component is highlighted. For example, you might want to change the generic name *tubular canister* to *regenerator*, or whatever. This will help you understand the purpose of your model components and is especially useful when there is more than one component present with the same generic name.

2.3 An Isothermal Heat Exchanger

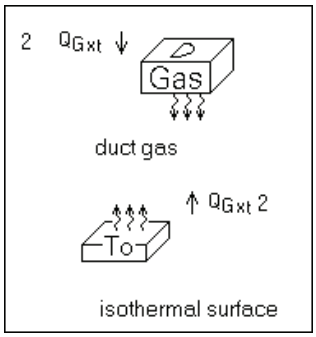
This example corresponds to sample file hxiso.stl

- From the component palette of the Sage form, drag an appropriate heat-exchanger component into the edit form, in this case a tube bundle. The

connector arrows will appear later.



- Double-click on the tube-bundle to open its child-component edit-window then drop in an isothermal surface from the *Duct Walls* page of the palette and a duct gas from the *Gas Domain* page. Establish a heat-flow connection between the gas and solid by dragging between connector arrows. The isothermal-surface temperature distribution becomes that specified for the Tinit input variable of the tube bundle.

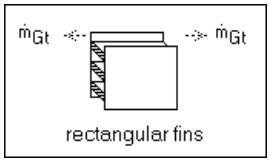


- Double-click on the duct gas, then drop in a negative and positive gas inlet from the palette. Move the $\dot{m}_{Gt}$ connector arrows up to the root-component and your heat exchanger is now ready for connection to other model elements.

2.4 An Internally-Finned Heat Exchanger

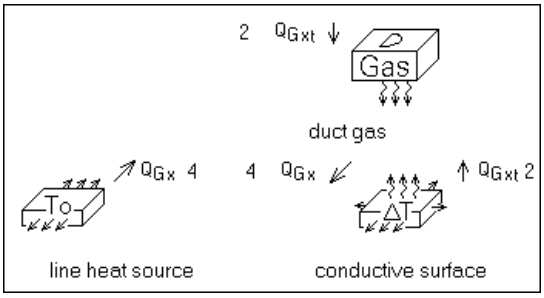
This example corresponds to sample file hxfin.stl

- From the component palette of the Sage form, drag a rectangular fins heat-exchanger into the edit form. The connector arrows will appear later.



- Double-click on the rectangular fins to open its child-component edit-window then drop in a conductive surface and line heat source from the

Duct Walls page of the palette and a duct gas from the *Gas Domain* page. Similarly, within the conductive surface and line heat source, drop in negative and positive heat flow faces from the palette and move the Q_{Gx} connector arrows up to the rectangular-fins level. Establish the heat-flow connections as indicated. Conductive surface input variable D is the fin conduction length and the fin base temperature is anchored to that of the line-heat-source which in turn comes from the Tinit input variable of the rectangular fins.

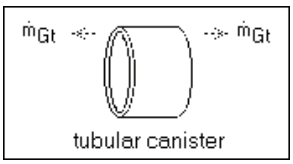


- Double-click on the duct gas, then drop in a negative and positive gas inlet from the palette. Move the $\dot{m}_{Gt}$ connector arrows up to the root-component and your heat exchanger is now ready for connection to other model elements.

2.5 An Isothermal Matrix Heat Exchanger

Heat exchangers can also be made from canister components. Many variations are possible. The simplest is this example where the matrix surface is presumed isothermal. *This example corresponds to sample file hxpori.stl*

- From the component palette of the Sage form, drag an appropriate canister into the edit form, in this case a tubular canister. The connector arrows will appear later.

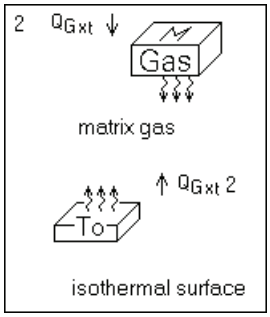


- Double-click on the canister to open its child-component edit-window then drop in an appropriate matrix from the *Matrices* page of the palette, in

this case a random-fiber matrix.



- Double-click on the random-fiber matrix to open its child-component edit-window then drop in an isothermal surface from the *Matrix Solids* page of the palette and a matrix gas from the *Gas Domain* page. Establish a heat-flow connection between the gas and solid by dragging between connector arrows. The isothermal-surface temperature distribution becomes that specified for the Tinit input variable of the canister.



- Double-click on the duct gas, then drop in a negative and positive gas inlet from the palette. Move the $\dot{m}_{Gt}$ connector arrows up to the root-component and your heat exchanger is now ready for connection to other model elements.

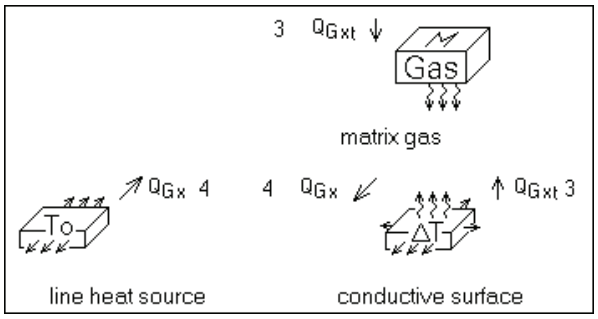
2.6 A Conductive Matrix Heat Exchanger

Another variation of a porous-matrix heat exchanger is where the matrix acts as a conduction path to the outer perimeter, which is held isothermal. *This example corresponds to sample file hxporc.stl*

- Proceed as for the first two steps of the previous example.

- In step three, instead of an isothermal surface within the random-fiber matrix drop a conductive surface and line heat source from the *Matrix Solids* page of the palette. Within the conductive surface and line heat source, drop in negative and positive heat flow faces from the palette and move the Q_{Gx} connector arrows up to the random-fiber matrix level. Establish the heat-flow connections as indicated. Conductive surface input variable

D is the effective matrix conduction length and the matrix perimeter temperature is anchored to that of the line-heat-source which in turn comes from the Tinit input variable of the matrix.



The effective matrix conduction length is just 1/4 the canister diameter for a cylindrical canister. Why? The exact solution governing temperature drop for radial heat flow in a porous circular cylinder with a uniformly distributed source or sink is:

$$Q = 8\pi(1-\beta)kL\Delta T \tag{2.1}$$

where Q is the total heat flow passing through the outer boundary, $(1-\beta)k$ is the effective solid conductivity and L is cylinder length (*see* [24], example 2-3, p. 35). The effective conductivity is written as the product of the solid fraction $1-\beta$ (β is porosity) and non-porous conductivity k in a crude attempt to account for the void part of the solid, which is presumed insulating. ΔT is the outer-boundary temperature minus the cross-section average temperature.

For the conductive-surface model, on the other hand, (*see* section 7.6.3) the y directed temperature drop is governed by

$$Q = k\frac{(A_s/D)}{D/2}L\Delta T \tag{2.2}$$

where factor A_s/D is the solid z-thickness and $D/2$ is the y-directed conduction length. Equating the two Q's and canceling obvious factors gives

$$D^2 = \frac{A_s}{4\pi(1-\beta)} \tag{2.3}$$

But $A_s/(1-\beta)$ is just the total canister cross section area $\pi D_c^2/4$. Making this substitution gives

$$D = D_c/4 \tag{2.4}$$

So all one has to do is set the effective conduction length D to 1/4 the canister diameter and the y-directed temperature drop in the conductive surface will be a good approximation to the radial temperature drop in the porous solid.

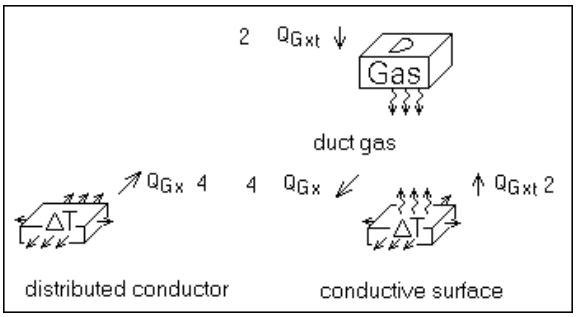
2.7 An External Conduction Path or Busbar

In all the previous examples, the heat exchanger was anchored to an isothermal temperature distribution modeled by an isothermal surface or line heat source. Sometimes it is desirable to replace the isothermal component with a solid conduction path to an external temperature source or sink at the root-component level. This conduction path is called a thermal *busbar* and might be a physical part of the model, such as the canister wall in a porous-matrix heat exchanger, or a fictitious component introduced solely for convenience. Why convenient? Because it can simplify energy-flow accounting by forcing all heat flows to terminate in a point temperature source at the root level and it is possible to optimize or map that temperature through a single variable.

The idea is illustrated here as a modification to the above internally-finned heat-exchanger example. *This example corresponds to sample file busbar.stl*

- Proceed as for the first step of internally-finned heat-exchanger example.

- In step two, instead of the line heat source within the rectangular fins, drop in a distributed conductor from the *Duct Walls* page of the palette. Within the distributed conductor, drop in a negative (or positive) heat-flow end and a positive heat flow face from the palette. Move the Q_{stdy} connector arrow from the heat-flow end up to the root-component level for connection there to a point temperature source or sink. Move the Q_{Gx} connector arrow from the heat-flow face up to the random-fiber matrix level and establish the heat-flow connection to the conductive surface as indicated.



- Input variable W of the distributed conductor should be set to the perimeter in contact with the heat exchanger — the heat-exchanger circumference in a cylindrical arrangement. Input variable D is the busbar thickness — the radial depth in a cylindrical arrangement. Input variable Solid is the busbar material. Copper gives the least temperature drop. Heat-flow in the above example is two-dimensional changing from the y-direction at the conductive-surface interface to the x-direction along the busbar to the ultimate source or sink.

- **Optional**: You could also replace the heat-flow end of the distributed conductor with a negative heat flow face for connection to an isothermal line heat source within the rectangular fins. This would model predominantly y-directed (one dimensional) heat flow within the distributed conductor. The line heat source has to reside within the rectangular fins because the fin length is not defined at the root-component level.

Tip: In the case of a porous-matrix heat exchanger, the distributed conductor in the *Heat Flows* page of the canister palette, which normally models wall conduction, can be used as the conduction path to the external point temperature source.

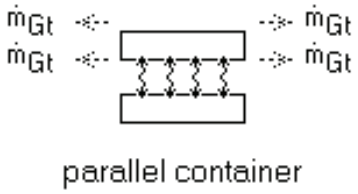
2.8 Interconnected Heat-Exchanger Passages

The previous heat exchanger examples all involved a single gas domain interacting with an isothermal surface or one or more thermal solids. The following examples show how to thermally connect two gas domains together using intermediary thermal solids.

2.8.1 Same Length Ducts

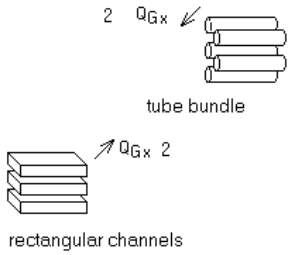
This example corresponds to sample file ParallelContainerHX.stl

- From the component palette of the Sage form, drag a parallel container into the edit form. The connector arrows will appear later. The parallel container is documented in chapter 13.

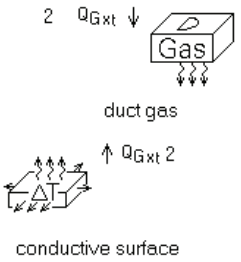


- Double-click on the parallel container to open its child-component edit-window then drop in two heat exchanger components, in this case a tube bundle and rectangular channels. The tube bundle might represent a central tube and the rectangular channels a surrounding annular channel.

They will be inter-connected later.



- Within each of these drop in a gas domain and conductive surface and establish a connection between the two by dragging between connector arrows.



- Within each of the gas domains drop in gas inlets, as needed, and promote the connector arrows three levels to the parallel-container level. Within the conductive surfaces drop in opposite heat-flow faces and promote the connector arrows two levels (the first common level) and connect them together, as shown two pictures above. This connection couples the two heat exchanger walls so that heat flows between the two gas domains. The conductive surface filters out any AC component of heat transfer so that only the DC component is transmitted (*see* section 7.6.3). Your parallel-flow heat exchanger is now ready for connection to other model elements by making gas-flow connections at the parallel container level.

2.8.2 Different Length Ducts

This example corresponds to sample file MLContainerHX.stl

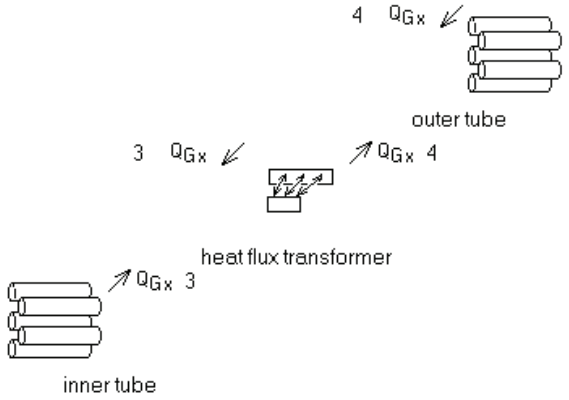
This example is much like the previous example except that a heat-flux transformer component connects together different-length heat exchangers within a special multi-length container component. (*see* section 13.2.2). A heat-flux transformer knows nothing about the detailed geometry of your heat exchanger. It only transfers the DC component of heat flow uniformly between corresponding positions of the two exchangers.

- From the component palette of the Sage form, drag a multi-length container into the edit form. The connector arrows will appear later.



Multi-length container

- Double-click on the multi-length container to open its child-component edit-window then drop in two heat exchanger components, in this case two tube bundles, one representing a straight central tube and the other a longer outer tube spiral-wrapped around the outside. The lengths are independent inputs for each tube bundle. Also drop in a heat-flux transformer from the basic page of the component palette. The heat flux transformer is born with two Q_{Gx} heat-connection arrows. The arrows attached to the two tubes will be promoted from conductive surfaces within.

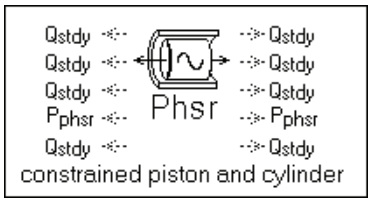


- Within each of the tube bundles drop in a gas domain and conductive surface with gas inlets and heat-flow faces as in the previous example. Promote the heat-flux connection arrows to the same level as the heat-flux transformer and connect them together, as shown above. Promote the gas-flow connection arrows up to the multi-length heat exchanger level for subsequent connection to other model elements.

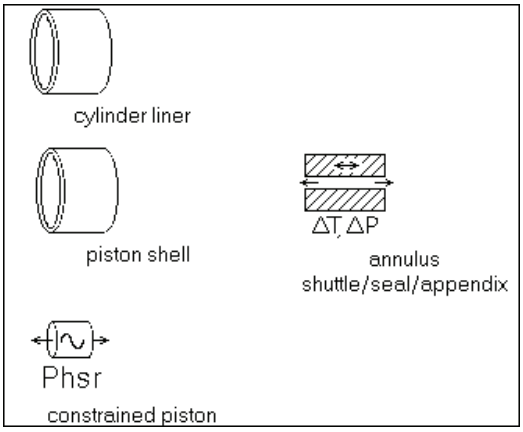
2.9 A Displacer and Cylinder

A stand alone displacer can be modeled with one of the piston components in the *Moving Parts* page of the component palette. But to model the combination of a displacer (or piston) within a cylinder, it is best to use one of the components on the *Composite* page. *This example corresponds to sample file discyl.stl*

- From the *Composite* page in the component palette of the Sage form, drop an appropriate piston/cylinder component into the edit form, in this case a constrained piston and cylinder. The connector arrows will appear later.



- Double-click on the piston and cylinder to open its child-component edit-window, revealing three built-in child components: the cylinder liner, piston shell and constrained piston. From the *Inter-Gap* page of the palette, drop in an optional annulus whose purpose in this case is to model shuttle heat transfer between the liner and shell. Move the annulus Q_{stdy} connector arrows up to the root-component level for connection there to point temperature sources.



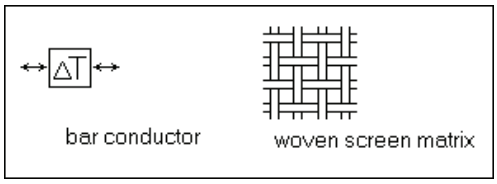
- Within the cylinder liner (double-click), drop in a bar conductor from the *Heat Flows* page of the component palette and move the Q_{stdy} connector arrows up to the root-component level for connection there to point temperature sources. The bar conductor models cylinder wall conduction.
- Similarly, within the piston shell, drop in a bar conductor and radiation transport path and move the Q_{stdy} connector arrows up to the root-component level. The bar conductor models displacer shell conduction and the radiation transport path models interior radiation transport.
- Within the constrained piston drop in positive and negative facing area attachments and move the P_{phsr} connector arrows up to the root-component level for connection there to variable-volume gas domains, such as the expansion and compression spaces.

- **Optional**: Within the constrained piston drop in positive and negative facing force attachments for connection to other moving parts and move the F_{phsr} connector arrows up to the appropriate level.

2.10 A Regenerative Displacer and Cylinder

In stirling refrigerators, the displacer often houses the regenerator. *This example corresponds to sample file disreg.stl*

- Proceed as for the previous displacer and cylinder example, completing all the steps.
- Within the piston shell, delete the radiation transport path (scissors tool) and drop in a matrix from the *Matrices* page of the palette, in this case a woven screen matrix.



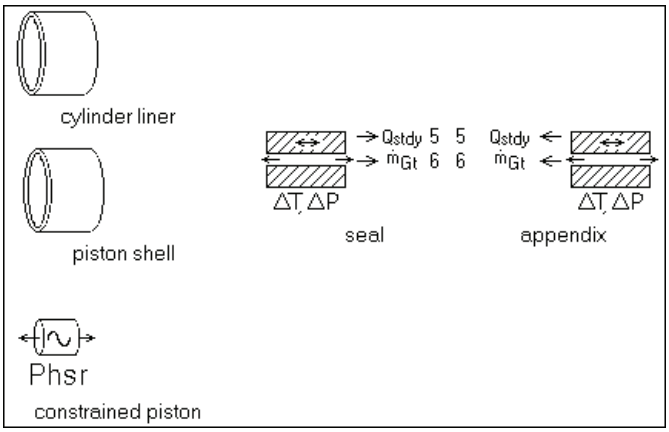
- At this point proceed as for the previous regenerator example, except that the regenerator is now contained in the piston-shell canister instead of a stand-alone canister at the root-component level.
- Set the displacer area attachments based on the frontal areas of the displacer shell the same as though the displacer were impervious. Effectively the regenerator matrix is modeled as a parallel gas-flow path even though it is physically within the displacer shell.

2.11 Displacer Appendix and Seal Leakage

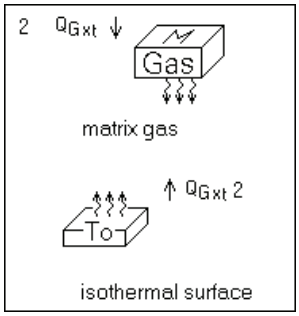
The annular region between the displacer shell and cylinder liner is often worth modeling. *This example corresponds to sample file disleak.stl*

- Proceed as for either previous displacer and cylinder example, in this case the displacer-and-cylinder example.
- Within the displacer and cylinder, rename the existing annulus as "seal" and drop in a second annulus, renamed "appendix". The seal and appendix model the two parts of the displacer leak in series. The relative positions of the seal and appendix are defined by their XNeg and XPos input variables and their radial gaps by the Gap input variable. To correctly model shuttle heat transfer, connect the Q_{stdy} arrows between the

seal and appendix and move the arrows at the far ends up to the root-component level for connection there to temperature sources. The $\dot{m}_{Gt}$ arrows, representing gas flow, are connected in a later step.



- Within the seal, drop in an isothermal surface and a matrix gas and connect them. The isothermal surface anchors the gas temperature in the seal, usually a good assumption. Within the matrix gas drop in negative and positive gas inlets. Move the negative-facing $\dot{m}_{Gt}$ connector arrow up to the root-component level for connection there to another gas domain, usually the compression space. Move the positive-facing $\dot{m}_{Gt}$ connector arrow up to the piston and cylinder level for connection to the appendix gas.

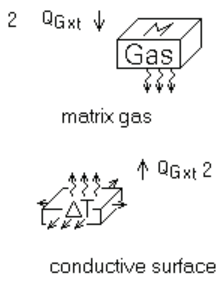


- Within the appendix, drop in a conductive surface and a matrix gas and connect them. The conductive surface adjusts to the gas temperature in the appendix, preventing it acting as an effective heat exchanger wall. Within the matrix gas, drop in negative and positive gas inlets. Move the positive-facing $\dot{m}_{Gt}$ connector arrow up to the root-component level for connection there to another gas domain, usually the expansion space. Move the negative-facing $\dot{m}_{Gt}$ connector arrow up to the piston and cylin-

der level for connection to the seal gas and make the connection by dragging with the mouse.

The conductive surface within the appendix inherits solid material and cross-sectional area information from the cylinder liner and piston shell. So you can and should use it to model the axial conduction in the appendix liner and shell. To do this just drop negative and positive heat-flow ends into the distributed conductor and move their Q_{stdy} arrows up to the root component level for connection there to temperature sources. The conduction so modeled corresponds to that in the combined cylinder liner and piston shell over that portion of the length specified by the XNeg and XPos input variables.

NOTE: When modeling appendix wall conduction this way do not also use bar conductors within the cylinder liner or piston shell to do the same thing. You do not want to model the same conduction paths twice.



Tip: Use at least 2 cells (NCell) in the seal component lest frictional pressure drop not be modeled.

Chapter 3

Boundary Connections

Stirling model components communicate with each other using the following boundary connections. As usual you may only connect together like connectors of opposite sign (opposite-facing arrows).

3.1 Force Connections

F_{phsr} or F_{Gt}

These represent either phasor or time ring forces acting on points of attachment. The points of attachment will share the same motion when connected together. Force connections are used primarily for connecting springs and dampers to moving parts, as in the Gizmo (resonant system) model class documented in the Sage User's Guide.

3.2 Pressure Connections

P_{phsr} or P_{Gt}

These represent either phasor or time ring pressure variations acting on area faces. The area faces share the same volume displacement when connected together. They are used primarily for connecting pistons and the like to gas domains.

3.3 Heat Flow Connections

Q_{stdy}, Q_{Gx} or Q_{Gxt}

These represent either steady, spatial grid, or space-time grid heat flows acting on thermal boundaries. Boundaries share the same temperature when connected together. Steady heat flows are useful for time-averaged parasitic conduction losses. Spatial grid heat flows are useful for steady but distributed heat flows,

such as occurs two dimensional fins. Space-time heat flows are for connecting thermal solids to gas domains.

3.4 Gas Flow Connections

$\dot{m}_{\mathsf{Gt}}$

These represent the flow of gas from one gas domain inlet into another. Two inlets conserve mass flow rate, energy and momentum when connected together.

3.5 Density Connections

ρ_{stdy}

This represents the common mass density between a gas domain and a pressure reservoir. The two share the same mean pressure when connected together. Density connections are used to connect the stirling working gas to a fixed-pressure source in order to establish charge pressure. This connection type is generally used only once per stirling model instance, but its use is critically important.

Chapter 4

Entropy Generation

Many stirling model components involve irreversible entropy-generating processes and keep tabs of entropy generation in terms available energy output variables. Available energy or availability for short is just entropy multiplied by the negative of ambient temperature T_0, which is really just input Tnorm in the stirling root model component. A loss in availability equates to a decrease (in an engine) or increase (in a cooler) of PV power.

4.1 Lost Available Energy

The concept of availability arises from the net mechanical work W_r available from a reversible heat engine with a net inflow of heat Q (positive) at a temperature T and outflow Q_0 (negative) at a sink temperature T_0:

$$W_r = Q + Q_0 = Q(1 - T_0/T) \tag{4.1}$$

In contrast, an irreversible-cycle might produce work $W_i < W_r$ obtained from the same input Q. According to the first law of thermodynamics, and in terms of heat rejected Q_0^*

$$W_i = Q + Q_0^* \tag{4.2}$$

where Q_0^* is no longer equal to $-QT_0/T$. The lost available energy is just

$$W_r - W_i = -QT_0/T - Q_0^* = -T_0(Q/T + Q_0^*/T_0) \tag{4.3}$$

Evidently, this is just $-T_0$ times the net entropy increase of the surrounding universe as a result of the heat flows into and out-of the system. This leads us to generally characterize all internal entropy generations in the stirling model class in terms of losses in *availability* according to the definition

$$\text{Availability Loss} = -T_0 \times \text{Entropy Generation} \tag{4.4}$$

The value of the availability-loss concept is that it allows us to think about entropy generation in terms of the more concrete notion of lost mechanical work.

It is important to note that availability losses vary with the sink temperature T_0, which is given by input variable Tnorm in the stirling root model component. Therefore, availability losses really equate to lost PV powers only when all points of heat rejection are indeed at the temperature Tnorm. See reference [4] for a standard treatment of available-energy accounting.

4.2 Second-Law Balance

Entropy generation (or available energy loss) may be measured in one of two ways: by entropy flow across the external boundaries of a stirling model instance or by entropy generated by internal processes. The previous example accounts for entropy generation in the first way. The surrounding universe suffers a loss of available energy due to heat flow through the boundaries of a stirling machine. When we get specific about individual stirling model components we will also be able to account for entropy generation in the second way — by tallying up the individual entropy generations in all internal processes.

In principle, the two methods of accounting should give the same answer. But often they do not. This is because there is no conservation-of-entropy-generation principle built into the stirling model components, as there is for conservation of energy. Discrepancies arise, usually attributable to numerical artifacts in the computational solution. These include finite-differencing truncation errors, for both spatial and temporal partial derivatives, as well as interpolation errors needed to obtain solution variables at off-solution grid points. So model components that tally available energy losses generally have an available-energy discrepancy output, as well, that monitors the difference between external-boundary and internal-process accounting.

It is tempting to look at the relative magnitude of the availability accounting discrepancy as a measure of the solution accuracy. Large discrepancies may be produced by difficult to model features such as heat exchangers with highly nonlinear axial temperature distributions, large pressure ratios, etc. In such cases it is usually helpful to refine the computational grid — that is, add more time nodes (NTnode) or more control volumes (NCell).

Chapter 5

Stirling Root Component

The highest level stirling model component, and the only model component you see when you create a new model instance, is the root model. Its purpose is to define a few global variables and some normalization constants as a framework for all its children which you create, or someone created before, by selecting from the component palette. Its variables are:

NTnode : (dimensionless) The number of time nodes in the computational grids of all underlying model components having such grids. Changing this innocent variable causes these computational grids to re-initialize themselves. In most cases this is almost as drastic as re-initializing the whole solution. Aside from that, NTnode is important because it dramatically affects the solution time and memory requirements, not to mention solution accuracy. Treat it gingerly by making only gradual, small changes.

Lnorm : (real, m) Length scale normalization constant. The value of this input may make the difference between a solution that converges and one that doesn't. As a rule of thumb, Lnorm should be, roughly, the cube root of the largest swept volume amplitude in the underlying stirling model. Some fiddling around may be required to achieve optimal solution convergence rate. Keep in mind that a small change may have a big effect, since many quantities are normalized by the cube of Lnorm.

FreqNorm : (real, Hz) Frequency scale normalization constant. This one is not as critical as Lnorm. Generally, set it to the actual operating frequency Freq. It is a separate constant because Freq may change during optimization.

Pnorm : (real, Pa) Pressure scale normalization constant. Again, not as critical as Lnorm, but important because it establishes the initial pressure in all underlying gas domains. It should be close to the charge pressure Pcharge in the pressure-source component. Otherwise, the pressure solutions in gas domains may go unstable.

Tnorm : (real, K) Temperature scale normalization constant. Usually the ambient temperature, or about 300 K. Keep in mind that it scales available energy outputs in certain model components (see chapter 4).

Qnorm : (real, W) Heat flow normalization constant. Generally, set it to the magnitude of the total heat rejection expected for the stirling model. Increasing Qnorm can sometimes help a solution converge when the convergence error is hovering near the target error and the problem is due to surface heat flux stiffness (small temperature fluctuation producing large heat flux fluctuation) as can happen, for example, with a particularly effective regenerator matrix.

Freq : (real, Hz) The actual operating frequency for the stirling machine. Although it is set as an input, it may be optimized to satisfy an objective function or constraints.

Omega : (real, radians/s) A convenience output that converts Freq to angular frequency by multiplying by 2π.

Gas : (enumerated) The working gas type (see below).

5.1 Working Gas

The gas variable is deceptively simple. It is an enumerated-type variable that selects the working gas from a list of choices. Each available gas is itself a software object with sufficient data encapsulated within it to know its important properties.

ideal If the gas is an ideal gas, these properties are:

T0 : (K) Representative temperature T_0.

Rgas : (J/(kg K)) Gas constant R.

specific heat : (J/(kg K)) A cubic spline variable with temperature vs zero-pressure specific heat $(T, c_p(T))$ data pairs covering a broad range of temperatures.

viscosity : (kg/(m s)) A cubic spline variable with temperature vs viscosity $(T, \mu(T))$ data pairs.

conductivity : (W/(m K)) A cubic spline variable with temperature vs conductivity $(T, k(T))$ data pairs.

Redlich-Kwong If it is a Redlich-Kwong gas then it has additional properties:

Tcrit : (K) Critical temperature T_c

Pcrit : (Pa) Critical pressure P_c

C1 : (dimensionless) Redlich-Kwong constant c_1

C2 : (dimensionless) Redlich-Kwong constant c_2

BVirialErr : (dimensionless) Overall RMS error $\tilde{B}$ in the second virial coefficient.

tabular If it is a tabular gas then a table of state values takes the place of Redlich-Kwong properties.

Z(v, T) : (dimensionless) a rectangular table of $Z(v_j, T_k)$ values where $Z = Pv/RT$ is compressibility, v is specific volume and T is temperature. The first (left) column contains the T_k values and the first (top) row the v_j values. The corresponding Z values lie in the body of the table.

The values of the individual properties, including cubic-spline interpolation pairs or state table, appear after the enumerated identifier in display windows and the output listing. The gas choices come from a default database file gas.dta, or a data base file customized by you. (*see* chapter 14)

An instance of a gas object has methods (subroutines) that can be called upon to return various properties, as functions of temperature, or various state variables, as functions of other state variables. The underlying gas-domain model components call upon these methods when needed. Sometimes, this requires only cubic-spline interpolation using the appropriate set of data pairs. Sometimes, it gets a little more complicated.

5.1.1 Conductivity $k(T)$ and Viscosity $\mu(T)$

All gas objects calculate conductivity and viscosity from cubic-spline interpolation as a function of T, without any pressure dependence. In most cases, ignoring pressure dependence is reasonable. Even when it is not reasonable, it will cause no great harm because k and μ are only used for calculating such things as Reynolds, Valensi and Prandtl numbers which, in turn, are used for calculating viscous friction, surface heat transfer and axial gas conduction. And correlating expressions for such phenomena are not extremely accurate in the first place.

One way to achieve some pressure dependence would be to insert additional gases into a customized database file, each one a reasonably good approximation in some pressure range. For example, you might define gas named "Tabular Helium 5 MPa", valid only in the vicinity of 5 MPa, and so forth.

5.1.2 Ideal Gas Objects

As the name implies, ideal-gas objects presume the ideal gas equation of state

$$Pv = RT \tag{5.1}$$

Available methods are presented below in functional notation. For example, $c_p(T)$ is the heading for the method that returns c_p as a function of T.

Specific Heat Ratio γ

This is actually a data field, not a method, initialized as

$$\gamma = \frac{1}{1 - R/c_p} \tag{5.2}$$

where c_p is the zero-pressure limit at reference temperature T_0. This is the only reason for the existence of T_0 as a data field. It is important that γ be a constant, rather than a function of T, because internal energy and entropy both depend on γ ($c_v = R/(\gamma - 1)$) as does sonic velocity.

Specific Heat $c_p(T)$

Zero-pressure c_p limit, that is. This is just a matter of cubic-spline interpolation from the specific-heat data pairs.

Prandtl Number $P_r(T)$

Calculated as a function of temperature as $P_r = k/(\mu c_p)$.

Viscosity $\mu(T)$

Molecular viscosity interpolated from viscosity-temperature data pairs.

Conductivity $k(T)$

Molecular conductivity interpolated from conductivity-temperature data pairs.

Sonic Velocity $u_s(T)$

Returns the isentropic, small amplitude, phase velocity of an acoustical wave

$$u_s = \sqrt{\gamma RT} \tag{5.3}$$

Compressibility $Z(T, P)$

Returns compressibility, defined as

$$z = \frac{Pv}{RT} \tag{5.4}$$

which equals one, by definition, for an ideal gas.

Temperature $T(\rho, \rho e, u)$

Returns temperature as function of mass density, volume-specific energy density and velocity

$$T = \frac{\gamma - 1}{R}\left((\rho e)/\rho - u^2/2\right) \tag{5.5}$$

Pressure $P(\rho, T)$

Returns pressure as function of mass density and temperature

$$P = \rho RT \tag{5.6}$$

Energy Density $\rho e(\rho, T, u)$

Returns volume-specific energy density as function of mass density, temperature and velocity

$$\rho e = \rho \left(R/(\gamma - 1)T + u^2/2 \right) \tag{5.7}$$

where $R/(\gamma - 1) = c_v$.

Entropy $s(\rho, T)$

Returns mass-specific entropy as function of mass density and temperature

$$s = R \left(\ln(T)/(\gamma - 1) + \ln(1/\rho) \right) \tag{5.8}$$

Equation of State Error $\tilde{Z}(\rho, T)$

Returns zero because the ideal equation of state relative error is zero by assumption.

5.1.3 Redlich-Kwong Gas Objects

Redlich-Kwong gas objects employ the real-gas equation of state by the same name. They were originally intended for use in low-temperatures cryocoolers and high pressures engines where ideal gas properties no longer apply. But subsequent development of so-called tabular gas objects (tabulated state properties) has essentially eliminated the need for Redlich-Kwong gases. Tabular gas objects contain more accurate gas properties with negligible impact on computational performance. So the use of Redlich-Kwong gases is no longer recommended but they are still documented here for completeness.

The Requirements

The chemical engineering literature is rife with real-gas equations of state. The problem is picking the best one for the Stirling model class whose requirements are very much different from those of general chemical engineering practice. Stirling models require:

- Good accuracy in the gaseous phase above the critical temperature and over a wide range of pressures
- Lesser accuracy acceptable near the liquefaction region, where stirling machines are unlikely to operate

- Validity for helium and hydrogen which tend to be exceptions to the rule compared to most other simple gases
- Fast evaluation

The fast-evaluation requirement for a real-gas equation of state severely restricts the field. This is primarily because the relationship between internal energy and temperature is no longer the simple $\varepsilon = c_v T$, as it is for an ideal gas, but the more complicated formula

$$\varepsilon = \int_0^T c_v^0(T) dT - \int_v^\infty \left(T \left(\frac{\partial P}{\partial T} \right)_v - P \right) dv \tag{5.9}$$

This formula is a consequence of integration-path independence for changes in entropy and is derived in any comprehensive thermodynamics book, for example [43]. The integrand $c_v^0(T)$ is the zero-pressure limit of the constant-volume specific heat, which is generally a function of temperature. Even if we are willing to approximate $c_v^0(T)$ by a mean effective constant value $\bar{c}_v$, as we will do, we are still left with the equation

$$\varepsilon = \bar{c}_v T - \int_v^\infty \left(T \left(\frac{\partial P}{\partial T} \right)_v - P \right) dv \tag{5.10}$$

To get the needed inverse expression $T(\varepsilon, v)$ may or may not be possible in explicit form. An iterative solution is always possible, but this is a costly proposal when the calculation must be repeated thousands of times for a typical stirling model solution.

Selecting the Redlich-Kwong Equation

The so-called cubic equations of state (cubic in v) are the simplest equations which are capable of representing the liquid-like, vapor-like and ideal-gas-like regions of the state surface in P-v-T space. Of these, the one which seems most promising for stirling models, is the Redlich-Kwong equation [40].

$$P = \frac{RT}{v - b} - \frac{a}{T^{1/2} v (v + b)} \tag{5.11}$$

Parameter b pertains to the volume of the molecules and a to the forces of cohesion between them. They are typically given as

$$a = c_1 \frac{R^2 T_c^{2.5}}{P_c} \tag{5.12}$$

$$b = c_2 \frac{R T_c}{P_c} \tag{5.13}$$

where $c_1 = 0.42748$ and $c_2 = 0.08664$, which are derived by the condition that the critical-temperature isotherm in the P-v plane have an inflection point at P_c and v_c — namely, that at P_c and v_c, $\frac{\partial P}{\partial v} = 0$ and $\frac{\partial^2 P}{\partial v^2} = 0$.

The Redlich-Kwong equation enjoys an honored position in the chemical engineering literature because of its reasonable accuracy over a wide range of densities and temperatures. For simple gases like Ar, Kr, and Xe, Abbott [1] demonstrates the superiority of the Redlich-Kwong equation over some other cubic equations of state in tracking experimental temperature dependence at low density and specific-volume dependence along critical-temperature isotherms. To the extent that pressure dependence and temperature dependence are separable, one infers that the Redlich-Kwong equation is pretty good for all temperatures and pressures.

Low-density temperature dependence of an equation of state may be evaluated by first putting it in the standard virial form

$$\frac{Pv}{RT} = 1 + B(T)/v + C(T)/v^2 + \cdots \tag{5.14}$$

and then comparing the predicted second virial coefficient $B(T)$ against experimental data. The Redlich-Kwong equation can be put into virial form by using a Taylor power-series expansion in $1/v$ (density) about the point $1/v = 0$

$$\frac{Pv}{RT} = 1 + \left(b - \frac{a}{RT^{3/2}}\right)/v + \left(b^2 + \frac{ab}{RT^{3/2}}\right)/v^2 + \cdots \tag{5.15}$$

One infers from this that $B(T) = b - a/(RT^{3/2})$ for the Redlich-Kwong equation.

Tables of $B(T)$ tabulated for a large number of gases are found in [8]. The gases of most importance to stirling models are helium, hydrogen and nitrogen. Unfortunately, the Redlich-Kwong second virial coefficient agreement with helium and hydrogen leaves much to be desired, although agreement with nitrogen is quite good.

One can attempt to correct for this disagreement by using data modeling techniques to pick the numerical coefficients c_1 and c_2 in the expressions for a and b to best fit the tabulated data. The default gas-property data base file incorporates best-fit values for c_1's and c_2's (see chapter 14).

The overall RMS error in second virial coefficient B, over the range of samples considered, is the data field $\tilde{B}$ in the gas object. It may be used as a rough measure of the inaccuracy of the Redlich-Kwong equation for a particular gas in question. Specifically, for low to moderate densities, the error $\tilde{Z}$ in compressibility is just

$$\tilde{Z} \approx \tilde{B}/v \tag{5.16}$$

This result follows directly from the virial expansion (5.14). Note that the preceding error estimate ignores errors in third and higher virial coefficients which become important at high densities. However for our purposes, it will do just fine.

The property methods for Redlich-Kwong gases are the same as for ideal gases. But The state-variable methods and the equation of state error are quite different.

Compressibility $Z(T,P)$

Compressibility is now $Pv/(RT)$ based on algebraic solution of cubic Redlich-Kwong equation of state. The crucial part is solving specific volume v as a function of T and P.

Computing $v(T,P)$ is not always as easy as it might sound. You can appreciate why by imagining isotherms on a P-v plot. For $T > T_c$ or $P > P_c$, the isotherms are nearly hyperbolas with a single value for $v(T,P)$. No problem. But if $T < T_c$ and $P < P_c$ one may be in the liquefaction zone where real-life $\frac{\partial P}{\partial v} = 0$ along an isotherm — in other words, where there is no unique solution to $v(P,T)$. Redlich-Kwong isotherms, as those of all cubic equations of state, are not flat in the liquefaction zone, but they do wiggle in such a way that there are three real solutions to $v(P,T)$. Furthermore, the Redlich-Kwong equation suffers from physically meaningless negative-v solutions at high pressures. When solving $v(T,P)$, one must always be aware of these possibilities. My strategy for dealing with multiple solutions is to always take the largest one. In the liquefaction zone, this corresponds to the most gas-like state. When there are meaningless solutions present, the largest one is always meaningful with $v > b$.

Because of the possibility of multiple solutions, and the need to distinguish the largest one from the others, I prefer an algebraic solution to $v(T,P)$ over an iterative one. The Redlich-Kwong equation (5.11) can be put into standard cubic form by multiplying through by $v(v-b)(v+b)$. Collecting terms and introducing $v_0 = RT/P$ one obtains

$$v^3 - v_0 v^2 + \left(\frac{a}{PT^{1/2}} - b^2 - v_0 b\right) v - \frac{ab}{PT^{1/2}} = 0 \tag{5.17}$$

Although the above cubic equation can be solved for v, there is some advantage to re-writing it with dimensionless compressibility $z = v/v_0$ as the independent variable and solving directly for z. This is easily done by multiplying through by $1/v_0^3$:

$$z^3 - z^2 + \left(\frac{a}{PT^{1/2}v_0^2} - \frac{b^2}{v_0^2} - \frac{b}{v_0}\right) z - \frac{ab}{PT^{1/2}v_0^3} = 0 \tag{5.18}$$

Again, from [39], the solution proceeds in terms of

$$Q = \frac{1 - 3a_2}{9} \tag{5.19}$$

$$S = \frac{-2 + 9a_2 - \frac{27ab}{PT^{1/2}v_0^3}}{54} \tag{5.20}$$

where

$$a_2 = \frac{a}{PT^{1/2}v_0^2} - \frac{b^2}{v_0^2} - \frac{b}{v_0} \tag{5.21}$$

If $S^2 - Q^3 > 0$ then there is a single solution given by

$$z = -\mathrm{sgn}(S)\left((\sqrt{S^2 - Q^3} + |S|)^{1/3} + \frac{Q}{(\sqrt{S^2 - Q^3} + |S|)^{1/3}}\right) + 1/3 \tag{5.22}$$

Otherwise, there are three roots, namely:

$$z = -2\sqrt{Q}\cos(\theta/3 + 2n\pi/3) + 1/3 \quad (5.23)$$

$$\theta = \arccos(S/\sqrt{Q^3}) \quad (5.24)$$

where $n = 0$, 1, or 2. Since θ is between 0 and π, it follows that $\theta/3$ is between 0 and $\pi/3$ and the largest solution we are seeking occurs for $n = 1$.

Temperature $T(\rho, \rho e, u)$

Unfortunately, the internal energy formulation for the Redlich-Kwong equation is rather expensive to invert for the temperature. We first note that mass specific internal energy is always

$$\varepsilon = (\rho e)/\rho - u^2/2 \quad (5.25)$$

Then, evaluating energy equation (5.10) produces

$$\varepsilon = \bar{c}_v T - \frac{3}{2}\frac{a}{b}\frac{\ln(1 + b/v)}{T^{1/2}} \quad (5.26)$$

which can be solved for T either analytically or iteratively, by Newton's method. To solve it analytically one can multiply through by $T^{1/2}$ which results in a cubic equation in $T^{1/2}$. The general solution of a cubic equation involves two quantities Q and S (see [39]), which in the present case amount to $Q = \varepsilon/(3\bar{c}_v)$ and $S = -3a\ln(1 + b/v)/(4b)$. if $Q^3 - S^2 \geq 0$ then the equation has three real roots, otherwise it has only one. In the present case, $Q^3 - R^2$ may be either positive or negative. When it is negative the solution involves evaluating the cosine function and its inverse. This is a rather expensive proposition to repeat thousands of times per stirling solution. Alternately, to employ Newton's method, one can define the function

$$F(T) = \varepsilon - \bar{c}_v T + \frac{3}{2}\frac{a}{b}\frac{\ln(1 + b/v)}{T^{1/2}} \quad (5.27)$$

which takes on the value zero at the required solution. So long as $\varepsilon > 0$, the sequence starting with $T_0 = \varepsilon/\bar{c}_v$ and with general term

$$T_{n+1} = T_n - F(T_n)/F'(T_n) \quad (5.28)$$

is guaranteed to converge to the solution. This follows from the strictly-decreasing concave-up nature of $F(T)$, and the fact that the T_n sequence is strictly increasing. Although this iterative process might seem costly, it is less so than the algebraic solution.

The above iterative process can also be fixed up to work with the case $\varepsilon \leq 0$, however, negative internal energy only occurs with $T < T_c$ near the liquid state, where cohesive forces dominate. Experiments indicate that stirling models dramatically fail to converge with negative-energy states, so it is probably safe to exclude this option from consideration.

Pressure $P(\rho, T)$

Pressure is a function of mass density and temperature directly from the Redlich-Kwong equation of state (5.11).

Energy Density $\rho e(\rho, T, u)$

Volume-specific energy density is function of mass density, temperature and velocity by evaluating mass-specific internal energy ε from equation (5.26), then using

$$\rho e = \rho \left(\varepsilon + u^2/2 \right) \tag{5.29}$$

Entropy $s(\rho, T)$

Gas entropy is of secondary importance in the stirling model components. It is never explicitly required during the solution stage. However, it is useful for certain second-law outputs.

Mass-specific entropy s is generally defined, to within an integration constant, by the equation

$$ds = \frac{d\varepsilon}{T} + \frac{P dv}{T} \tag{5.30}$$

Since s is a property of state, its value may be calculated at any point (v, T), with respect to a reference value $s_0 = s(v_0, T_0)$, by integrating this equation along any path in the v-T plane connecting (v_0, T_0) to (v, T).

For the Redlich-Kwong equation of state, it is necessary to take a somewhat complicated integration path consisting of an isochore from (v_0, T_0) to (v_0, ∞), followed by an infinite-temperature *isotherm* to (v, ∞), followed by another isochore to (v, T). The use of ∞ in the previous sentence is understood as the limiting case of some intermediate third temperature as it approaches infinity. The net result of this is the formula

$$s - s_0 = \int_{T_0}^{\infty} \left(\frac{\frac{\partial \varepsilon}{\partial T}}{T} \right)_{v_0} dT + \int_{v_0}^{v} \left(\frac{P}{T} \right)_{T=\infty} dv + \int_{\infty}^{T} \left(\frac{\frac{\partial \varepsilon}{\partial T}}{T} \right)_{v} dT \tag{5.31}$$

The above equation uses the fact that $d\varepsilon = \frac{\partial \varepsilon}{\partial T}\, dT$ along an isochore. For the Redlich-Kwong equation, the integrands in the above integrals work out to

$$\frac{\partial \varepsilon}{\partial T}/T = \frac{\bar{c}_v}{T} + \frac{3}{4}\frac{a}{b}\frac{\ln(1 + b/v)}{T^{5/2}} \tag{5.32}$$

$$\frac{P}{T} = \frac{R}{v - b} - \frac{a}{T^{3/2} v (v + b)} \tag{5.33}$$

Carrying out the integrations and simplifying produces the final entropy formula

$$s = c_v \ln T + R \ln(v - b) - \frac{1}{2}\frac{a}{b}\frac{\ln(1 + b/v)}{T^{3/2}} \tag{5.34}$$

Equation of State Error $\tilde{Z}(\rho, T)$

This is just $\tilde{B}\rho$ according to the virial equation of state.

5.1.4 Tabular Gas Objects

Tabular gas objects abandon the idea of an algebraic equation of state in favor of an interpolation approach. As a result, tabular gases are more accurate and versatile than Redlich-Kwong gases.

Creating a *good* table of compressibility values $Z(v_j, T_k)$ may be somewhat delicate and time consuming. Chapter 14 covers this issue in some detail. Of course, once a data table exists, it is easy to use.

Computational Issues

The essential idea is to interpolate the equation of state $P(v, T)$ from a table of known values, using bicubic spline interpolation (*see* section 3.6 of [39]). However, the values $P(v_j, T_k)$ are likely to vary over a large range, which might lead to data input errors and interpolation errors. So instead of tabulating P directly, it is better to tabulate the compressibility $Z = Pv/(RT)$, which has the value one in the ideal-gas range. It is easy enough to recover $P(v_j, T_k)$ from $Z(v_j, T_k)$.

It is also possible to recover volume-specific internal energy $\varepsilon(v, T)$ and mass-specific entropy $s(v, T)$ from the state table $Z(v_j, T_k)$, although the calculations involve computationally expensive path integrations. So it is best to do these calculations only once and then store $\varepsilon(v, T)$ and $s(v, T)$ as derived interpolation tables. But how does one calculate $\varepsilon(v, T)$ and $s(v, T)$?

Internal Energy According to reference [43] (equation (232), p. 285), the differential of mass-specific internal energy is

$$d\varepsilon = c_v dT + \left[T \left(\frac{\partial P}{\partial T} \right)_v - P \right] dv \tag{5.35}$$

In terms of compressibility Z, one can substitute $R\rho TZ$ for P and $R\rho \left[Z + T \left(\frac{\partial Z}{\partial T} \right)_v \right]$ for $\left(\frac{\partial P}{\partial T} \right)_v$ so that the internal energy differential becomes

$$d\varepsilon = c_v dT + \left[R\rho T^2 \left(\frac{\partial Z}{\partial T} \right)_v \right] dv \tag{5.36}$$

Then one can build a table of ε values as follows:

First build the v_{max} row of the table by temporarily assigning the corner point $\varepsilon(v_{max}, T_{min})$ the value zero, then calculating the remaining points in the row by integrating (5.36) with respect to T only, from one point to the next. Assuming ideal-gas conditions at v_{max}, it is valid to substitute the zero-pressure limit $c_{v0}(T)$ for $c_v(T)$, then $c_{p_0}(T) - R$ for $c_{v0}(T)$, since $c_{v0}(T)$ is not directly

tabulated. $c_{p_0}(T)$, on the other hand (the zero-pressure limit of c_p), is available as a cubic-spline data object.

Then, in a similar point-by-point process, complete the columns of the table by integrating equation (5.36) with respect to v only, starting at each of the (v_{max}, T_k) points already calculated and using bicubic-spline routines operating on the Z table to calculate the value of $\left(\frac{\partial Z}{\partial T}\right)_v$.

The process so far does not establish an appropriate constant of integration. Usually this is done to give a particular reference value for internal energy at some state point (v_0, T_0). In principle, this is not a problem since the equations in Sage's gas domains are affected only by changes in internal energy, not absolute values. On the other hand, some built-in solution convergence safeguards prevent internal energy values from going negative, which might be the case in the low T and low v portion of the table. So, to avoid this, an integration-constant offset is added to the entire table so that the minimum energy value is at least $c_{v0}T_{min}$.

Entropy The calculation of the entropy table proceeds along similar lines, except the differential of mass-specific entropy is

$$ds = \frac{d\varepsilon}{T} + \frac{P}{T}dv \tag{5.37}$$

again according to reference [43] (equation (103), p. 102). By substituting $\left(\frac{\partial \varepsilon}{\partial T}\right)_v dT + \left(\frac{\partial \varepsilon}{\partial v}\right)_T dv$ for $d\varepsilon$ this becomes

$$ds = c_v \frac{dT}{T} + \left[\left(\frac{\partial \varepsilon}{\partial v}\right)_T + P\right]\frac{dv}{T} \tag{5.38}$$

Then substituting $R\rho T^2 \left(\frac{\partial Z}{\partial T}\right)_v$ for $\left(\frac{\partial \varepsilon}{\partial v}\right)_T$, according to equation (5.36), and $R\rho Z$ for P/T, the differential for entropy becomes

$$ds = c_v \frac{dT}{T} + R\rho\left[Z + T\left(\frac{\partial Z}{\partial T}\right)_v\right]dv \tag{5.39}$$

As before, the v_{max} row of the table is produced by integrating equation (5.39) with respect to T only, point-by-point, starting with a temporary value of zero for $s(v_{max}, T_{min})$, and substituting $c_{p_0}(T) - R$ for $c_v(T)$. Then the columns of the table are completed by integrating equation (5.39) with respect to v only, starting at each of the (v_{max}, T_k) points just calculated.

As for the energy table, this approach may lead to negative values in the low T, low v corner of the table. This is not a problem for Sage gas domains, but more of a problem with human perception. So an integration-constant offset is added to the entire table so that the minimum entropy value is some convenient value, such as gas constant R.

Derived Properties All of the gas properties Sage requires can be calculated in terms of the above $Z(v,T)$, $\varepsilon(v,T)$ and $s(v,T)$ tables. The property methods

that perform these calculations are the same as those for the other gases, except the internal calculation details are entirely different. Interpolation replaces algebraic calculation and the specific volume v and mass-specific internal energy ε used in the state-table formulation are converted back and forth into Sage variables density ρ and volume-specific total energy ρe.

Compressibility $Z(T, P)$

For this method the independent variables are (T, P) instead of (T, v), so it is not possible to interpolate the result directly from the table of Z values. Instead, Sage finds the appropriate $v(T, P)$, then returns $Z = Pv(T, P)/(RT)$, according to its definition. Finding $v(T, P)$ is a matter of using Newton's method to iteratively solve

$$F(v) = P_{in} - P(v, T) = 0 \tag{5.40}$$

Where P_{in} is the given pressure input and $P(v, T)$ is the pressure interpolated from the table of Z values. Required in the process is $\frac{\partial P}{\partial v}$, which is supplied by bicubic spline routines operating on the Z table.

Newton's method requires a good starting value for v, otherwise it may not converge. This starting value comes from a binary search in the Z table to find the v-indices that bracket P_{in} (the indices j_{lo} and $j_{hi} = j_{lo} + 1$ so that $P(v_{j_{lo}}, T) \geq P_{in} \geq P(v_{j_{hi}}, T)$) followed by a linear interpolation between the bracketing values. The bracketing technique is reliable because P always decreases with increasing v for all gases. Pressure may remain constant during the liquid to vapor transition but it never increases.

Temperature $T(\rho, \rho e, u)$

Sage uses temperature in the solution process so this is a critical method. The first step is to convert volume-specific total energy ρe to mass-specific internal energy ε using

$$\varepsilon = (\rho e)/\rho - u^2/2 \tag{5.41}$$

then finding $T(\rho, \varepsilon)$ is once again a matter of using Newton's method, this time to iteratively solve

$$F(T) = \varepsilon_{in} - \varepsilon(1/\rho, T) \tag{5.42}$$

Where ε_{in} is the given energy input and $\varepsilon(1/\rho, T)$ is the energy interpolated from the table of ε values. Required in the process is $\frac{\partial \varepsilon}{\partial T}$, which is supplied by bicubic spline routines operating on the ε table.

The initial guess at T comes from a binary search in the ε table to find the T-indices that bracket ε_{in} (similar to above) followed by a linear interpolation between the bracketing values. The bracketing technique is reliable because ε always increases with increasing T for all gases.

Pressure $P(\rho, T)$

This is directly available by interpolation in the Z table as

$$P = Z(1/\rho, T)R\rho T \tag{5.43}$$

Energy Density $\rho e(\rho, T, u)$

This is directly available by interpolation in the ε table as

$$\rho e = \rho(\varepsilon(1/\rho, T) + u^2/2) \tag{5.44}$$

Entropy $s(\rho, T)$

This is directly available by interpolation in the s table.

Equation of State Error $\tilde{Z}(\rho, T)$

This could be based on some measure of error in the actual tabulated values of $Z(v_j, T_k)$ as well as some estimate of the cubic spline interpolation error. But it is not worth the bother since $\tilde{Z}$ is required only for an output variable in the Sage listing. So $\tilde{Z}$ is presumed zero until further notice.

Chapter 6

Moving Parts

Stirling-class moving parts are additions to the generic *resonant-system* model components already documented in the *Sage User's Guide*. Generally they are more physically specific yet remain connector compatible with their generic counterparts.

6.1 Simple Crank Piston



This model component is actually a descendant of the time-ring constrained piston where Fourier-series displacement is now calculated as a dependent variable in terms of geometrical input variables pertaining to a simple crankshaft and connecting-rod kinematic mechanism.

Mathematically, displacement is given by

$$x = r\left(\cos\theta + \sqrt{\cos^2\theta + L^2/r^2 - 1}\right) - L \tag{6.1}$$

where r is crank-throw radius, L is connecting-rod length and θ is crank angle, defined in terms of angular frequency ω and a fixed phase angle ρ as

$$\theta = \omega t + \rho \tag{6.2}$$

In Sage, the above variables become:

Rcrank : (real, m) Crank-throw radius r.

Lratio : (real, dimensionless) Connecting-rod length divided by crank-throw radius L/r.

Phase : (real, m) Crank-angle phase shift ρ.

Any or all of these may be optimized if need be.

6.2 Flexure Springs

These model components descend from the spring components documented under the resonant-system model class in the Sage manual. Except linear spring stiffness is now calculated as a dependent variable in terms of physical dimensions for spiral-arm flexure springs. And the mass of the spring is reflected in its attachment force. Displacement x is now solved from net applied boundary force F, stiffness K and effective mass m_e using the equation

$$F = Kx + m_e\ddot{x} \tag{6.3}$$

There are two phasor versions. The first version represents a stack of flexure elements located at one point. The second represents a stack of flexure elements separated into two parts by some axial distance, acting as a couple. There are no time-ring flexures.

The theory behind Sage spiral arm flexures is an extrapolated case of helical coil spring theory. It applies, more or less, to flexures having spiral arms of uniform width and thickness. Since the analysis is only approximate, a number of calibration constants are included in the model. The idea is that you can reset their default values, if you want, based on a separate finite-element analysis of your particular geometry.

6.2.1 Flexure Stack



This component is intended as a plug-compatible replacement for a generic phasor spring in a resonant-system model. You would use this component during an advanced stage of design where you wanted to actually design and optimize a spiral-arm flexure spring, rather than just work with its generic spring stiffness within a larger system. The flexure-stack component introduces new variables:

Ca, Cr, Cs, Cw, Cm : (real, dimensionless) Empirical calibration constants according to following theory.

E : (real, N/m^2) Elastic modulus E of spring material.

Rho : (real, N/m^2) Density ρ of spring material.

T : (real,m) Thickness t of a single flexure.

W : (real, m) Spiral arm width w.

D : (real, m) Active diameter D. Defined by circle where arms attach to outer rim.

M : (real, dimensionless) Number k of arms per flexure.

N : (real, dimensionless) Number n of flexures in stack.

Kr : (real, N/m) Radial stiffness K_r. Spring stiffness for deflection transverse to spring axis.

Sm : (real, N/m^2) Peak torsional stress τ_m.

Ws : (real, 1/s) Surge angular frequency ω_s. The operating frequency at or above which the analysis breaks down due to internal resonance.

Me : (real, kg) Effective mass M_e. That mass which moving with the same displacement amplitude as the flexure central attachment point produces the same reaction force.

Tn : (real, radians) Spiral arm total turning angle θ_T. The angle through which a radius vector rotates as it traverses a spiral arm from its inner to outer attachment points.

6.2.2 Flexure-Stack Couple



This component is a minor variation of the above flexure component (a descendant actually) that presumes the stack of flexures is divided into two groups spaced by some distance along the central axis, so as to produce a restoring moment to a tilting moment load. It, too, is a plug-compatible replacement for the generic phasor spring component. The angular stiffness within this component is intended for use in optimization constraints rather than some sort of transverse force connection to other model components. New variables introduced by the flexure-stack couple are:

Sc : (real, m) Couple separation distance S.

Ka : (real, N m / radian) Angular stiffness K_a.

You could optimize one of these subject to a constraint on the other.

Angular stiffness, defined as the restoring moment produced per unit tilt angle in radians, may be expressed in the form

$$K_a = \frac{1}{4} K_r S^2 \tag{6.4}$$

where K_r is the total radial stiffness of both groups of flexures combined and S is the distance separating the groups. A simple force diagram will convince you this is so.

6.2.3 Theory

The principle assumption is that spiral flexure arms behave at each point like the coils in a helical spring with the same radius of curvature. This is reasonable provided that (1) the arms actually turn through some minimum angle as they spiral out from the center (so as to produce load moments similar to those in helical springs), (2) they are longer than some minimum length (so that we may ignore the stresses due to slope and roll constraints at the end attachments) and (3) the radius of curvature does not change too abruptly.

Spiral Geometry

A spiral flexure may be geometrically characterized by four independent variables:

D	active diameter
t	thickness
w	arm width
k	number of arms

If we take $r(\theta)$ to be the radius (distance from center to spiral-arm neutral axis) as a function of turning angle, then the spiral arm shape is defined by the geometrical requirement that $\Delta r/\Delta\theta = k(1+\epsilon)w/2\pi$. In English: r increases by k arm widths for each turn. Factor $(1+\epsilon)$ accounts for the gap between arms and the fact that the radial distance between edge-bounding involutes progressively deviates from perpendicular arm width as r decreases. If $dr/d\theta$ is as above then arm shape $r(\theta)$ must be

$$r(\theta) = r_0 + \frac{k(1+\epsilon)w}{2\pi}\theta \tag{6.5}$$

where r_0 is the inner radius. The total turn angle for each spiral arm is that angle θ_T for which $r(\theta_T) = D/2$. If we ignore r_0 then total turn angle is approximately

$$\theta_T \approx \frac{\pi D}{k(1+\epsilon)w} \tag{6.6}$$

In the following analysis, we will require that θ_T should always be at least $\pi/2$ or so. And we shall assume that ϵ is a design constant of minor consequence to be subsumed by calibration constants.

Axial Spring Stiffness

From Wahl's classic tome of spring design (reference [53], equation 10-9, p. 129), we find the formula for axial deflection x per unit coil turning angle in a rectangular-bar helical spring under applied load F_x

$$\frac{dx}{d\theta} \approx 8.3\frac{F_x}{wt^3E}r^3 \tag{6.7}$$

This is for Poisson's ratio = 0.3, $t \ll w$ and $w \ll D$. Applying this locally to each point of our spiral arm gives total axial deflection

$$x = \int_0^{\theta_T}\frac{dx}{d\theta}d\theta = \frac{2\pi}{k(1+\epsilon)w}\int_0^{D/2}\frac{dx}{d\theta}dr = \frac{2\pi}{k(1+\epsilon)w}\frac{8.3F_x}{wt^3E}\int_0^{D/2} r^3 dr \tag{6.8}$$

Taking $\epsilon = 0.5$, for example, and carrying out the integration gives per-arm total axial deflection

$$x \approx 0.54\frac{D^4}{kw^2t^3E}F_x \tag{6.9}$$

and axial spring stiffness for the total flexure

$$K_a = \frac{kF_x}{x} \approx 1.8\frac{k^2w^2t^3E}{D^4} \tag{6.10}$$

Radial Spring Stiffness

The calculation of radial stiffness proceeds along similar lines. Again from Wahl (equation 24-7 p. 280) we get the formula for radial deflection y per unit coil turning angle in a rectangular-bar coil spring under shear load F_y

$$\frac{dy}{d\theta} \approx 6.0 \frac{F_y}{tw^3 E} r^3 \tag{6.11}$$

Integrating this over the total turning angle and taking $\epsilon = 0.5$, exactly as before, gives per-arm total radial deflection

$$y \approx 0.39 \frac{D^4}{kw^4 tE} F_y \tag{6.12}$$

and radial spring stiffness for the total flexure

$$K_r = \frac{kF_y}{y} \approx 2.6 \frac{k^2 w^4 tE}{D^4} \tag{6.13}$$

This averages out some large fluctuations that occur each quarter turn, so it is important that we not apply it to turning angles below about $\pi/2$. It also ignores buckling instability.

Peak Stress

Assuming the peak arm stress is pure torsion due to axial displacement, then according to Wahl (equation 10-7, p 128) it occurs at the outer end of the arms and is given by

$$\tau_m \approx 1.5 \frac{F_x D}{wt^2} \tag{6.14}$$

This is for $t \ll w$, $w \ll D$ and ignores any stress concentrations at the arm endpoint connections. Solving previous deflection equation (6.9) for F_x in terms of deflection x and substituting for F_x gives the equation for peak stress in terms of axial deflection

$$\tau_m \approx 2.8 \frac{kwtE}{D^3} x \tag{6.15}$$

Effective Mass

Based on kinetic and potential energy arguments the spiral-arm effective reciprocating mass as a fraction of actual mass is the area-weighted displacement-squared divided by the center displacement squared, or

$$M_e/M = \frac{\int_0^1 x^2(r) 2\pi r \, dr}{\pi x_0^2} \tag{6.16}$$

The above equation presumes a spiral arm of unit outer radius and center deflection x_0. From the previous axial-deflection formula (6.9) we may take,

$x(r) = x_0(1 - r^4)$ — highly nonlinear with most of the drop-off near the outer radius. Substituting into the previous equation and integrating gives the result

$$M_e/M = 0.53 \tag{6.17}$$

Actual mass M is just density ρ multiplied by the part of flexure disk area occupied by spiral arms, or $M = \rho(1/(1+\epsilon))t\pi D^2/4$. Taking $\epsilon = 0.5$, effective mass works out to

$$M_e = 0.28\rho t D^2 \tag{6.18}$$

Surge Frequency

Once we know axial spring stiffness and effective reciprocating mass, we may approximate surge frequency as

$$\omega_s \propto \sqrt{K_a/M_e} \approx 2.5\frac{kwt}{D^3}\sqrt{E/\rho} \tag{6.19}$$

which is just the resonant frequency of the flexure with no attached mass. This is not quite correct, for surge frequency is the frequency of a standing wave (half-wave) in the spiral arms with inner and outer ends clamped — not the same waveform used in deriving effective mass or axial spring stiffness. However, with the way these things usually go, the above formula is off by no more than a constant of proportionality which can be calibrated out with finite-element analysis.

Calibrated Formulae

Introducing into our equations the effect of n, the number of flexures stacked together, the above formulae become:

axial stiffness

$$K_a = C_a n\frac{k^2w^2t^3E}{D^4} \tag{6.20}$$

radial stiffness

$$K_r = C_r n\frac{k^2w^4tE}{D^4} \tag{6.21}$$

peak stress

$$\tau_m = C_\tau\frac{kwtE}{D^3}x \tag{6.22}$$

effective reciprocating mass

$$M_e = C_m n\rho t D^2 \tag{6.23}$$

surge angular frequency

$$\omega_s = C_\omega\frac{kwt}{D^3}\sqrt{E/\rho} \tag{6.24}$$

6.3 Nonlinear Spring



A nonlinear spring is a generic spring where the spring stiffness K varies as a quadratic function of position x. Most actual springs are nonlinear to some extent, depending on how far extended from their rest position $x = 0$. This component allows you to model the effects of operating a spring beyond its linear range. The stiffness function is defined by four inputs:

K0 : (real, n/m) Stiffness K_0 at $x = 0$.

Xm : (real, m) Reference extension x_m.

Rp : (real, dimensionless) Stiffness ratio $R_p = K/K_0$ at $x = x_m$.

Rn : (real, dimensionless) Stiffness ratio $R_n = K/K_0$ at $x = -x_m$.

You can read these, more-or-less directly, from a plot of local spring stiffness K vs x generated either experimentally or computationally. In terms of the restoring force $F(x)$, the spring stiffness is $K(x) = -dF/dx$. A more precise approach would involve finding the best-fit parabola (not necessarily symmetric about the rest position) to the function $K(x)$ over the intended operating range and then defining K0, Xm, Rp, Rn consistent with the parabola.

For internal processing, Sage translates the above inputs into a quadratic spring stiffness in the form

$$K(x) = K_0 \left(1 + a(x/x_m) + b(x/x_m)^2\right) \tag{6.25}$$

Coefficients a and b come from substituting x_m and $-x_m$ for x in equation (6.25), which results in the equation system

$$R_p = 1 + a + b \tag{6.26}$$
$$R_n = 1 - a + b \tag{6.27}$$

with the solution

$$a = (R_p - R_n)/2 \tag{6.28}$$
$$b = (R_p + R_n)/2 - 1 \tag{6.29}$$

The restoring force provided by a nonlinear spring is

$$F(x) = -K_0 x \left(1 + \frac{a}{2}(x/x_m) + \frac{b}{3}(x/x_m)^2\right) \tag{6.30}$$

The position $x = 0$ is the neutral force position.

You can attach a nonlinear spring to a time-ring reciprocating mass (not a phasor reciprocating mass) for modeling the resultant motion of nonlinear spring-mass systems. Even when driven by a sinusoidal forcing function, such a system will produce a reciprocating motion with higher harmonics. And the resonant frequency will change, depending on amplitude.

6.4 Relative Nonlinear Spring



A relative nonlinear spring is just like the above nonlinear spring, except it is connected between two moving parts instead of between a moving part and ground (fixed inertial frame). The same input variables and governing equations apply except that position coordinate x is replaced by the difference of the endpoint coordinates $x_{pos} - x_{neg}$. The relative nonlinear spring descends from the relative moving parts documented in the *Sage User's Guide*.

6.5 Linear Motors

A linear motor provides a reciprocating force as a function of electrical current, which is an input. There is no detailed analysis of electromagnetic fields in physical geometries — all the physics is collapsed into a single electromagnetic force coefficient input. There are phasor and time-ring versions.

You can connect a linear motor to a reciprocating mass through a standard force connection, just like you would a spring or damper. The motor will drive the mass according to the current and force-coefficient inputs. In the time-ring case, a spring is mandatory to determine the equilibrium position of the reciprocating mass. Otherwise the solver will tend to drift around without converging. Generally, even a very weak spring will do.

In a full-fledged stirling model, a linear motor can drive a reciprocating mass which, in turn, has a face area attached to the compression space gas. In the time-ring case, besides a spring attached to the reciprocating mass, there must also be a balancing area face on the other side of the reciprocating mass (opposite the compression-space attachment), attached to a buffer space. Otherwise there would be a great pressure-force imbalance on the piston which the spring would have difficulty offsetting. This inconvenience is the price for realism.

6.5.1 Phasor Motor



In the phasor motor, the current and resultant force are sinusoidal. Its effects can be produced by the built-in forcing function input of a reciprocating mass, although the motor component also calculates the electrical resistance (I^2R) losses in the coil. The motor is defined by four inputs:

Icoil : (phasor, A) Electrical current I in the coil.

Cf : (real, N/A) Force coefficient C_f produced by electromagnetic interaction.

Rcoil : (real, ohm) Electrical resistance R of the coil.

Inorm : (real, A) Current scale used to normalize Icoil for internal Sage purposes.

with a new output variable (besides those inherited from the ancestral moving part component):

Wcoil : (real, W) Electrical resistance I^2R losses in the coil.

The force produced by the motor is just the product of force coefficient and current

$$F = C_f I \tag{6.31}$$

6.5.2 Time-Ring Motor



In the time-ring motor the current is a Fourier series input and the force coefficient is a quadratic function of position. It can produce forces beyond the scope of even a Fourier series forcing function. One option would be to play around with the interaction between a nonlinear current and a nonlinear force coefficient, perhaps canceling each other out. Replacing the phasor-motor inputs Icoil and Cf, the time-ring motor has the following inputs:

FIcoil : (Fourier series, A) Electrical current I in the coil.

Cf0 : (real, n/m) Force coefficient C_{f0} at $x = 0$.

Xm : (real, m) Reference extension x_m.

Rp : (real, dimensionless) Force-coefficient ratio $R_p = C_f/C_{f0}$ at $x = x_m$.

Rn : (real, dimensionless) Force-coefficient ratio $R_n = C_f/C_{f0}$ at $x = -x_m$.

As with the analogous inputs for a nonlinear spring, you can read Cf0, Xm, Rp, Rn more-or-less directly, from a plot of C_f vs x generated either experimentally or computationally, or more precisely from a best-fit parabola to the function $C_f(x)$ over the intended operating range.

The force produced by the motor is still the product of force coefficient and current, as in equation (6.31), but now C_f is the quadratic function of position

$$C_f(x) = C_{f0}\left[1 + a(x/x_m) + b(x/x_m)^2\right] \tag{6.32}$$

where coefficients a and b are formulated in terms of inputs as

$$a = (R_p - R_n)/2 \tag{6.33}$$
$$b = (R_p + R_n)/2 - 1 \tag{6.34}$$

just as for the nonlinear spring stiffness function.

Chapter 7

Thermal Solids

Thermal solids represent the solid heat flow pathways found in stirling machines. Some are designed to be connected to the gas model components within heat exchangers. Some are designed to be connected only to other thermal solid components. The boundary connections to thermal solids may represent point contacts or line (surface) contacts carrying steady or time-varying heat flows. Some thermal solids are born with their boundary connectors while others get them via palette-created child model components. Ultimately your job is to connect the thermal solids in your stirling machine to form pathways from the working gas to a number of isothermal temperature sources or sinks.

7.1 Attachment Child Components

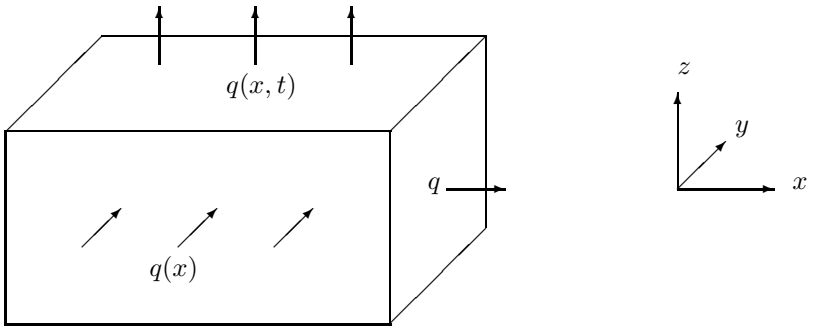
There are several types of thermal solids, with mathematical representations involving anything from single points, to space grids, to space-time grids. Some of these are born with appropriate heat-flow connectors, intended for connection to a gas component or another thermal solid component. Others have optional connections depending on which of the following child-model components you drag and drop into your edit form:

Icon	Purpose
	steady negative heat-flow end
	steady positive heat-flow end
	space-grid negative heat-flow face
	space-grid positive heat-flow face
	space-grid negative heat-flow end
	space-grid positive heat-flow end

When you drop one of these into the edit form, it is born with a connection arrow which you can then move up one level to the parent-model page for connection there to a mate, perhaps originating in another thermal solid component. The type of heat-flow connector arrow you get depends on the individual child component you create. You should have no trouble sorting out which connectors get connected where from the context of your problem and subsequent documentation.

7.2 Coordinate Conventions

All of the thermal solids attempt to adhere to a coordinate convention, both in their underlying model and their icon representation. A typical thermal solid domain is a rectangular solid that looks like this:



Generally speaking, the space-time varying $q(x,t)$ heat flow on the upper z face (with unit normal $\boldsymbol{z}$) is reserved for connection to a gas domain, the steady but spatially-distributed $q(x)$ heat flow on either y face is for connection to another a mating y face, while the steady single-point q heat flow on either x face is for connection to a mating x face. Generally, one or two, but not all, of these heat flow connections are available in any given thermal solid. Computationally, we discretize our solid in the x direction only — never the y or z directions. That is, we slice it into a stack of wafers rather than dice it into tiny cubes. Individual thermal solid model components make different assumptions about the physics within the volume elements. Sometimes there is time variation, sometimes there is not.

7.3 Entropy Generation

Most thermal solids involve the conduction of heat across finite temperature differences and therefore represent irreversible entropy-generating processes (see chapter 4).

The second law of thermodynamics applied to the external surface of our thermal solid, over a full periodic cycle is

$$\text{External Entropy Generation} = \oint_{dt} \int_{ds} \frac{\boldsymbol{n} \cdot \boldsymbol{q}}{T} \tag{7.1}$$

where $\boldsymbol{n} \cdot \boldsymbol{q}$ is the surface normal heat flux and T is the surface absolute temperature. Entropy generation defined this way refers to the increase in entropy in the universe surrounding the thermal solid domain as a result of internal irreversibilities.

The internal generation of entropy may be calculated as

$$\text{Internal Entropy Generation} = -\oint_{dt}\int_{dv}\frac{\boldsymbol{q}\cdot\nabla T}{T^2} \tag{7.2}$$

where the integrand is the local rate of entropy production due to heat flow in a temperature gradient. Since this is the only irreversible process in a thermal solid, its integral must equal the external entropy generation. We can prove this is so by applying Gauss's theorem to the external entropy generation formula, obtaining

$$\text{External Entropy Generation} = \oint_{dt}\int_{dv}\nabla\cdot\frac{\boldsymbol{q}}{T} \tag{7.3}$$

which can be simplified by substituting the vector identity $\frac{1}{T}\nabla\cdot\boldsymbol{q}+\boldsymbol{q}\cdot\nabla(1/T)$ for $\nabla\cdot\frac{\boldsymbol{q}}{T}$, then ignoring the first term because $\nabla\cdot\boldsymbol{q}\propto\frac{\partial T}{\partial t}$, which gives $\oint_{dt}\frac{1}{T}\nabla\cdot\boldsymbol{q}\propto\oint_{dT}dT/T=0$. Then substituting $-\nabla T/T^2$ for $\nabla(1/T)$ in the second term, we wind up with the above internal generation formula. This simplification holds only approximately when we substitute finite differences for exact derivatives.

Entropy generations are presented as available-energy outputs by multiplying the entropy integrals by Tnorm, the normalization temperature of the root stirling model component. Each model component evaluates its entropy integrals in a manner appropriate to the physics and temperature discretization embodied within it.

7.4 Solid Material

Certain thermal solid model components have an enumerated-type variable which selects the solid type from a list of choices. Each available solid is itself a software object with sufficient data encapsulated within it to know its important properties. These properties are.

density : (kg/m^3) Constant mass density ρ_s.

conductivity : (K, W/(m K)) A cubic spline variable with temperature vs conductivity $(T, k_s(T))$ data pairs covering a broad range of temperatures.

specific heat : (K, J/(Kg K)) A cubic spline variable with temperature vs specific-heat $(T, c_s(T))$ data pairs covering a broad range of temperatures.

The values of density and the cubic-spline interpolation pairs appear after the enumerated identifier in display windows and the output listing. The solid choices come from a default data base file solid.dta, or a data base file customized by you. (*see* chapter 14)

An instance of a solid object has methods (subroutines) that can be called upon to return conductivity k_s, specific heat c_s or thermal diffusivity α as functions of temperature T, whenever needed by the model component. This is, essentially, just a matter of cubic-spline interpolation using the appropriate set of data pairs.

7.5 Heat Sources

These may serve as either heat sources or sinks with a fixed temperature that you specify as an input. The heat flow in any connection is supposed to be regulated by the physics of the adjoining model component which, in effect, sees a fixed temperature boundary. Connecting together two heat sources is likely to result in a singular solution.

7.5.1 Point Heat Source



This component allows any number of steady point heat-flow connections to the positive or negative x ends of adjoining thermal solids. Its variables are:

Ts : (real, K) Midpoint temperature.

QNeg : (real, W) Net heat flow through negative x end.

QPos : (real, W) Net heat flow through positive x end.

7.5.2 Line Heat Source



This component allows any number of steady distributed heat-flow connections to the positive or negative y faces of adjoining thermal solids. Its variables are:

QyNeg : (real, W) Net (x integrated) heat flow through negative y face.

QyPos : (real, W) Net heat flow through positive y face.

Since a line source is always created as a child component to a parent, it always gets its temperature distribution $T(x)$ from that parent, in the form of a cubic spline variable.

7.5.3 Isothermal Surface



indexisothermal surface

This component allows a single time-varying distributed heat-flow connection to the negative z face of a gas domain. It functions as an isothermal heat exchanger surface. Its variables are:

QwNet : (real, W) Net (t averaged and x integrated) heat flow through positive z face.

In principle, the source temperature could have x and t variation, but it doesn't. Only x variation is allowed much like the preceding component. Also, like a line source, this component always gets its temperature distribution $T(x)$ from a cubic spline constant in a parent model component. Unlike a line source, though, heat flow does generally vary both in x and t on a space-time grid.

7.6 Heat Conductors

These are intended as parasitic conduction paths or intermediate paths from primary surfaces in contact with a gas component to heat sources or sinks. A regenerator pressure wall or a heat exchanger fin would be good examples.

7.6.1 Bar Conductor



This component models a simple linear solid conduction path with built-in steady heat-flow connectors at the positive and negative x ends. Heat flow is governed by

$$Q = \frac{k_s A}{L} \Delta T \tag{7.4}$$

where conductivity k_s is a temperature-dependent property of the solid material, area A and length L are specified directly and temperature difference ΔT is the difference between two hidden internal variables T_p and T_n, at the positive and negative x ends. Displayed variables are:

A : (real, m^2) Cross section area A.

L : (real, m) conduction length L.

Solid : (enumerated) Solid material.

TsNeg : (real, K) Temperature T_n at negative x end.

TsPos : (real, K) Temperature T_p at positive x end.

QNeg : (real, W) Net heat flow Q_n through negative x end.

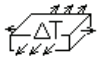
QPos : (real, W) Net heat flow Q_p through positive x end.

AEQ : (real, W) Available energy loss to heat flow.

QNeg and QPos are necessarily equal after solution, but they are listed as separate outputs because, logically, they are.

There is a variation of this component where variables A, L and Solid do not appear in the display. They are instead inherited from the parent model component. This type of conductor is typically used as a child component to a parent model component whose main purpose is not heat conduction but in whom heat conduction needs to be tagged on as an independent parasitic phenomenon. For example, a conductive-surface component within a canister (*see* chapter 9). All this is completely transparent to the user so there should be no confusion.

7.6.2 Distributed Conductor



This component models a rudimentary two-dimensional solid conduction path with any number of steady point heat-flow connections to the positive or negative x ends of adjoining thermal solids and any number of steady distributed connections to the positive or negative y faces. A typical use would be for an intermediate conduction path between a fixed-temperature heat source (sink) and one of the subsequent heat-exchanger surface components. A thermal busbar, if you will.

The thermal solid domain now has an axial center-line temperature distribution $T_s(x)$, discretized on a spatial grid, beginning at the negative x end and ending at the positive x end. It also has two additional discretized temperature distributions $T_n(x)$ and $T_p(x)$, parallel to $T(x)$, centered in the negative and positive y faces. Axial (x directed) heat flux is given by

$$q_x = k_s \frac{\partial T_s}{\partial x} \tag{7.5}$$

with $\frac{\partial T_s}{\partial x}$ being replaced by the finite-difference equivalent. Transverse (y directed) heat flux at the negative and positive y faces are given by

$$q_n = k_s \frac{T_n(x) - T_s(x)}{D/2} \tag{7.6}$$

and

$$q_p = k_s \frac{T_s(x) - T_p(x)}{D/2} \tag{7.7}$$

where D is the solid depth in the y direction. The governing equation for the interrelationship among q_x, q_n and q_p is the heat-flow continuity equation $\nabla \cdot \boldsymbol{q} = 0$ which may be written in the form

$$\frac{\partial q_x}{\partial x} + \frac{q_p - q_n}{D} = 0 \tag{7.8}$$

Actual heat flow per unit length in the y direction is either $q_n W$ or $q_p W$, where W is the solid thickness in the z direction. Actual heat flow in the x direction is $q_x A_s$ where $A_s = WD$ is the cross-sectional area.

Distributed conductor variables are:

W : (real, m) Solid thickness in z direction. Increasing this dimension gives more y heat flow for a given temperature drop.

D : (real, m) Solid depth D in y direction. Increasing this dimension gives less y heat flow for a given temperature drop.

Solid : (enumerated) Solid material.

Mass : (real, Kg) Solid mass, provided for use in constraints in case it is part of a reciprocating mass system.

QyNeg : (real, W) Net (x integrated) heat flow through negative y face.

QyPos : (real, W) Net heat flow through positive y face.

QxNeg : (real, W) Net heat flow through negative x end.

QxPos : (real, W) Net heat flow through positive x end.

AEQy : (real, W) Available energy loss to y directed heat flow, according to internal generation formula (7.2).

AEQx : (real, W) Available energy loss to x directed heat flow, according to internal generation formula (7.2).

AEdiscr : (real, W) Available energy discrepancy of above two losses compared to external generation as calculated by (7.1). (see section 7.3)

TsNeg : (real, K) Temperature T_s at negative x end.

TsPos : (real, K) Temperature T_s at positive x end.

The initial temperature distribution and length of a distributed conductor come from the parent model component.

There is a variation of this component which inherits its conduction-path cross-section area A_s and solid properties (Solid variable) from a parent component. Cross sectional area typically comes from a variable like Asolid in a canister component (*see* chapter 9) which may be either real or cubic-spline valued. Solid z-thickness W then becomes a dependent variable according to

$$W = A_s/D \tag{7.9}$$

W is either real or cubic-spline valued depending on the type of A_s in the parent. D remains an independent input, although its value is irrelevant (provided nonzero) if the component is used only to model axial heat flow (no y-face heat-flow connections). Otherwise, D may be used to control the y temperature difference for distributed heat flow, as described above.

7.6.3 Conductive Surface



This component is similar to the previous distributed conductor except it also allows a single time-varying distributed heat-flow connection from its positive z face to the negative z face of a gas domain. In other words, the positive z face models the wetted surface of a heat exchanger. The interior models a conduction pathway to the ultimate source or sink, which may be connected through either x end or y face. A typical use would be for a rectangular heat-exchanger fin, although it is adaptable to any geometry. Like a distributed conductor, a conductive surface inherits its initial temperature distribution and length from a parent model component.

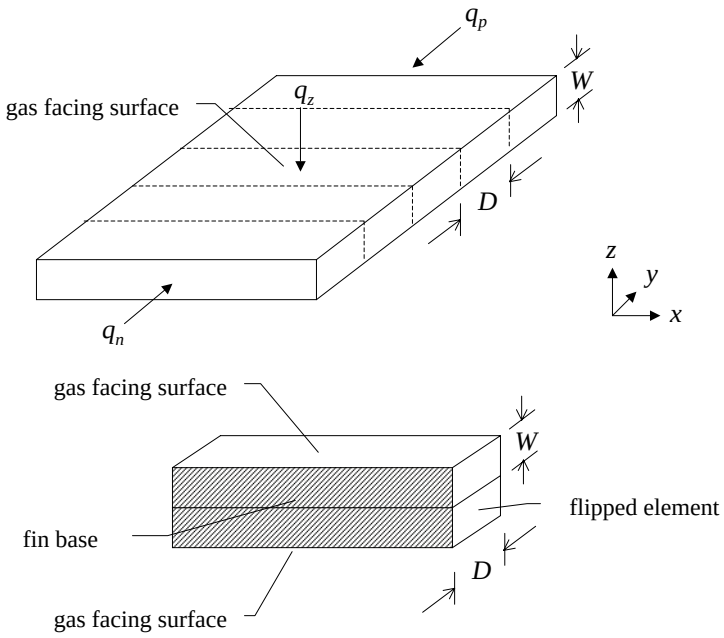


Figure 7.1: For illustration purposes, a conductive surface morphed into heat exchanger fins. The top picture corresponds to the basic thermal solid domain of thickness W broken into $2n$ pieces of width D (D is independent input variable). In the bottom picture the individual pieces are re-arranged into n heat exchanger fins, each of height D and thickness $2W$. Only one fin is shown. The same one-dimensional solution grid applies to all the fins together.

A conductive surface also inherits cross-section area A_s and wetted perimeter S_x from its parent component, typically from variables Asec and Pwet in a heat-exchanger component (*see* chapter 10), which may be either real or cubic-spline valued. This information is translated into $2n$ rectangular solid domains of z-thickness W and y depth D as shown in figure 7.1. W is a dependent variable calculated as

$$W = A_s/S_x \tag{7.10}$$

and D is independent input variable. The implied number of rectangular domains is just the quotient of A_s and WD or

$$2n = \frac{S_x}{D} \tag{7.11}$$

In the event the conductive surface represents rectangular fins n corresponds to the fin number. The assumption is that these rectangular domains act in parallel. The z-thickness W of each one is always a reasonable approximation of the parent heat exchanger wall thickness. The y-depth D is up to you. As you reduce D you are making y-directed heat flow easier by effectively breaking the total wetted perimeter of the heat-exchanger wall into $2n$ parallel segments, as illustrated in figure 7.1. Although this may seem complicated, most of the complexity is managed internally, Your only responsibility is to enter the correct value for D, the conduction-segment y-length, based on your understanding of the heat exchanger geometry.

Although the gas-surface heat flux may have a time-varying component, only the steady component contributes to the interior solid solution. In other words, the solid filters out any AC heat flow components, passing only the DC component to the source or sink. This is appropriate where the net steady component of gas-surface heat transfer dominates the time-varying component — as in a heat rejector or acceptor. Otherwise, in a regenerator for example, one of the quasi-adiabatic surface components might be more appropriate.

In the conductive surface the thermal solid domain has an axial temperature distribution $T_w(x)$ centered in the positive z face (serving all $2n$ fin faces of fin morphing in figure 7.1), in addition to $T_s(x)$, $T_n(x)$ and $T_p(x)$ as previously described for the distributed conductor. Gas surface heat flux at the positive z face is given by

$$\langle q_w \rangle = k_s \frac{T_s(x) - T_w(x)}{W/2} \tag{7.12}$$

where W is the solid thickness in the z direction and $\langle\rangle$ symbolizes the time-average operator. The equations for axial and transverse heat fluxes q_x, q_n and q_p are the same as those given previously for the distributed conductor, except that q_x includes a tortuosity factor as explained below. The governing equation for the interrelationship among heat fluxes is again the heat-flow continuity equation $\nabla \cdot \boldsymbol{q} = 0$ which may now be written in the form

$$\frac{\partial q_x}{\partial x} + \frac{q_p - q_n}{D} + \frac{\langle q_w \rangle}{W} = 0 \tag{7.13}$$

The y-directed heat fluxes q_n and q_p are always based on the total z thickness A_s/D. The z-directed heat flux q_w is always based on the total wetted perimeter S_x. So, even if the parent geometry is not rectangular fins, the model continues to make sense because variables A_s and S_x are always defined.

Because a conductive surface can represent the solid surface of a matrix-type heat exchanger the axial heat flow q_x is discounted by a possible tortuosity factor $f_s < 1$ (*see* section 10.0.1). Axial heat flow q_x is calculated as

$$q_x = -k_s A_e \frac{\partial T_s}{\partial x} \tag{7.14}$$

where k_s is solid conductivity and A_e is the effective solid conduction area. For axially-uniform regenerator matrices, such as wrapped foil, A_e is just the mean solid area A_s. For axially-irregular matrices like stacked screens, A_e is the mean solid area reduced by a tortuosity factor f_s to account for the small contact areas between wires (*see* section 10.0.1). Sage does not apply the tortuosity factor to thermal conduction in the z or y directions because the first generally pertains to conduction normal to the surface of individual matrix particles and the second pertains to bulk conduction out of the matrix perpendicular to the flow direction, which may be along a continuous path (e.g. stacked screens in plane of screens). If it is necessary to model some tortuosity in the y direction you can always increase the D input.

Conductive surface variables are:

D : (real, m) Individual fin depth D. Often thought of as the fin conduction length in a typical sectional view. In general, the conduction path length (of one or more parallel segments) from the wetted surface of the heat-exchanger wall to the point(s) where heat is added or removed at the outer perimeter.

Solid : (enumerated) Solid material.

Mass : (real, Kg) Total solid mass, provided for use in constraints in case it is part of a reciprocating mass system.

W : (real, m) Effective heat-exchanger wall thickness. Half the thickness of a symmetrically-heated fin. In general, A_s/S_x, where cross-section A_s and wetted perimeter S_x come from a parent model component. Spatial average value if A_s and S_x are cubic-spline valued.

Tortuosity : (real, dimensionless) Mean (x average) solid axial conduction multiplier f_s (*see* section 10.0.1). $f_s A_s$ is the effective area A_e for thermal-solid axial conduction (*see* equation (7.14)). So named because of the tortuous solid conduction path in porous materials.

QwNet : (real, W) Net (t averaged and x integrated) heat flow through positive z gas-facing surface.

QyNeg : (real, W) Net (x integrated) heat flow through negative y face.

QyPos : (real, W) Net heat flow through positive y face.

QxNeg : (real, W) Net heat flow through negative x end.

QxPos : (real, W) Net heat flow through positive x end.

AEQw : (real, W) Available energy loss to z directed gas-surface heat flow, according to internal generation formula (7.2).

AEQy : (real, W) Available energy loss to y directed heat flow, according to internal generation formula (7.2).

AEQx : (real, W) Available energy loss to x directed heat flow, according to internal generation formula (7.2).

AEdiscr : (real, W) Available energy discrepancy of above three losses compared to external generation as calculated by (7.1). (see section 7.3)

TsNeg : (real, K) Temperature T_s at negative x end.

TsPos : (real, K) Temperature T_s at positive x end.

7.7 Quasi-Adiabatic Surfaces

These thermal solids all presume the positive z surface is subject to time-varying sinusoidal heat flux, with zero or near zero mean, usually as the primary surface in contact with the gas within a regenerator or variable-volume space. Accordingly they are born with a surface heat-flow connector, intended for connection to a gas model component. The presumed zero-mean heat flow is where the term quasi-adiabatic comes from.

Aside from that they are in many respects similar to the previous conductive surface component. You may connect either x-end or y-face to other thermal solids via optional steady point or distributed heat-flow connections. But only the DC (steady) component of heat-flow passes through such connections. Heat-flow connections are made via optional negative or positive heat-flow ends or faces available on the Thermal Attachments page of the child-model creation palette.

As with a conductive surface the actual solid geometry, which may be an extremely complicated porous structure, is shoe-horned into an equivalent rectangular solid based on the cross-section area A_s and wetted perimeter S_x inherited from the parent component. Independent input variable D defines the effective depth for any y directed heat flows you may wish to model.

The thermal solid domain has an axial center-line temperature distribution $T_s(x,t)$ and a temperature distribution $T_w(x,t)$ centered in the positive z face. Both are discretized on a space-time grid, beginning at the negative x end and ending at the positive x end. In support of y-directed heat flows there are also two time-independent temperature distributions $T_n(x)$ and $T_p(x)$ centered in

the negative and positive y-faces, similar to those of a distributed conductor. These are discretized on a separate spatial grid.

The governing equation for the thermal solid is now the time-dependent energy equation $\frac{\partial e_s}{\partial t} + \nabla \cdot \boldsymbol{q} = 0$ where e_s is the volume-specific solid energy. If we section-average this equation through the entire z depth of the solid we may write it in the form

$$\rho_s c_s A_s \frac{\partial T_s}{\partial t} + \frac{\partial q_x}{\partial x} + Q_w + Q_p - Q_n = 0 \tag{7.15}$$

where

A_s	=	mean solid cross section
c_s	=	solid specific heat
q_x	=	solid-mode axial heat flow
Q_w	=	z surface heat transfer per unit length
Q_p	=	positive y face heat flow per unit length
Q_n	=	negative y face heat flow per unit length
ρ_s	=	solid density

The purpose of the solid energy equation is to determine T_s as an implicit variable.

Equation (7.15) is not in conservative form (energy conserving) because factor c_s is not time-constant. As it stands, numerical differencing errors would result in $\oint \rho_s c_s A_s \frac{\partial T_s}{\partial t}\, dt$ not quite zero, as it should be. To avoid this problem, Sage replaces $c_s(T_s)$ with $c_s(\langle T_s \rangle)$, where $\langle T_s \rangle$ is time-mean T_s.

Axial heat flow q_x based on the effective solid cross-section area A_e (mean area A_s reduced by a tortuosity factor f_s) the same as for a conductive surface and is given by equation (7.14).

The steady transverse (y directed) heat flows per unit length at the negative and positive y faces are given by

$$Q_n = (A_s/D)\, q_n = (A_s/D)\, k_s \frac{T_n(x) - \langle T_s \rangle (x)}{D/2} \tag{7.16}$$

and

$$Q_p = (A_s/D)\, q_p = (A_s/D)\, k_s \frac{\langle T_s \rangle (x) - T_p(x)}{D/2} \tag{7.17}$$

where D is the solid depth in the y direction, A_s/D is the total solid width in the z direction (*see* section 7.6.3) and $\langle\rangle$ symbolizes the time average operator. Note that for the same temperature difference the y directed heat flux varies inversely with D^2, according to the above equations. So it important to set the value of D carefully when modeling y-directed heat flows.

For surface heat transfer Q_w Sage uses a formulation involving the temperature difference $T_s - T_w$, where T_w is understood as the surface temperature. To obtain our formulation we must digress briefly into the exact temperature solution in a solid layer subject to sinusoidal heat flux — more specifically, a solid slab of thickness d, insulated at the face $z = 0$ and subject to heat flux $q_z = e^{i\omega t}$ at face $z = d$.

This is approximately what's going on in a regenerator matrix, with the face $z = d$ corresponding to the gas-solid interface. Thickness d would represent foil half-thickness, in the case of a foil matrix. In general, d is taken as the solid volume divided by wetted surface. This is also approximately what's going on in the walls of a variable-volume space, so long as heat flows to and from the wall with zero mean heat transfer, as is often the case.

In a complex formulation, the exact temperature field solving this problem is

$$\boldsymbol{T} = -\frac{\delta}{k_s(1+i)} \frac{\cosh((1+i)z/\delta)}{\sinh((1+i)d/\delta)} e^{i\omega t} \tag{7.18}$$

where

$$\begin{aligned} \delta &= \sqrt{(2\alpha)/\omega}\text{; thermal penetration depth} \\ \alpha &= k_s/(\rho_s c_s)\text{; thermal diffusivity} \end{aligned}$$

In particular, the complex temperature at the gas-solid interface ($z = d$) is

$$\boldsymbol{T}_w = \boldsymbol{T}(d) = -\frac{\delta}{k_s(1+i)\tanh((1+i)d/\delta)} e^{i\omega t} \tag{7.19}$$

with complex heat flow per unit length

$$\boldsymbol{Q}_w = -k_s S_x \left(\frac{\partial \boldsymbol{T}}{\partial z}\right)_w = S_x e^{i\omega t} \tag{7.20}$$

where $S_x = A_s/d$ is wetted perimeter. The section-average temperature is

$$\boldsymbol{T}_s = \frac{i\delta^2}{2k_s d} e^{i\omega t} \tag{7.21}$$

which is also the solution of our mean-parameter solid energy equation (7.15) with $\frac{\partial q_x}{\partial x} = 0$ and $\boldsymbol{Q}_w = (A_s/d)e^{i\omega t}$.

Complex heat flux may be written as

$$\boldsymbol{Q}_w = \frac{k_s}{d} S_x \boldsymbol{N}_s (\boldsymbol{T}_s - \boldsymbol{T}_w) \tag{7.22}$$

where $\boldsymbol{N}_s$ is a dimensionless number not unlike the Nusselt number of gas heat transfer. Solving equation (7.22) for $\boldsymbol{N}_s$ and substituting the above solutions (7.19), (7.20) and (7.21) for $\boldsymbol{T}_w$, $\boldsymbol{Q}_w$ and $\boldsymbol{T}_s$ gives

$$\boldsymbol{N}_s = \frac{2i(d/\delta)^2}{\frac{(1+i)d/\delta}{\tanh((1+i)d/\delta)} - 1} \tag{7.23}$$

Interesting are the limiting cases for $d/\delta \to \infty$ (thick wall, high frequency, low conductivity) and for $d/\delta \to 0$ (thin wall, low frequency, high conductivity). These are

$$\boldsymbol{N}_s = \begin{cases} (1+i)d/\delta & \text{if } d/\delta \to \infty \\ 3 & \text{if } d/\delta \to 0 \end{cases} \tag{7.24}$$

For thick solids, where thermal activity is confined to a layer roughly δ thick, surface heat flux is phase-shifted 45 degrees ahead of temperature difference $\boldsymbol{T}_s - \boldsymbol{T}_w$. For thin solids, where the entire solid is thermally active, there is no phase shift.

In practice, the physical heat flux we are interested in is the real part of $\boldsymbol{Q}_w$

$$Q_w = \frac{k_s}{d} S_x \Re(\boldsymbol{N}_s(\boldsymbol{T}_s - \boldsymbol{T}_w)) \tag{7.25}$$

The complex temperature difference $\boldsymbol{T}_s - \boldsymbol{T}_w$ required for the right side is formulated in terms of the physical real temperature difference $T_s - T_w$ as

$$\boldsymbol{T}_s - \boldsymbol{T}_w = (T_s - T_w) - \frac{\partial}{\partial \omega t}(T_s - T_w) \tag{7.26}$$

This is valid so long as the temperature difference varies sinusoidally, because if $T_s - T_w = \Re(\boldsymbol{T}_s - \boldsymbol{T}_w) = \Re(\boldsymbol{C}e^{i\omega t})$, then $-\frac{\partial}{\partial \omega t}(T_s - T_w) = \Re(-i\boldsymbol{C}e^{i\omega t}) = \Im(\boldsymbol{C}e^{i\omega t}) = \Im(\boldsymbol{T}_s - \boldsymbol{T}_w)$. And it is not Q_w that is solved from equation (7.25) but, rather, the other way around. Sage solves the two variables T_s and T_w implicitly using equations (7.15) and (7.25), considering sinusoidal heat flux Q_w as given.

There are quasi-adiabatic model components for $\boldsymbol{N}_s$ computed rigorously, according to equation (7.23), as well as the thick- and thin-wall approximations according to (7.24). They all descend from a common ancestor which has the following variables:

Kmult : (real, dimensionless) Empirical multiplier for axial conduction q_x.

D : (real, m) Effective solid-conduction distance between the center line and the surface where any y face heat flow is extracted. The value of D only matters when there is a distributed heat-flow connection made at either y face.

Solid : (enumerated) Solid material.

Mass : (real, Kg) Solid mass, provided for use in constraints in case it is part of a reciprocating mass system.

Dskin : (real, m) Skin thickness d in z direction, calculated as solid volume divided by wetted perimeter (A_s/S_x).

Lambda : (real, m) Mean thermal wavelength λ in z direction, calculated as ($2\pi\sqrt{2\alpha/\omega}$).

Tortuosity : (real, dimensionless) Mean (x, t average) solid axial conduction multiplier f_s (*see* section 10.0.1). $f_s A_s$ is the effective area A_e for thermal-solid axial conduction (*see* equation (7.14)). So named because of the tortuous solid conduction path in porous materials.

FQwNet : (Fourier series, W) Net (x integrated) heat flow through positive z gas-facing surface.

QxNeg : (real, W) Net heat flow through negative x end.

QxPos : (real, W) Net heat flow through positive x end.

AEQw : (real, W) Available energy loss to z directed gas-surface heat flow, according to internal generation formula (7.2).

AEQx : (real, W) Available energy loss to x directed heat flow, according to internal generation formula (7.2).

AEdiscr : (real, W) Available energy discrepancy of above two losses compared to external generation as calculated by (7.1). (see section 7.3)

TsNeg : (real, K) Temperature T_s at negative x end.

TsPos : (real, K) Temperature T_s at positive x end.

FTsMean : (Fourier series, K) Spatial averaged (x average) temperature T_s.

Length l, cross sectional area A_s, z surface wetted perimeter S_x and initial temperature distribution T_s all come from the parent model component.

7.7.1 Thick Surface



This quasi-adiabatic surface presumes the skin thickness d is much larger than the thermal wavelength λ. It is intended for modeling something like a thermally massive cylinder space wall, where the time-varying heat flux is confined to a thin layer near the surface.

7.7.2 Thin Surface



This quasi-adiabatic surface presumes the skin thickness d is much smaller than the thermal wavelength λ. It is intended as a fast alternative to the following rigorous surface for modeling something like a regenerator matrix when the wire diameter, particle size, foil thickness, or whatever, is very small. If you use this component, you should check that output variable Dskin is indeed smaller than Lambda. If not, then you should use the rigorous surface component instead.

7.7.3 Rigorous Surface



This quasi-adiabatic surface presumes the skin thickness d is on the same order as the thermal wavelength Lambda. It is intended for modeling any sort of regenerator matrix or duct wall, although the slightly-faster thin-surface model component may be justified for *finely divided* matrices. A rigorous surface calculates the Nusselt-like number $\boldsymbol{N}_s$ from equation (7.23) which takes a bit more time than the thick- or thin-surface approximations.

7.8 Floating Isothermal Surface



This component was originally intended to be a faster version (in computational time) of the above quasi-adiabatic surfaces for use as the primary surface anchoring the gas temperature within a variable-volume space when the thermal solution within the solid was not important. It is equivalent to the thin-surface quasi-adiabatic model component in the limit of infinite thermal mass and zero axial conduction. It presumes the positive z surface is subject to time-varying sinusoidal heat flux, with zero mean. The negative z surface along with both x and y surfaces are presumed insulated. It has a discretized temperature distribution $T_s(x)$ with no time variation. T_s serves as both the solid interior temperature and the temperature at the positive $z$surface. Sage implicitly solves T_s to achieve zero net (time-mean) surface heat flux.

As it turned out, floating isothermal surfaces were not noticeably faster than other quasi-adiabatic solids and the solid temperature would often drift to physically unrealistic values because of the lack of solid axial conduction. In some cases this led to convergence problems. So the use of floating isothermal surfaces is no longer recommended but they are still documented here for completeness and available in the software for special purposes.

A floating isothermal surface receives its initial temperature distribution T_s from its parent model component, but has no variables of its own to appear in display windows or output listings. Everything you need to know about surface heat transfer and temperature is available from the gas-domain component it is connected to.

Chapter 8

Gas Domains

The gas-domain model components are at the heart of the stirling model class. Because of their complicated mathematical structure, they are also the most time consuming to solve and prone to convergence problems. From your point of view, they appear simply as palette-created child model components within heat-exchanger geometries. There are gas-domain variations for all of the basic types of fluid flow: duct flow, porous matrix flow and flow into a variable-volume space (formed by a piston in its cylinder) through an inlet. All gas domains are born with a thermal boundary connector intended for connection to a thermal solid of your choice. Gas domains, being one dimensional, admit to flow connections with other gas domains at either their positive or negative ends. Since you may want to have flow connections at only one end, at both ends, or multiple connections at one end, flow connectors are palette-created child-model components. Variable-volume gas domains have palette-created displaced-volume connectors intended for connection to area faces of moving parts.

There remains one more gas-domain connection: the charge line. Exactly one gas domain model component must have a charge-line connection to a pressure-source component to establish the amount of working gas in the sum-total of all interconnected gas domains. This is very important. Without a pressure-source connection there will be no way for Sage to determine the amount of working gas in your system and solving will not converge. Charge-line connectors are palette created because there is no way for Sage to decide which gas component should get it.

8.1 Attachment Child Components

All gas domains have the ability to produce child model components representing charge-line connectors and flow inlets. Some can also produce volumetric displacements.

When you drop a gas domain model component from the stirling palette into the edit form, it starts out with only a thermal connector arrow, intended for

connection to a thermal solid component. In order to make additional connections, you must first drop in one or more of the following child model components from the palette.

Icon	Purpose
	gas charge line
	negative gas inlet
	positive gas inlet
	phasor negative-facing volume displacement
	phasor positive-facing volume displacement
	time-ring negative-facing volume displacement
	time-ring positive-facing volume displacement

When you drop one of these into the edit form, it is born with a connection arrow which you can then move up one level to the parent-component page of the edit form for connection there to a mate, perhaps originating in another gas domain.

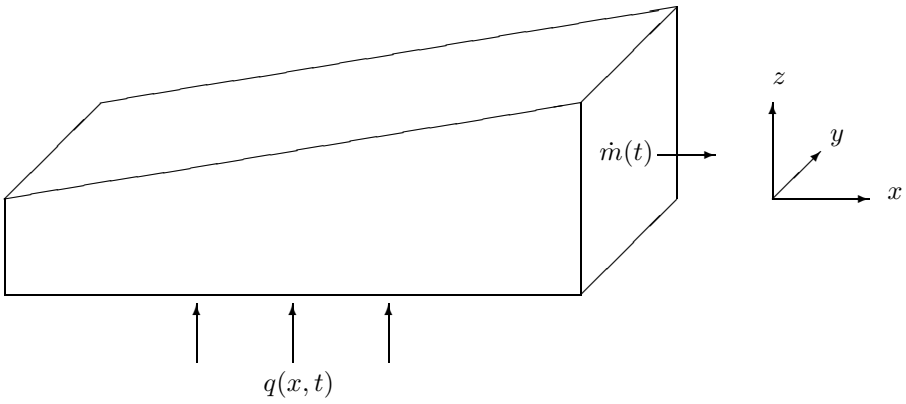
The gas charge line is for connection to a pressure-source component. It is required for exactly one gas domain component in your total interconnected gas domain, as explained above.

The two types of gas inlets are for connection to other gas domains as you see fit. Generally you will want zero or one flow inlets at each end, depending on whether your component is functioning as a dead-ended passage or through passage. In some instances you will create two (or more) flow inlets at the end of a gas component which enables it to act as a T or Y component.

The volume displacements are available only in variable-volume gas domains. They are intended for connection to area faces of moving parts like the reciprocator or constrained piston components documented in the Sage manual under the *resonant system* model class.

8.2 Coordinate Conventions

A gas domain may be thought of as a one-dimensional rectangular region like this:



Gas flow $\dot{m}(t)$ may pass through either the positive or negative x boundaries, but nowhere else. Heat flux $q(x,t)$, generally to a thermal solid, may pass through the negative z boundary, but nowhere else. Discretization is in the x direction only, similar to a thermal solid. In principle, the cross-sectional area may vary with x, as drawn. It may even vary with time, as if the upper z surface were moving in accordance with a volume displacement.

8.3 Entropy Generation

Gas domains involve the conduction of heat across finite temperature differences as well as irreversible flow processes. Both generate entropy (see chapter 4).

The second law of thermodynamics applied to the external surface of our gas domain, over a full periodic cycle is

$$\text{External Entropy Generation} = \oint_{dt} \int_{ds} \frac{\boldsymbol{n} \cdot \boldsymbol{q}}{T} + \left[\oint_{dt} \dot{m} s \right]_{x=a}^{x=b} \tag{8.1}$$

where $\boldsymbol{n} \cdot \boldsymbol{q}$ and T are the z-surface normal heat flux and absolute temperature, and $\dot{m}$ and s are the x-end mass flow rate and mass-specific entropy. The $[\cdot]_{x=a}^{x=b}$ notation means the difference of the enclosed expression evaluated a the two endpoints. Entropy generation defined this way refers to the increase in entropy in the universe surrounding the gas domain as a result of internal irreversibilities.

There are two internal entropy generating mechanisms, heat flow and viscous friction. There are actually two distinct heat-flow mechanisms, axial conduction and film heat transfer, but at this level of generality the entropy generated by either is covered by the formula

$$\text{Conductive Entropy Generation} = - \oint_{dt} \int_{dv} \frac{\boldsymbol{q} \cdot \nabla T}{T^2} \tag{8.2}$$

where the integrand is the local rate of entropy production due to heat flow in a temperature gradient. And the entropy generation of viscous friction, in one-dimensional flow, may be calculated as

$$\text{Viscous Entropy Generation} = -\oint_{dt}\int_{dx}\frac{uAF}{T} \tag{8.3}$$

where uA is volumetric flow rate and F represents that part of the pressure gradient due to viscous friction. The numerator in the integrand is the pumping dissipation per unit length.

Entropy generations are presented as available-energy outputs by multiplying the entropy integrals by Tnorm, the normalization temperature of the root stirling model component. Each model component evaluates its entropy integrals in a manner appropriate to the physics and solution-variable discretization embodied within it.

8.4 Pressure Source Component



The pressure source component establishes the charge pressure for your stirling model. You must have exactly one pressure source, connected to some gas domain. A pressure source acts as an infinite isobaric gas reservoir. Its only variable is

Pcharge : (real, Pa) Charge pressure.

The result of connecting a pressure source to a gas domain is that the density in the gas domain adapts itself so that t average pressure is continuous across the connection — so that the time-average pressure at the negative end of the gas domain becomes Pcharge. This is explained mathematically in section (8.6).

8.5 Gas Domain Components

There are three distinct variations of gas domains, distinguished primarily by the method by which they track the onset of turbulence. They all come with a large assortment of variables, mostly outputs, sufficient for most of your needs but not too numerous to unduly clutter your output listings. For more information you can always dump the solution grid with the File|Save Solution Grid command. The following list includes all possible variables in all types of gas domains. Some variables may be missing from individual gas domains, depending on what makes sense. Mathematical symbols, as yet undefined, will be found in the subsequent sections on theory at the end of this chapter.

Fmult : (real, dimensionless) Empirical multiplier for viscous pressure drop F.

Hmult : (real, dimensionless) Empirical multiplier for z-surface heat transfer Q_w.

Kmult : (real, dimensionless) Empirical multiplier for axial conduction q in domain interior.

KmultBnd : (real, dimensionless) Empirical multiplier for axial conduction q at domain endpoints. Default is 0 which blocks thermal conduction between gas domains. Set to 1 in both gas domains on either side of a flow connector to allow continuous thermal conduction across the flow connector.

UpwindFrac : (real, dimensionless) The relative weight factor for upwind influence in density (temperature) interpolation between control volumes in the computational grid (*see* section 8.6.6). Increasing the value will help smooth out a jagged axial temperature profile at some cost in reduced accuracy. Based on computational experiments, the default of 1.0E-2 provides sufficient stability for almost all situations with a negligible effect on accuracy. In the event of temperature instability (which you can see by plotting a file of computational solution variables), increasing the value to 1.0E-1 should stabilize almost any gas domain. Poor convergence may be a sign of temperature instability. Inputs are restricted to the range zero to one.

Klocal : (real, dimensionless) Local frictional pressure-drop loss coefficient K (see equation (8.14). Used for including entrance, exit and bend effects only in duct-type gas domains.

TbInNeg : (real, dimensionless) Incoming relative turbulence $\mathcal{T}$ at the positive end of a duct-type gas domain. The default is 1, corresponding to a sharp entrance where flow separation is expected or the upstream turbulence level is high. A value of 0 implies completely smooth incoming flow corresponding to a continuous flow area or a perfectly diffused transition and low upstream turbulence level.

TbInPos : (real, dimensionless) Like TbInNeg except at the duct-gas negative end.

FQcombust : (Fourier series, W) Combustion firing rate as a function of time. Available as an input variable only in the combustion-space gas component within a generic-cylinder component. It directly determines the combustion heating term Q_c in the gas energy equation. (*see* section 8.6.3).

PV : (Fourier series, W) Net (x integrated) PV power output ($\int_{dx} P\frac{\partial A}{\partial t}$). Available only in variable-volume gas domains where flow area A is a function of time. The mean value is the t-average PV power which most people think of when they think of PV power. A positive value indicates work done by the gas on the walls. The higher harmonics are available if you are interested.

FQwNet : (Fourier series, W) Net (x integrated) heat flow through negative z solid-facing surface (positive is from solid to gas).

HNeg : (real, W) t average stagnation enthalpy flow through negative x inlet. Stagnation enthalpy $uA(\rho e + P)$ includes all forms of energy flow — internal, PV and kinetic.

HPos : (real, W) t average stagnation enthalpy flow through positive x inlet.

AEfric : (real, W) Available energy loss to viscous flow friction, according to internal generation formula (8.3).

AEQw : (real, W) Available energy loss to z directed solid-surface heat flow, according to internal generation formula (8.2).

AEQx : (real, W) Available energy loss to x directed heat flow, according to internal generation formula (8.2).

AEdiscr : (real, W) Available energy discrepancy of above losses compared to external generation as calculated by (8.1). (see section 8.3)

QwNeg : (real, W/m) t average heat flow per unit length through negative z solid-facing surface, at negative x end. Gives insight into surface heat flow distribution.

QwPos : (real, W/m) Same as QwNeg except at positive x end.

QxMean : (real, W) t and x averaged axial heat flow q.

TNeg : (real, K) t average temperature T at negative x end.

TPos : (real, K) t average temperature T at positive x end.

Vmean : (real, m^3) Gas domain volume. Provided for fixed as well as time-varying volumes, although in that case Vmean is the t average volume. *See also:* FV for variable-volume gas domains.

FTMean : (Fourier series, K) Spatial averaged (x average) temperature T.

FPMean : (Fourier series, Pa) Spatial averaged (x average) pressure P.

FPNeg : (Fourier series, Pa) Pressure P at negative boundary.

FPPos : (Fourier series, Pa) Pressure P at positive boundary.

FDP : (Fourier series, Pa) Pressure difference across domain (positive − negative x ends).

FV : (Fourier series, m^3) Domain volume for a variable-volume gas domain. The mean value (t average) and first harmonic are generally of the most interest.

FM : (Fourier series, kg) Total mass in a gas domain. The sum of all mean values (t averages) over all gas domains is the total gas mass in the system. The time derivatives of the first and higher harmonics are theoretically the same as the difference of inlet mass flow rates (FRhoUAPos - FRhoUANeg). But due to inaccuracies of finite time-differencing, this is exact only for the first harmonic and in the limit of large time nodes for higher harmonics.

FHmean : (Fourier series, W) Spatial averaged (x average) stagnation enthalpy flow $uA(\rho e + P)$.

FRhoUAmean : (Fourier series, kg/s) Spatial averaged (x average) mass flow rate ρuA.

FRhoUANeg : (Fourier series, kg/s) Mass flow rate ρuA through negative x inlet.

FRhoUAPos : (Fourier series, kg/s) Mass flow rate ρuA through positive x inlet.

MachMean : (real, dimensionless) t and x averaged Mach number.

TdMean : (real, dimensionless) t and x averaged tidal amplitude divided by length L. Total tidal displacement is computed as the mean absolute velocity multiplied by half a cycle period. Tidal amplitude is half that. Only available in duct or matrix gas domains.

ReMean : (real, dimensionless) t and x average Reynolds number R_e in matrix or duct gas domains or turbulent Reynolds number R_t in variable-volume gas domains.

VaMean : (real, dimensionless) t and x averaged Valensi number V_a in duct gas domains.

TbMean : (real, dimensionless or J/m^3) t and x averaged relative turbulence $\mathcal{T}$ in duct gas domains or turbulence intensity $\rho\kappa$ in variable-volume gas domains.

ZMean : (real, dimensionless) t and x averaged gas compressibility $P/(\rho RT)$ as returned by the gas property variable Gas selected in the stirling root component. This is always one for an ideal gas. A Redlich-Kwong gas returns a more realistic value.

EOSErrMean : (real, dimensionless) t and x averaged equation of state relative error as returned by the gas property variable Gas selected in the stirling root component. This is always zero for an ideal gas. A Redlich-Kwong gas returns a more realistic estimate.

Coming from the parent component are the length L, mean flow area A, wetted perimeter S_x, initial axial temperature distribution $T(x)$ in the form of a cubic spline variable and working gas (properties).

A primary purpose of a gas domain is to calculate the empirical relationships for friction factor f, Nusselt number N_u and axial conductivity ratio N_k appropriate for the flow geometry in which it finds itself (see section (8.6). Accordingly, there is a gas-domain variation for each individual type of heat exchanger geometry. But they all fit into one of the following three classes.

8.5.1 Matrix Gas Domains



A *matrix* gas domain is used within a porous matrix or within uniform channels of tiny hydraulic diameter. A matrix gas domain knows about Reynolds number, defined as

$$R_e = \frac{\rho |u| d_h}{\mu} \tag{8.4}$$

where u is mean-flow velocity and d_h is hydraulic diameter defined as

$$d_h = \frac{4A}{S_x} \tag{8.5}$$

It also knows about matrix porosity, which is a variable imported from the parent component.

Wetted perimeter is the wetted surface area per unit length. The notation S_x is shorthand for $\frac{\partial S}{\partial x}$, where $S(x)$ is the cumulative wetted surface between position zero and x. For domains where S varies linearly with x, wetted perimeter is just the total wetted surface divided by domain length.

8.5.2 Duct Gas Domains



A *duct* gas domain is used within relatively short flow ducts (generally tubes or rectangular channels) with not so tiny hydraulic diameters. A duct gas domain knows about Reynolds number, Valensi number and turbulence intensity. Reynolds number R_e is defined as in equation (8.4). Valensi number is

$$V_a = \frac{\rho \omega d_h^2}{4\mu} \tag{8.6}$$

It is related to the ratio of hydraulic diameter to viscous penetration depth $\delta_\nu = \sqrt{2\mu/(\omega\rho)})$ according to

$$d_h/\delta_\nu = \sqrt{2V_a} \tag{8.7}$$

Turbulence intensity $\mathcal{T}$ is discussed in detail in section 8.8. Duct domains also know about aspect ratio, which is a variable imported from the parent component. And they introduce the local-loss pressure-drop coefficient K (input variable Klocal), used in frictional pressure-drop equation (8.14).

8.5.3 Variable-Volume Gas Domains



A *variable-volume* gas domain is used within the generic-cylinder model component for modeling piston-cylinder spaces of stirling models — i.e. the volume between the pistons and the cylinders in which they ride. A variable-volume gas domain knows about turbulent Reynolds number and Valensi number. Valensi number V_a is the same as in equation (8.6) and turbulent Reynolds number R_t is defined in section 8.8. Flow area A is no longer constant. Rather, it varies with time in accordance with palette-generated volume displacements connected to the area faces of moving-part components.

8.5.4 Combustion-Space Gas Domains



A *combustion-space* gas domain is alternative to the above variable-volume gas domain, also available within the generic-cylinder model component. It is intended for modeling internal-combustion spaces in a simplified way. Input variable FQcombust specifies the combustion firing rate independently of fuel or air flow rates or combustion chemistry. You are responsible for ensuring the conditions for combustion are actually met. The combustion firing rate directly determines source term Q_c in the gas energy equation (*see* section 8.6.3)).

Combustion heating is modeled as a completely reversible process that does not involve entropy generation. It is perfectly feasible to specify a negative firing rate, in which case it amounts to a reversible cooling process.

As a software object, this component descends from the above variable-volume gas domain. This means it inherits all of the physics of the variable-volume gas domain in regard to turbulent Reynolds number, etc.

8.6 Gas Domain Theory

The following equations are somewhat nonstandard in the computational fluid dynamics literature because they are designed specifically for one-dimensional internal flows with space- and time-variable flow area. Most CFD textbooks take a more general world view. So, rather than just quote the equations, we are forced to do a bit of derivation — as concisely as possible. Our starting point is the general Navier-Stokes equations in integral form, as found in Peyret and Taylor [38], except that we shall neglect body forces and relax the prevailing convention that control volume v be fixed. We shall assume that v has fixed inlet and exit boundaries, but allow moving side boundaries — time-variable flow area. The side boundaries are impermeable so that flow enters and leaves only through the inlet and exit boundaries. A rubber tube with space- and time-variable cross section is a useful thought picture. The principle flow axis is the tube axis. In this control volume the integral gasdynamic equations are:

continuity

$$\frac{d}{dt}\int_v \rho\, dv + \int_s \rho \boldsymbol{n}\cdot\boldsymbol{V}_r\, ds = 0 \tag{8.8}$$

momentum

$$\frac{d}{dt}\int_v \rho \boldsymbol{V}\, dv + \int_s [(\boldsymbol{n}\cdot\boldsymbol{V}_r)\rho\boldsymbol{V} - \boldsymbol{n}\boldsymbol{\sigma}]\, ds = 0 \tag{8.9}$$

energy

$$\frac{d}{dt}\int_v \rho e\, dv + \int_s \boldsymbol{n}\cdot(\rho e\boldsymbol{V}_r - \boldsymbol{\sigma}\boldsymbol{V} - \boldsymbol{q})\, ds = 0 \tag{8.10}$$

where

e = $\varepsilon + u^2/2$; mass-specific total gas energy
$\boldsymbol{n}$ = unit outward normal of s
$\boldsymbol{q}$ = heat flux vector
s = surface of v
t = time
v = control volume
$\boldsymbol{V}$ = Newtonian-frame flow velocity vector
$\boldsymbol{V}_r$ = boundary-relative flow velocity vector
ε = mass-specific internal gas energy
ρ = gas density
$\boldsymbol{\sigma}$ = stress tensor

The continuity equation states, in English, that the time rate of change of mass in the control volume equals the rate at which mass leaves through its surface boundaries. By definition, $\boldsymbol{n} \cdot \boldsymbol{V}_r$ is zero along the side surfaces, so the mass enters and leaves only through the inlet and exit boundaries.

The momentum equation is a statement of Newton's second law of motion — the time rate of change of control-volume momentum, less the momentum leaving through its surface boundaries, equals the net force acting on all surfaces. We must be a bit more careful here because momentum (velocity) must be measured in reference to a fixed Galilean frame for Newton's second law to be valid. So the "$\boldsymbol{V}$" in $\rho\boldsymbol{V}$ must continue to be absolute, not relative velocity. We will keep this distinction in mind but, fortunately, it will not affect us much because it applies only to the transverse velocity components (associated with converging, diverging or moving side boundaries) and we are necessarily ignoring transverse components in the momentum equation in a one-dimensional model.

The energy equation states that the time rate of change of control-volume internal plus kinetic energy, less that leaving through its surface boundaries equals the net heat flux through its boundaries plus the mechanical work done on its boundaries. Mechanical work, like momentum, must be measured in an inertial frame so the "$\boldsymbol{V}$" in the tensor product $\boldsymbol{\sigma V}$ must also remain absolute, not relative.

We now convert our equations to one-dimensional differential equations in conservative form.

In all equations we replace dv in the time integrals with Adx, where A is flow area and x is the principle flow direction, take the limit as $\Delta x \to dx$ then divide through by dx. For example, the term $\frac{d}{dt}\int_v \rho\, dv$ in the momentum equation will transform to $\frac{\partial}{\partial t}(\rho A)$, and so forth.

For the surface integrals, we must consider the inlet and outlet boundaries separately from the side boundaries on a case-by-case basis.

Continuity Equation Surface integral $\int_s \rho \boldsymbol{n} \cdot \boldsymbol{V}_r\, ds$ in the continuity equation, after taking the limit as $\Delta x \to 0$ and dividing out dx, becomes $\frac{\partial}{\partial x}(\rho u A)$, where u is mean-flow velocity in the x direction.

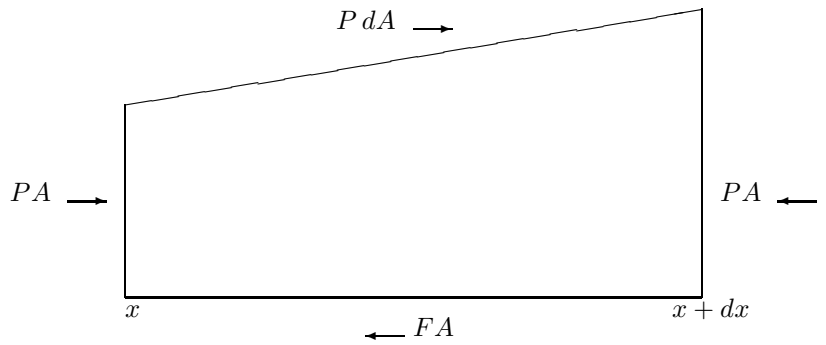


Figure 8.1: Axial surface forces $\int_s -\boldsymbol{N}\boldsymbol{\sigma}$ resolved into pressure and empirical flow-friction terms

Momentum Equation Surface integral $\int_s (\boldsymbol{n} \cdot \boldsymbol{V}_r)\rho \boldsymbol{V} ds$ in the momentum equation becomes, simply, $\frac{\partial}{\partial x}(u\rho u A)$ because there is no momentum flux through the side boundaries.

Integral $\int_s -\boldsymbol{n}\boldsymbol{\sigma} ds$, representing the forces applied to the surface, is more involved. Straight off, since this is a one-dimensional model, we will empiricise the stress tensor $\boldsymbol{\sigma}$ into thermodynamic pressure P and a viscous flow resistance term F which we may think of as the force per unit length per unit flow area due to surface shear stress. F amounts to a frictional pressure gradient. Then, pressure forces on the inlet and exit boundaries transform to $\frac{\partial}{\partial x}(PA) - P\frac{\partial A}{\partial x} = A\frac{\partial P}{\partial x}$. Viscous forces on the side boundaries transform into just $-FA$. The sketch in figure 8.1 may help make this clear:

Energy Equation Surface integral $\int_s \boldsymbol{n} \cdot (\rho e \boldsymbol{V}_r) ds$ in the energy equation becomes, simply, $\frac{\partial}{\partial x}(u\rho e A)$ because there is no energy flux through the side boundaries.

Integral $\int_s -\boldsymbol{n}\cdot\boldsymbol{\sigma}\boldsymbol{V} ds$ for the inlet and exit boundaries transforms to $\frac{\partial}{\partial x}(uPA)$. No F term is involved since F is applied to the side walls. For the side walls the integral transforms to $P\frac{\partial A}{\partial t}$, the PV work done on the wall. $\frac{\partial A}{\partial t}$ derives from the transverse velocity component of the moving walls. Once again, there is no F term. The reason is because axial velocity is zero at the walls where F is applied. See figure 8.2:

Integral $\int_s -\boldsymbol{n} \cdot \boldsymbol{q} ds$ transforms to a term $\frac{\partial}{\partial x}(qA)$ for the end boundaries, where q is axial heat flux. For the negative z boundary we introduce empirical film heat transfer per unit length Q_w to represent the integral.

In terms of these simplifications our equation set becomes:

continuity

$$\frac{\partial \rho A}{\partial t} + \frac{\partial \rho u A}{\partial x} = 0 \tag{8.11}$$

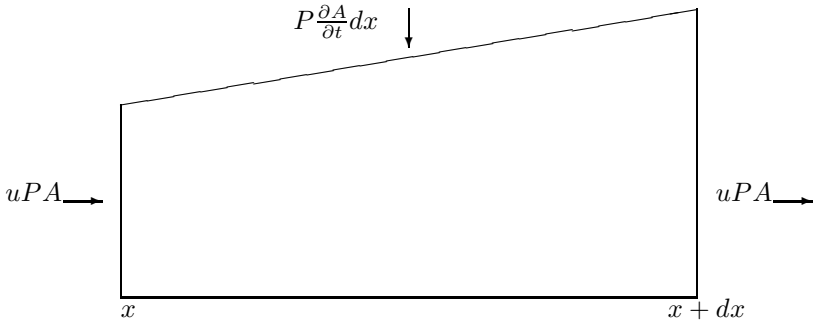


Figure 8.2: Surface work $\int_s -\boldsymbol{n} \cdot \boldsymbol{\sigma V}$ resolved into pressure terms

momentum

$$\frac{\partial \rho u A}{\partial t} + \frac{\partial u \rho u A}{\partial x} + \frac{\partial P}{\partial x} A - FA = 0 \tag{8.12}$$

gas energy

$$\frac{\partial \rho e A}{\partial t} + P\frac{\partial A}{\partial t} + \frac{\partial}{\partial x}(u\rho eA + uPA + q) - Q_w = 0 \tag{8.13}$$

The purpose of these gasdynamic equations is to determine three implicit solution variables, which we hereby choose to be ρ, $\rho u A$ and ρe. The reason for this choice is that all are roughly continuous across area discontinuities and, except for the factor A in the second, are standard in the field. Empirical terms F, Q_w and q are explained further below.

8.6.1 Viscous Pressure Gradient F

The term F in the momentum equation takes the place of the viscous terms in the Stokes stress tensor, which cannot be resolved directly in a one-dimensional model. The units of F are force per unit length, per unit flow area (same as $\frac{\partial P}{\partial x}$). F may be formulated in terms of Darcy friction factor f and total local loss coefficient K in a heat exchanger of hydraulic diameter d_h and length L as:

$$F = -\left(f/d_h + K/L\right)\rho u|u|/2 \tag{8.14}$$

Friction factor may be calculated as an empirical function of instantaneous local flow conditions, according to some published correlation. Loss coefficient is generally a fixed constant, equal to the sum of local loss coefficients for sharp-edged area changes, bends, etc.

Complex Formulation for Ducts

In the case of laminar flow in duct gas domains (*see* section 8.5.2), Sage allows for a phase shift between viscous pressure gradient and velocity by formulating F in terms of a *complex* wall-shear-stress function $\boldsymbol{s} = s_r + is_i$ instead if a conventional friction factor. The following formulation also accounts for the contribution of any time-mean (DC) velocity that may be present, which sometimes happens in novel Sage applications:

$$F = -(K/L)\rho u|u|/2 - (8\mu/d_h^2)s_{r0}u_m - (8\mu/d_h^2)\Re(\boldsymbol{s}\boldsymbol{u}) \tag{8.15}$$

Here, μ is viscosity and s_{r0} is the steady-flow limit for s_r. The factor $8\mu/d_h^2$ may also be written $2\rho\omega/V_a$. The first term on the right is a smeared-out local-loss per unit length carried over from the previous formulation. The last two terms amount to the superposition of the frictional pressure gradient solutions for a laminar steady-flow momentum equation and an laminar oscillating-flow momentum equation. Both equations are linear in u, so the superposition is valid.

In this formulation Sage's section-mean velocity $u(t)$ is broken into the time-mean value u_m plus a fluctuating component $u - u_m$ considered to be the real part of the complex-valued velocity

$$\boldsymbol{u}(t) \approx (u - u_m) - i\frac{\partial u}{\partial \omega t} \tag{8.16}$$

This is valid so long as velocity varies sinusoidally, because if $u - u_m = \Re\boldsymbol{u} = \Re(\boldsymbol{C}e^{i\omega t})$, then $-\frac{\partial u}{\partial \omega t} = \Re(-i\boldsymbol{C}e^{i\omega t}) = \Im(\boldsymbol{C}e^{i\omega t}) = \Im\boldsymbol{u}$.

The oscillating-flow wall-shear-stress function $\boldsymbol{s}$ is not a standard concept like friction factor. It is defined in reference [11], for incompressible laminar flow between parallel plates, as $-\boldsymbol{f}'_{wall}/\langle\boldsymbol{f}\rangle$ where $\boldsymbol{f}$ is the complex velocity profile function ($\mathbf{u}(\mathbf{y}) = \boldsymbol{f}(\mathbf{y})\mathbf{e}^{\mathbf{i}\omega\mathbf{t}}$). For other channel shapes, it can be related to the somewhat more standard $\boldsymbol{f}_\nu$ functions used in thermoacoustic theory [50] using

$$\boldsymbol{s} = \frac{iV_a}{2}\left(\frac{\boldsymbol{f}_\nu}{1 - \boldsymbol{f}_\nu}\right) \tag{8.17}$$

Both $\boldsymbol{s}$ and $\boldsymbol{f}_\nu$ are functions of Valensi number, or, as thermoacoustic practitioners prefer to say, functions of the ratio of some characteristic channel dimension to viscous penetration depth. (Viscous penetration depth is $\delta = \sqrt{2\mu/(\rho\omega)}$, which is related to Valensi number by $\sqrt{2V_a} = d_h/\delta$.)

The steady-flow (low-V_a) limit s_{r0} is related to the conventional Darcy friction factor by

$$f = \frac{16s_{r0}}{R_e} \tag{8.18}$$

In other words, $16s_{r0}$ is the constant factor that appears in the numerator of the laminar friction factor correlation. This relation can be used to cross-check $\boldsymbol{s}$ calculations against the laminar steady-flow friction-factor literature.

8.6.2 Film Heat Transfer Q_w

The term Q_w in the energy equation is the heat flow per unit length, through the negative z surface, due to film heat transfer. The normal way to formulate this is

$$Q_w = hS_x(T_w - T) = N_u(k/d_h)S_x(T_w - T) \tag{8.19}$$

where N_u is a Nusselt number, k is gas conductivity, d_h is hydraulic diameter, S_x is the wetted perimeter (wetted surface area per unit length) and $T_w - T$ is the temperature difference between the negative z surface and section-average. This is appropriate for heat transfer in phase with the film temperature difference. Matrix gas domains (*see* section 8.5.1) use this approach.

Complex Formulation for Cylinders

Variable-volume gas domains (*see* section 8.5.3), allow for a phase shift between heat transfer and film temperature difference by using a *complex* Nusselt number formulation

$$Q_w = (k/d_h)S_x\left[N_{u0}(T_w - T_m) - \Re(\boldsymbol{N}_u\boldsymbol{T})\right] \tag{8.20}$$

In this formulation Sage's gas temperature $T(t)$ is broken into the time-mean value T_m plus a fluctuating component $T - T_m$ considered to be the real part of the complex-valued temperature

$$\boldsymbol{T}(t) \approx (T - T_m) - i\frac{\partial T}{\partial \omega t} \tag{8.21}$$

This is valid so long as temperature varies sinusoidally, because if $T - T_m = \Re\boldsymbol{T} = \Re(\boldsymbol{C}e^{i\omega t})$, then $-\frac{\partial T}{\partial \omega t} = \Re(-i\boldsymbol{C}e^{i\omega t}) = \Im(\boldsymbol{C}e^{i\omega t}) = \Im\boldsymbol{T}$. The complex-valued Nusselt number $\boldsymbol{N}_u$ generally comes from some linear time-varying laminar theory [29]. The reason behind the form of equation (8.20) is that the terms $(1/d_h)N_{u0}(T_w - T_m)$ and $-(1/d_h)\boldsymbol{N}_u\boldsymbol{T}$ are, because of the way the Nusselt numbers are defined in such theories, just the normal temperature gradients $(\frac{\partial}{\partial y})$ at the wall for the steady and time-varying components of the theoretical temperature solution, which are additive. Complex Nusselt numbers have a significant imaginary component only for high Valensi number laminar flows, when hydraulic diameter is large compared to thermal diffusion length. In this case, the quantity $-(k/d_h)S_x\Re(\boldsymbol{N}_u\boldsymbol{T})$ in equation (8.20), which represents the physical component of the complex heat flow per unit length, may be phase shifted ahead of $T_m - T$ by as much as 45 degrees. For turbulent flows or for low Valensi numbers the complex heat-flow formulation essentially reduces to the conventional real formulation (8.19).

Complex Formulation for Ducts

Duct gas domains (*see* section 8.5.2), attempt to improve upon the complex formulation by breaking the fluctuating complex temperature $\boldsymbol{T}$ into compression-driven and advection-driven components $\boldsymbol{T}_c + \boldsymbol{T}_a$, arising from pressure variations and temperature gradients carried along by the flow, the two being independent mechanisms. The reason for decomposing temperature like this is

that the Nusselt numbers for the two sources of temperature variation differ, compression-driven temperature variations being roughly twice as effective at driving wall heat transfer as advection-driven temperature variations in the high Valensi-number limit — according to first-order oscillating-flow solutions [14, 49]. So to be as accurate as possible, duct gas domains formulate wall heat transfer as

$$Q_w = (k/d_h)S_x \left[N_{u0}(T_w - T_m) - \Re(\boldsymbol{N}_{uc}\boldsymbol{T}_c) - \Re(\boldsymbol{N}_{ua}\boldsymbol{T}_a)\right] \tag{8.22}$$

where $\boldsymbol{N}_{uc}$ is the compression-driven complex Nusselt number and $\boldsymbol{N}_{ua}$ is the advection-driven complex Nusselt number. The importance of this formulation is mainly for high Valensi number laminar flows. For turbulent flows or for low Valensi numbers, $\boldsymbol{N}_{uc}$ and $\boldsymbol{N}_{ua}$ are equal, or nearly equal, and real-valued so the heat-flow formulation essentially reduces to the previous complex formulation (8.20) or the real formulation (8.19).

Of course, the trick is how to evaluate $\boldsymbol{T}_c$ and $\boldsymbol{T}_a$, given that the only temperature available directly from the Sage solution is the combined result $\boldsymbol{T}$. This goes roughly as follows:

- Take $\boldsymbol{T}_c = \boldsymbol{s}_c\boldsymbol{T}_c^\infty$ where $\boldsymbol{s}_c$ is some complex scale factor which varies with Valensi number V_a and $\boldsymbol{T}_c^\infty$ is the high V_a limit of the compression-driven temperature variation

  $$\boldsymbol{T}_c^\infty = \frac{1}{\rho_m Cp}\boldsymbol{P_1} \tag{8.23}$$

  according to Swift's first-order acoustic solution [49]. $\boldsymbol{P}_1$ is the complex pressure variation, which may be calculated as $\boldsymbol{P}_1 = (P - P_m) - i\frac{\partial P}{\partial \omega t}$.

- Similarly, take $\boldsymbol{T}_a = \boldsymbol{s}_a\boldsymbol{T}_a^\infty$ where $\boldsymbol{s}_a$ is another complex scale factor and $\boldsymbol{T}_a^\infty$ is the high V_a limit of Swift's advection-driven temperature variation

  $$\boldsymbol{T}_a^\infty = \frac{1}{\omega}\frac{\partial T}{\partial x} i \langle \boldsymbol{u}_1 \rangle \tag{8.24}$$

  $\langle \boldsymbol{u}_1 \rangle$ is the complex section-average velocity $\langle \boldsymbol{u}_1 \rangle = u - i\frac{\partial u}{\partial \omega t}$.

- Assume the same scale factor $\boldsymbol{s}$ serves for both $\boldsymbol{s}_c$ and $\boldsymbol{s}_a$, which is justified by examination of Swift's exact solution for temperature (section-averaged) over the entire range of V_a.

- Calculate this common scale factor as

  $$\boldsymbol{s} = \frac{\boldsymbol{T}}{\boldsymbol{T}_c^\infty + \boldsymbol{T}_a^\infty} \tag{8.25}$$

  except when the denominator is zero or close to zero — whenever $\boldsymbol{T}_c^\infty$ and $\boldsymbol{T}_a^\infty$ are nearly equal and opposite. When this happens calculate $\boldsymbol{s}$ directly from an approximation based on Swift's theory, then $\boldsymbol{T}_c = \boldsymbol{s}\boldsymbol{T}_c^\infty$ as above, but $\boldsymbol{T}_a = \boldsymbol{T} - \boldsymbol{T}_c$ so the sum $\boldsymbol{T}_a + \boldsymbol{T}_c$ is always $\boldsymbol{T}$.

Explicit Surface Temperature

Although we might tend to think of T_w as the independent variable in the preceding heat-flow equations, from the point of view of Sage's solver it is actually Q_w that is the independent variable; implicitly solved so that T_w is continuous across the gas-solid connection at the negative z surface. Given Q_w, gas domains explicitly calculate negative z surface temperature as either

$$T_w = T + \left(\frac{d_h}{k} \frac{Q_w}{S_x N_u} \right) \tag{8.26}$$

or

$$T_w = T_m + \frac{1}{N_{u0}} \left(\frac{d_h}{k} \frac{Q_w}{S_x} + \Re(\boldsymbol{N}_u \boldsymbol{T}) \right) \tag{8.27}$$

or

$$T_w = T_m + \frac{1}{N_{u0}} \left(\frac{d_h}{k} \frac{Q_w}{S_x} + \Re(\boldsymbol{N}_{uc} \boldsymbol{T}_c) + \Re(\boldsymbol{N}_{ua} \boldsymbol{T}_a) \right) \tag{8.28}$$

depending on which heat-transfer formulation is used.

8.6.3 Combustion Heating Q_c

In rare instances where it may be appropriate to consider an internal heating source within the gas (*see* the combustion-space gas domain of section 8.5.4), an energy source Q_c is added into energy equation (8.13), on the same footing as the above film heat transfer term, except it represents internally generated heat per unit length. This energy input is treated as a simple energy source independent of any actual combustion chemistry.

8.6.4 Gas Axial-Conduction Heat Flux q

Term q in the gas energy equation is instantaneous axial heat flux which we may formulate as

$$q = -k_e \frac{\partial T}{\partial x} A \tag{8.29}$$

where k_e is the effective gas conductivity — which may exceed molecular conductivity due to turbulent enhancement. This may be formulated in terms of an axial-conductivity enhancement ratio N_k as:

$$q = -N_k k \frac{\partial T}{\partial x} A \tag{8.30}$$

8.6.5 Equations of State

Tying the gasdynamic equations together is an equation of state which serves to formulate the dependent variables P and T as functions of independent variables ρ, $\rho u A$ and ρe — specifically as $P(\rho, T)$ and $T(\rho, \rho e, \rho u A)$. The gas-property objects defined in the stirling root model provide these functions (see chapter 5).

8.6.6 Solution Method

The gasdynamic equations are discretized over a grid of points (x_i, t_j) uniformly spaced throughout the domain and over the cycle period. Upon this grid, variables ρ, ρuA and ρe, are solved implicitly or interpolated explicitly according to the logic of a *staggered-grid* formulation. The essential idea is to solve ρuA at alternate nodes from ρe and ρ to avoid solution instability and produce global conservation of mass, momentum and energy. Solution staggering is with respect to the spatial direction only.

The concept of control volumes is helpful for keeping things straight in the staggered-grid scheme. Control volume boundaries can be thought of as lying on alternate spatial nodes. One can think of it this way: ρuA is solved at the control volume boundaries, while ρ and ρe are solved at the midpoints. The entire domain comprises an integral number of control volumes or, including endpoints, an odd number of nodes greater than or equal to three. Global conservation of energy and mass automatically hold for the solved variables ρ and ρe if one is careful to use first-order central differencing in the continuity and energy equations so that, in effect, quantities leaving one control volume enter the adjacent one.

Interior Points

Those nodes lying strictly within the domain interior give rise to implicit finite-difference equations for the solved variables ρuA, ρ and ρe. Variables that are not solved at interior nodes are interpolated. In the case of ρuA and ρe, interpolation takes the form of a central polynomial interpolation of neighboring solved values. The degree of the interpolation (linear or cubic) can be specified in the Options|Sage dialog. In the case of ρ, interpolation is a weighted average of central interpolation and interpolation with the set of interpolation points shifted toward the upwind direction. This weights the interpolation toward the currently upwind direction in order to stabilize the solution. You can specify the relative fraction of upwind interpolation with input variable UpwindFrac, should the default value be too small. Without the upwind weighting, density and temperature profiles can sometimes show a static zig-zag instability in gas domains where there is not a lot of thermal diffusion to dampen the effect.

Endpoint Boundary Points

These are the nodes lying at the negative and positive domain boundaries — the flow inlets. Solved variables ρ, ρuA, ρe get special treatment at such nodes, depending on whether or not there is a flow connection to an adjoining gas domain or not. This is explained fully in the following section. Other variables are simply extrapolated from the interior.

Minimal Requirement for Resolving Pressure Drop

One idiosyncracy of the solution method is worthy of note: Pressure-drop, whether due to flow resistance or acceleration, will be ignored in any gas domain containing only one control volume. According to the staggered-grid solution scheme of the gas domain, the momentum equation will never be invoked when only one cell is present, leaving mass flow rate determined solely by Bernoulli's law in adjacent flow connectors. This behavior is not necessarily bad. It may even be desirable in some cases. But it is something to keep in mind.

The Mean Pressure Constraint

Although it may not be immediately apparent, the solution scheme so far outlined will result in a singular system of implicit equations. In physical terms, there is no mechanism to establish the mean pressure.

Resolving this dilemma requires a close inspection of the mass-continuity equations enforced in the gas domain interior. Taken together, these equations imply that the cyclic-net mass passing through the endpoint boundaries is zero, meaning that we cannot independently set boundary mass flow rate at both endpoint boundaries for all time nodes. Looking at this in another way, if we do specify boundary mass flow rates at all time nodes (such that cyclic-net mass through boundaries is zero) and enforce the mass-continuity equation at all but one interior cell, say (i_0, j_0), then mass continuity is automatically guaranteed for (i_0, j_0). This frees us to replace the mass continuity equation at (i_0, j_0) with a mean-pressure constraint instead — which is what we do.

The penultimate sentence in the preceding paragraph is really a mathematical theorem that follows from the conservation property of central spatial differencing in our staggered-grid scheme and the time-ring-differencing property

$$\sum_j \frac{\partial f}{\partial t}(i,j) = 0 \tag{8.31}$$

which holds no matter what. To show the mass-continuity equation indeed holds at (i_0, j_0) we first use the time ring differencing property to conclude that

$$\frac{\partial \rho A}{\partial t}(i_0,j_0) = -\sum_{j\neq j_0} \frac{\partial \rho A}{\partial t}(i_0,j) \tag{8.32}$$

we immediately replace the right-hand side using

$$-\sum_{j\neq j_0} \frac{\partial \rho A}{\partial t}(i_0,j) = \sum_{j\neq j_0} \frac{\partial \rho u A}{\partial x}(i_0,j) \tag{8.33}$$

$$= \sum_j \frac{\partial \rho u A}{\partial x}(i_0,j) - \frac{\partial \rho u A}{\partial x}(i_0,j_0) \tag{8.34}$$

the first part of the above equation follows from the presumed local mass continuity for all nodes but (i_0, j_0) and the second part is just regrouping. The first

term on the right may be further resolved into

$$\sum_j \frac{\partial \rho u A}{\partial x}(i_0, j) \propto \sum_j \rho u A(i_0 + 1, j) - \sum_j \rho u A(i_0 - 1, j) \tag{8.35}$$

$$\propto \sum_j \rho u A(2M, j) - \sum_j \rho u A(0, j) \tag{8.36}$$

$$\tag{8.37}$$

which follows from central differencing formula used in the computational grid, then summing the local mass-continuity equations over j at successive columns in the grid from $i_0 + 2$ to the positive boundary $i = 2M$ and from $i_0 - 2$ to the negative boundary $i = 0$. Combining the above results we are left with

$$\frac{\partial \rho A}{\partial t}(i_0, j_0) + \frac{\partial \rho u A}{\partial x}(i_0, j_0) \propto \sum_j \rho u A(2M, j) - \sum_j \rho u A(0, j) \tag{8.38}$$

which is the mass-continuity equation at (i_0, j_0) we are after, provided the right-hand side evaluates to zero, which it will if the net mass flow through the endpoint boundaries is zero.

In a system comprising many gas domains joined by flow connectors, the above result still applies to the system as a whole. So long as zero net mass flows through the ultimate boundaries of the system, we are free to replace a local mass-continuity equation at exactly one node (i_0, j_0) with a mean-pressure constraint. In a Sage model the gas domain to which the mean-pressure constraint applies is the one attached to the pressure-source component (*see* section 8.4).

8.7 Flow Connector Theory

Flow area may change abruptly between adjacent gas domains, requiring special consideration in Sage's one-dimensional formulation. Between every two gas domains that are connected together there is a special flow connector object that manages the connection behind the scenes. There are three conditions to satisfy: conservation of mass, momentum and energy. Conservation of mass is automatic because of the choice of mass flow rate $\rho u A$ as a solution variable. Sage uses momentum continuity (Bernoulli's law) as the implicit equation for solving mass flow rate in flow connectors. This is entirely consistent with the staggered-grid treatment for $\rho u A$ in the bounding gas domains. This leaves only energy conservation. Here Flow connectors pass responsibility back to the bounding gas domains but help them out by keeping track of the enthalpy on the other side of the connector and the thermal conduction through the connector. The bottom line is that because of a Bernoulli pressure change, mass density ρ and energy density ρe may jump slightly across the connector, but energy is always conserved. The inter-domain flow connector model resides in unit FlowCnct.

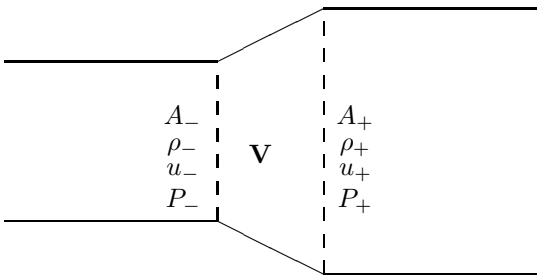


Figure 8.3: Control volume v used to evaluate momentum flow continuity; $A =$ flow area, $\rho =$ density, $u =$velocity, $P =$ pressure.

8.7.1 Bernoulli's Law

Momentum flow continuity is a limiting case of the integral-form momentum equation applied to the connector volume itself when the $-$ boundary approaches the $+$ boundary in the stream tube **V** shown in figure 8.3. The only terms that survive in the limit are the net momentum flow leaving through the boundaries $[u\rho uA]_+ - [u\rho uA]_- = \rho uA(u_+ - u_-)$ and the net pressure force acting on the boundaries $P_-A_- + P_m(A_+ - A_-) - P_+A_+$, the middle term being the axial component of force on the side walls with P_m some intermediate pressure. Taking $P_m = (P_- + P_+)/2$, expanding pressure terms and simplifying yields the following momentum flow continuity condition

$$\rho uA(u_+ - u_-) + (P_+ - P_-)A_m = 0 \tag{8.39}$$

where $A_m = (A_- + A_+)/2$. This can be put in the more familiar form of Bernoulli's law by using the previous mass flow continuity condition to replace ρuA with $\rho_m u_m A_m$, where $u_m = (u_- + u_+)/2$ and ρ_m is some intermediate density, leaving

$$P_+ - P_- = \rho_m u_m(u_- - u_+) = \rho_m \frac{u_-^2 - u_+^2}{2} \tag{8.40}$$

Flow connectors use this equation to implicitly determine mass flow rate ρuA through the connector, thereby ensuring that Bernoulli's law governs the pressure change across the connector.

8.7.2 Energy Continuity

Energy flow continuity is similarly derived by integrating the gas energy equation across a transition volume between adjacent components and then taking the limit as volume approaches zero. The result is

$$[(\rho e + P)uA + q]_+ - [(\rho e + P)uA + q]_- = 0 \tag{8.41}$$

A problem with this equation is that advected enthalpy $(\rho e + P)uA$ is mixed up with thermal conduction q, making them impossible to resolve individually

with single equation. So flow connectors separate the energy continuity equation into two parts, enthalpy continuity and conduction continuity, to be separately enforced

$$[(\rho e + P)uA]_+ - [(\rho e + P)uA]_- = 0 \tag{8.42}$$

$$q_+ - q_- = 0 \tag{8.43}$$

Enthalpy Part

How should enthalpy continuity equation (8.42) be implemented in Sage's solution scheme? First of all it is helpful to write stagnation enthalpy flow in the form

$$H = (\rho e + P)uA = \rho uA(\rho e + P)/\rho = (\rho uA)\, h \tag{8.44}$$

where h is the mass specific enthalpy of the gas, a thermodynamic state property. In this form it is clear that h must be conserved across the flow connector. The question is how? One problem is that there are two h's to determine, one on either side of the flow connector. Another problem is that h is not a primary solution variable. It is calculated in terms of primary solution variables ρ and ρe and dependent variable P. How can one equation (enthalpy continuity) determine all these variables? The answer is that it cannot. In the Sage solution scheme enthalpy continuity determines only the density at the downstream endpoint. Other variables are evaluated according to this table:

variable	**upstream endpoint**	**downstream endpoint**
ρ	extrapolated	enthalpy continuity
ρe	extrapolated	extrapolated
P	extrapolated	extrapolated

The word "extrapolated" in the above table means that the variable is extrapolated from its values at interior nodes located at the centers of computations cells. Density ρ at the downstream endpoint (which generally changes with time) is chosen as the variable used to enforce enthalpy continuity because density is a property carried with the flow and the principle of upstream causality suggests that upstream conditions determine downstream conditions, not vice-versa. In the case of pressure P it makes sense to extrapolate its value at the endpoints regardless of flow direction because pressure information propagates at sonic velocity which is *not* carried with the flow in the low subsonic flow range where all good stirling machines operate. In the case of volumetric energy density ρe it makes sense to extrapolate both endpoint values because that is consistent with Sage's staggered-grid solution scheme and also because ρe is directly proportional to pressure for ideal-gas conditions so it should be treated similarly.

Endpoint Temperature Glitch Sage calculates temperature at gas domain endpoints from the equation of state $T(\rho, \rho e, u)$. Under ideal gas conditions the above endpoint solution scheme produces continuous temperature across flow

connectors. Under non-ideal conditions there may be a slight discontinuity of temperature due to the fact all three variables ρ, ρe and P are extrapolated at the upstream endpoint and, according to the equation of state, only two are generally independent. In other words it is not guaranteed that extrapolated pressure is consistent with the equation of state as a function of ρ and ρe, except under ideal-gas conditions where P is essentially equivalent to ρe. Attempting to correct this problem by simply evaluating pressure according to the equation of state at the endpoint does not work because doing so tends to decouple the interior pressure solution (based on the momentum equation) from the enforcement of Bernoulli's law across the flow connector.

So the values of dependent solution variables P and T at domain endpoints (in fact at all computational cell boundaries) may be slightly inconsistent with the equation of state and primary solution variables ρ and ρe, as a result of interpolation. In the case of T this is more of an apparent problem than a real problem because none of the governing equations depends on T values at cell boundaries. In the case of P it is a real problem, though generally a slight one, because pressure values at cell boundaries affect the enthalpy flows (computed according to equation (8.44)) entering the computational cells, thereby affecting their energy equations. In principle any cell-boundary inconsistencies in P will vanish as the number of computational cells (NCell) increases.

There is a way around the problem by evaluating ρe at cell boundaries implicitly so that ρ, ρe and P are consistent with the equation of state. However doing so is inconsistent with the principles of the staggered-grid solution scheme and also slows down the solving time significantly, so it was judged not worth the bother.

Complication of Multiple Connections

Gas domains support multiple connections at their endpoint boundaries. The simplest such case is a Y-branch where one domain splits into two or two domains converge to one, although more complicated connections are possible. Gas domains manage this complexity by logically breaking their endpoint boundaries into a number of distinct *flow inlets*, one for each flow connector. The flow connectors communicate with the individual flow inlets which then communicate with the gas domain. Flow inlets are software objects encapsulated together with the gas domain.

Under this scheme, gas domains obtain their endpoint mass flow rate by polling all flow inlets to return the mass flow rate from their associated flow connectors and summing the results. Since mass is a conserved quantity and mass flow rate is a solution variable this is completely straight forward.

It is more complicated for gas domains to evaluate the correct density which, according to the above logic, must be based on enthalpy continuity when the gas domain is downstream of the flow connector. The problem is that a gas domain with multiple inlets can be simultaneously downstream of one flow connector while upstream of another. In other words, one inlet can see incoming flow while the other sees outgoing flow. The way gas domains manage this is by defining

a separate mixed density ρ_m at each endpoint to be passed by flow inlets to their flow connectors. This mixed density is in-turn used by the gas domain to determine its endpoint density, consistent with upstream causality and enthalpy continuity.

At this point it is convenient to denote variables at the gas domain endpoint by the subscript e and those within the individual flow inlets by subscript i.

Enthalpy conservation requires that the sum-total of stagnation enthalpy flows passing through the inlets must equal the stagnation enthalpy flow computed at the gas-domain endpoint and used in its interior solution. This latter quantity is always calculated from gas-domain endpoint variables as

$$H_e = [\rho u A]_e \frac{[\rho e + P]_e}{\rho_e} \tag{8.45}$$

In particular, ρ_e is extrapolated from the interior gas domain solution for outgoing mass flow rate $[\rho u A]_e$ and taken as the above mixed density ρ_m for incoming $[\rho u A]_e$. Enthalpy flows H_i through the individual flow inlets are imported from the associated flow connector whenever the individual inlet mass flow rate $[\rho u A]_i$ is directed into the gas domain. For $[\rho u A]_i$ directed out of the gas domain they are calculated as

$$H_i = [\rho u A]_i \frac{[\rho e + P]_e}{\rho_m} \tag{8.46}$$

Under these assumptions the condition of enthalpy-flow continuity may be written

$$0 = H_e - \sum_i H_i \tag{8.47}$$

$$= [\rho e + P]_e \left(\frac{(\rho u A)_e}{\rho_e} - \frac{1}{\rho_m} \sum_{\text{out}} (\rho u A)_i \right) - \sum_{\text{in}} H_i \tag{8.48}$$

where designations *in* (in-flowing) and *out* (out-flowing) are relative to the gas domain. We may write this as

$$\frac{1}{[\rho e + P]_e} \sum_{\text{in}} H_i = \frac{(\rho u A)_e}{\rho_e} - \frac{1}{\rho_m} \sum_{\text{out}} (\rho u A)_i \tag{8.49}$$

This equation can be solved explicitly for ρ_m. For incoming $(\rho u A)_e$ one may substitute ρ_m for ρ_e (according to the above conventions) and $\sum_{\text{in}} (\rho u A)_i + \sum_{\text{out}} (\rho u A)_i$ for $(\rho u A)_e$, in which case the solution becomes

$$\rho_m = [\rho e + P]_e \sum_{\text{in}} (\rho u A)_i \; / \sum_{\text{in}} H_i \tag{8.50}$$

To avoid a division by zero this equation applies only to strictly non-zero incoming flow. For outgoing or zero $(\rho u A)_e$ no such simplification is possible

because ρ_e is extrapolated from the interior gas domain solution. In this case the solution of equation (8.49) is

$$\rho_m = \sum_{\text{out}} (\rho u A)_i \; / \left(\frac{(\rho u A)_e}{\rho_e} - \frac{1}{[\rho e + P]_e} \sum_{\text{in}} H_i \right) \tag{8.51}$$

In the case where both $(\rho u A)_e$ and $\sum_{\text{in}} H_i$ are both zero gas domains assign $\rho_m = \rho_e$, to avoid an indeterminate form.

Note that ρ_e may take a sudden jump at $(\rho u A)_e$ flow reversal, an event worthy of note to all practitioners of iterative nonlinear equation solving. Discontinuities like this have been known to slow down the rate of convergence or even force the solver to hunt back and forth near the discontinuity. Fortunately, in this case, ρ_e only affects interior gas-domain solution variables by amounts scaled in proportion to $(\rho u A)_e$ which is zero at flow reversal. Therefore the discontinuity is conveniently wiped out. On the other hand, ρ_m (which is exported to flow connectors) is continuous at $(\rho u A)_e$ flow reversal, as can be seen by substituting zero for $(\rho u A)_e$ in equation (8.51) after which it reduces to equation (8.50).

Thermal Conduction Part

Thermal conduction continuity is much easier than enthalpy continuity since it does not involve so many solution variables or change with flow direction. Flow connectors look at the conductance (conductivity times area kA) and temperature gradients of the adjoining gas domains in order to decide the appropriate value of q allowed to "pass through" the connection. That is, flow connectors explicitly calculate q which adjoining gas domains use at their endpoints.

This approach differs from the way solid domains are connected in Sage. In solid domains the connector calculates q *implicitly* according to the requirement that temperature is continuous across the connection. In gas domains this is not necessary because temperature continuity is already established by enthalpy continuity and the equation of state in the adjoining gas domains. Not only is it not necessary but if temperature continuity were separately imposed by the thermal-conduction continuity then it would completely disrupt the solution.

However, explicitly calculating thermal conduction is not completely trivial. One problem is that the adjoining gas domains may have wildly different conductances. To avoid disrupting the solution in the low-conductance domain, flow connectors calculate q based on the smaller of the two conductances. A second problem is calculating a representative temperature gradient. Sages uses a finite-difference approximation based on the temperatures T_+, a distance dx_+ beyond the endpoint of the positive domain, and T_-, a distance dx_- before the endpoint of the negative domain. dx_+ and dx_- are the node spacings in the two gas-domain grids. The final formulation for calculating q is

$$q = -\min(kA_-, kA_+) \left(\frac{T_+ - T_-}{dx_+ + dx_-} \right) \tag{8.52}$$

Doing it this way has several important properties:

- Satisfies the requirement that $q = 0$ if either $kA = 0$
- Continuous if the smaller kA changes
- Preserves the temperature gradient for the case where the temperature gradients of the two gas domains are the same.

Another problem is the complication of multiple connections. To deal with this, gas domains use the same logical breakdown into flow inlets as previously discussed for enthalpy conservation. Each inlet is allocated an equal fraction of the total gas domain conductance which it passes to the flow connector. For example, for a gas domain with N endpoint connections, each inlet is allocated conductance kA/N. The gas domains then obtain their endpoint conduction by polling all inlets to return the q's from their associated connectors and summing the result. This is similar to the way they obtain endpoint mass flow rate and works for the same reason: thermal conduction is a conserved quantity.

Conduction blocked by default In evaluating kA_- or kA_+ values at gas domain endpoints Sage factors in the empirical multiplier input KmultBnd. The default value for KmultBnd is zero, which has the effect of blocking thermal conduction through the endpoints. One reason for this is that it speeds up the solution process by limiting the scope of solution-variable dependencies between gas domains. Another reason is that thermal conduction is generally negligible between most gas components in a stirling-cycle model. When you want to model conduction between gas domains be sure to set KmultBnd equal to 1 in both domains.

8.8 Turbulence Models

Knowing the state of turbulence is critical for computing the empirical relationships for friction factor f, Nusselt number N_u and axial conductivity ratio N_k used in the various gas-domain model components, of which there are three basic types: matrix, duct and variable-volume.

8.8.1 Matrix Gas Domains

Matrix gas domains presume quasi-steady flow, meaning that local Reynolds number R_e completely characterizes the turbulence state. Entrance effects and time-constants for boundary-layer growth are neglected. In most porous materials, the issue of turbulence is moot anyway since relatively simple formulations for friction factor f, Nusselt number N_u and enhanced axial conductivity ratio N_k typically apply over the entire Reynolds number range of interest. In non-porous matrix materials, though, (notably wrapped foils) the state of turbulence does matter because formulations for f, N_u, and N_k tend to be completely different for laminar vs turbulent flow.

When turbulence matters, matrix gas domains correlate dimensionless groups as a weighted average of the laminar and turbulent cases. For example friction factor is computed from a laminar value f_l and a turbulent value f_t as

$$f = (1 - W)f_l + W f_t \tag{8.53}$$

The idea is that weight function W varies smoothly between 0 and 1 as flow goes from laminar to turbulent. Smooth is better than abrupt for numerical convergence reasons.

The weight function is formulated in terms of a normalized and zero-shifted Reynolds number R_e^* defined as

$$R_e^* \equiv \frac{R_e - R_l}{R_t - R_l} \tag{8.54}$$

Here $R_l = 2300$ and $R_t = 10,000$ are the critical below-which-is-laminar and above-which-is-turbulent Reynolds numbers, conventional in the steady flow literature. Note that R_e^* varies from 0 to 1 in the transitional Reynolds-number range. The turbulence weight function is now defined as

$$W(R_e^*) = \begin{cases} 0 & \text{if } R_e^* \le 0 \\ 2R_e^{*2} & \text{if } 0 < R_e^* \le 1/2 \\ 1 - 2(1 - R_e^*)^2 & \text{if } 1/2 < R_e^* \le 1 \\ 1 & \text{if } 1 < R_e^* \end{cases} \tag{8.55}$$

This formulation is first-derivative continuous everywhere, which should be smooth enough for Sage.

8.8.2 Duct Gas Domains

In duct gas domains we no longer have the luxury of neglecting entrance effects or time constants for boundary-layer growth. In addition to Reynolds number R_e, the state of turbulence depends on Valensi number V_a, elapsed time since flow reversal and convective triggering, which is turbulence triggered by a slug of turbulent fluid that enters through one of the inlets and is carried to the point of observation by the mean flow.

Modeling turbulence in such ducts is a tall order for a one-dimensional model. Our goal is keep things as simple as possible, relying heavily on empirical observation. Researchers Terry Simon, Joerg Seume and others at the University of Minnesota [45], [47], [46], [48] have observed that transition to turbulence is delayed in proportion to $\sqrt{V_a}$ for large Valensi numbers and that flow is always laminar according to:

$$\text{always laminar if } R_e/R_l < \max(\sqrt{V_a/V_c}, 1) \tag{8.56}$$

where $R_l \approx 2300$ is a critical Reynolds number and $V_c \approx 10$ is a critical Valensi number, below which flow is essentially similar to steady flow. Above the *always laminar* condition, the Minnesota researchers observed turbulence transition due

mainly to convective triggering, with turbulence persisting until the time of flow reversal, but generally not beyond, leaving the velocity profile uniformly zero across the tube cross-section at the time of flow reversal. The Minnesota observations were generally confined to mean-flow tidal displacements greater than the tube length. University of Michigan and MIT researchers R. Akhavan, R. D. Kamm and A.H. Shapiro, [3], support these conclusions and make additional observations pertaining to the spontaneous transition to turbulence of fluid in the duct interior, not affected by disturbances propagating from the inlets. They observed that turbulence was generally delayed during most of the acceleration phase of the cycle but exploded suddenly near the end of this phase (just before or just after the time of peak velocity), with turbulent flow persisting throughout the deceleration phase, until the time of flow reversal. They saw no turbulence for peak $R_e/\sqrt{V_a}$ less than about 700 – 800, or so, consistent with the above *always laminar* condition for $V_a > V_c$. And when turbulence was present, they, like the Minnesotans, observed that the velocity profile was uniformly zeroed out at the time of flow reversal. Then during the following acceleration phase the velocity profile was very close to the exact solution for start-up from rest of laminar flow in a tube, until the time of turbulence transition. They attributed the delay in turbulence transition to the stabilizing effects of fluid acceleration, although the viewpoint here (and of the Minnesota researchers) is that it just a matter of the time required for the boundary layer to grow to an unstable thickness.

To attempt to capture all these effects, in duct gas domains there is a special turbulence intensity variable $\mathcal{T}$ added to the set of state variables and solved implicitly from its own partial differential equation. This turbulence variable $\mathcal{T}$ functions much like the weight function W in the matrix gas-domain model. Again using friction factor as an example, we compute local instantaneous friction factor from the weighted average of a laminar value f_l and a turbulent value f_t

$$f = (1 - \mathcal{T}) f_l + \mathcal{T} f_t \tag{8.57}$$

Turbulence intensity $\mathcal{T}$ is to vary smoothly between 0 and 1 as flow goes from laminar to turbulent.

We may think of $\mathcal{T}$ as a normalized version of $\rho\kappa$, where κ is the time-averaged turbulence kinetic energy per unit mass

$$\kappa = 1/2 \left\langle u'^2 + v'^2 + w'^2 \right\rangle \tag{8.58}$$

More precisely, we might think of $\mathcal{T}$ as roughly proportional to $\rho\kappa/(\rho_0 u^2) \propto \kappa/u^2$, where $u(x,t)$ is the mean-flow velocity. We can estimate the constant of proportionality by noting that for the fully turbulent state ($\mathcal{T} = 1$), the RMS velocity fluctuation ($\sqrt{2\kappa}$) is about $0.08u$, according to [48]. So a reasonable estimate for $\mathcal{T}$ would be

$$\mathcal{T} \approx \frac{2\kappa}{(0.08u)^2} \tag{8.59}$$

Because of the variable nature of the normalization quantity u^2, $\mathcal{T}$ is not, strictly speaking, a tangible quantity carried along with the flow (like $\rho\kappa$ itself is).

Nonetheless, it is convenient to overlook this for the moment and presume that $\mathcal{T}$ is governed by a one dimensional differential equation like our other gasdynamic equations

$$\frac{\partial \mathcal{T} A}{\partial t} + \underbrace{\frac{\partial \mathcal{T} u A}{\partial x}}_{\text{convection}} = \underbrace{G(R_e, V_a)}_{\text{generation}} - \underbrace{D(R_e, V_a)}_{\text{decay}} \tag{8.60}$$

The only problems are deciding on inlet boundary conditions and decay and generation terms to impart the desired properties to the solution $\mathcal{T}$ — namely, that $\mathcal{T}$ be 0 when flow is to be laminar, 1 when flow is be turbulent and somewhere in between during transition. The convection term automatically takes care of convective triggering.

Inlet Boundary Conditions

The University of Minnesota work suggests that incoming fluid is always turbulent. This turbulence is either the result of high upstream turbulence level or continuously generated by separation of incoming flow at the inlet. Denoting the incoming turbulence level by $\mathcal{T}_{in}$, it appears then that a value $\mathcal{T}_{in} = 1$ is reasonable, although the value is adjustable to any value between 0 and 1 with input variables TbInNeg and TbInPos, which define the incoming turbulence level at the two ends of the duct. For outgoing flow, it makes physical sense to extrapolate $\mathcal{T}$ at the end boundary from interior values, denoted by $\mathcal{E}(\mathcal{T})$. There is no problem with $\mathcal{T}$ discontinuity at flow reversal since only the product $\mathcal{T}uA$ (always zero at flow reversal) affects the interior solution. The value of $\mathcal{T}$ at either inlet boundary is computed as

$$\mathcal{T} = \begin{cases} \mathcal{T}_{in} & \text{inflow} \\ \mathcal{E}(\mathcal{T}) & \text{outflow} \end{cases} \tag{8.61}$$

Generation

The universal observation seems to be that in turbulent oscillating flow the steady velocity profile zeros out uniformly across the duct cross-section at flow reversal. Essentially, each half cycle begins anew from rest. Simon et. al. argue in [48] that under these conditions transition should occur when the momentum-thickness Reynolds number

$$R_\delta = \frac{u\delta}{\nu} \tag{8.62}$$

reaches a critical value. u is the instantaneous mean flow velocity, $\nu = \mu/\rho$ is kinematic viscosity and δ is the so-called momentum thickness, roughly the boundary-layer thickness, but defined more precisely in Schlichting ([42], equation 7.38, p. 141) for a flat plate as

$$\delta = \int_{y=0}^{\infty} \frac{u}{U_\infty}\left(1 - \frac{u}{U_\infty}\right) dy \tag{8.63}$$

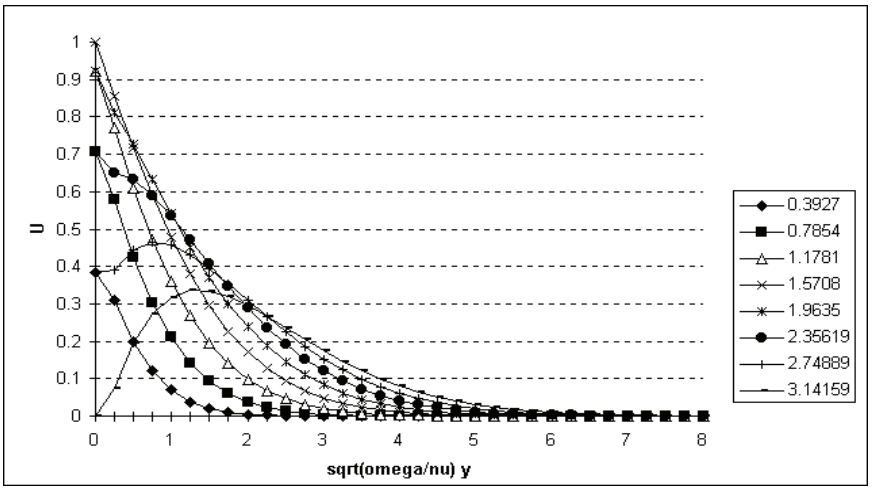


Figure 8.4: Velocity profiles at equally-spaced times $0 \leq \omega t \leq \pi$ for viscous flow governed by $\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2}$. Fluid initially at rest ($u(y,t) = 0$, for $t < 0$) with wall boundary condition $u(0,t) = \sin \omega t$ for $0 \leq \omega t \leq \pi$. From a numerical solution based on the Crank-Nicolson implicit method.

Here, y is the normal distance from the wall and $u(y)$ is velocity. Reference [2] shows that for undisturbed flow, R_δ may range between 1200 and 3000 from the start to end of transition, while the range of transition drops to R_δ between 200 and 400 with as little as 4% free-stream turbulence level.

The boundary-layer growth for a flat plate accelerated sinusoidally from rest can be seen in the velocity profiles shown in figure 8.4. In the figure the fluid is initially at rest with the wall suddenly started moving with velocity $u(0,t) = \sin \omega t$ at $\omega t = 0$. This is equivalent to the present problem of interest, which is a fixed wall with free-stream velocity suddenly started moving with velocity $u(\infty, t) = -\sin \omega t$ at $\omega t = 0$ (driven by a sinusoidal pressure gradient $\frac{\partial P}{\partial x}$), by the addition of $-\sin \omega t$ to the velocity solution everywhere. The advantage of the velocity profiles shown is that they better show the viscous boundary layer thickness.

The momentum thickness for the case of flow accelerated sinusoidally from rest is shown in figure 8.5. Except for a brief time at the beginning of the flow and lasting until well after the time of peak velocity, the momentum thickness grows linearly with time in close approximation to

$$\delta = \sqrt{\nu/\omega}(0.15 + 0.225\,\omega t) \text{ if } 0.3 \leq \omega t \leq 2.3 \tag{8.64}$$

So for sinusoidal flow starting from rest the momentum-thickness Reynolds number, and hence the point of turbulence transition, may be easily calculated. Transition so predicted is probably valid for $0 \leq \omega t \leq 2.3$, because in this

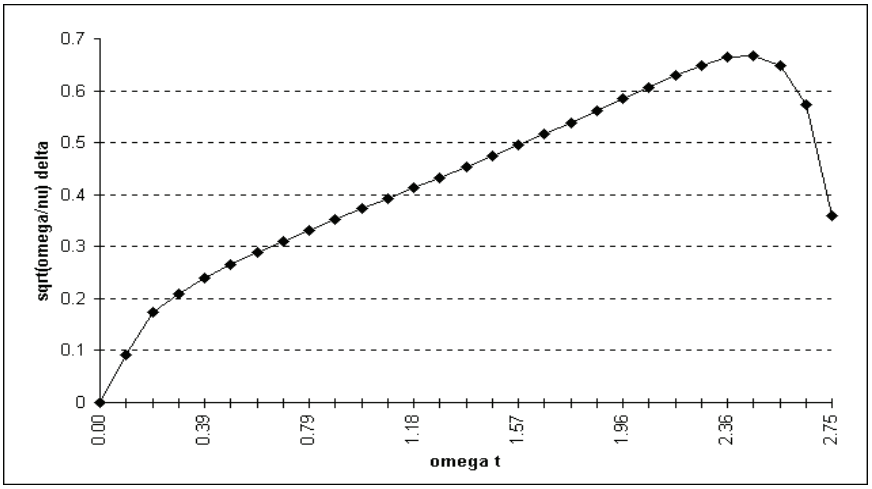


Figure 8.5: Growth of dimensionless momentum thickness $\sqrt{\omega/\nu}\delta$ with time ωt for flow accelerated sinusoidally from rest. Calculated according to formal definition (8.63) with integration performed numerically based on a finite difference solution of the velocity profile.

range the velocity profiles qualitatively resemble those of suddenly-accelerated flow ($u(\infty, t) = u_0$ for $\omega t \geq 0$), the conditions for which the correlation between momentum-thickness Reynolds number and turbulence transition were originally established. Beyond $\omega t = 2.3$, though, the velocity profile begins to show a pronounced maximum at some distance from the wall which, due to a mathematical technicality, has the effect of producing diminishing, then negative, momentum thicknesses even though the actual viscous-layer thickness continues to grow, as can be seen in figure 8.4. So the momentum-thickness Reynolds number cannot be used as a predictor of transition during the later stages of the flow half-cycle.

In order to discover critical value of the momentum-thickness Reynolds number, first write it in terms of the usual hydraulic-diameter Reynolds number as

$$R_\delta = Re\,\delta/d_h \tag{8.65}$$

Then replace factor δ/d_h by substituting approximation (8.64) for δ and expand into dimensionless variables, leaving

$$R_\delta \approx \frac{0.075 + 0.112\,\omega(t - t_0)}{\sqrt{V_a}} R_e \tag{8.66}$$

To avoid confusion, the elapsed time since flow reversal is written above and hereafter as $(t - t_0)$. And to save time, the numerator of the coefficient of R_e is hereafter written

$$F(t - t_0) \equiv 0.075 + 0.112\,\omega(t - t_0) \tag{8.67}$$

At this point, there is no need to work with R_δ directly at all. Taking the critical value of R_δ as an unknown constant $R_{\delta c}$, the turn-on time for turbulence generation may be stated

$$\text{laminar until } R_e > R_{\delta c}\frac{\sqrt{V_a}}{F(t-t_0)} \tag{8.68}$$

Even though approximation $F(t-t_0)$ is not valid for $\omega(t-t_0) > 2.3$, it will do no harm because transition will always occur before this, if at all, before or slightly after the time of peak velocity at $\omega(t-t_0) = \pi/2$.

The constant $R_{\delta c}$ may be evaluated according to the observed always-laminar condition (8.56) for high Valensi numbers ($V_a > V_c$), which states that for a cycle with peak Reynolds number R_{em} below about $730\sqrt{V_a}$, the flow remains laminar for the whole cycle, whereas for any higher R_{em}, flow becomes at least partly turbulent. On the other hand, substituting $R_{em}\sin(\omega(t-t_0))$ for R_e in momentum-thickness condition (8.68), the implication is that flow remains laminar until $R_{em}(\sin(\omega(t-t_0))F(t-t_0))$ exceeds $R_{\delta c}\sqrt{V_a}$. So, the largest R_{em} for which flow remains always laminar may be evaluated by setting $\omega(t-t_0) \approx 1.937$, where $\sin(\omega(t-t_0))F(t-t_0) \approx 0.274$ attains its maximum value. It follows that $730\sqrt{V_a}$ must equal $R_{\delta c}\sqrt{V_a}/0.274$, so that

$$R_{\delta c} \approx 200 \tag{8.69}$$

which is in reasonable agreement with the value reported in [2] for disturbed initial flow.

Transition criterion (8.68) applies only when Valensi number is above some lower limit because for $\sqrt{V_a}/(F(t-t_0))$ too low, the growth of the momentum-thickness will be limited by the duct dimensions. Below this, turbulence onset may be characterized by the well-known steady-flow criterion; namely, that flow is laminar until R_e rises above R_l. Just when this lower limit of $\sqrt{V_a}/(F(t-t_0))$ occurs may be taken as a constant c_1, conveniently determined as the value that makes the low- and high-Valensi number branches continuous in the following universal condition for the time of turbulence onset:

$$\text{if } \frac{\sqrt{V_a}}{F(t-t_0)} \geq c_1 \quad \text{then laminar until} \quad R_e = R_{\delta c}\frac{\sqrt{V_a}}{F(t-t_0)} \tag{8.70}$$

$$\text{if } \frac{\sqrt{V_a}}{F(t-t_0)} < c_1 \quad \text{then laminar until} \quad R_e > R_l \tag{8.71}$$

The value of c_1 is evidently $R_l/R_{\delta_c} \approx 2300/200$ or about 11.5. So the universal turbulence onset condition may be collapsed into the single condition

$$\text{laminar until } R_e > R_t \tag{8.72}$$

where R_t is the critical Reynolds number

$$R_t = 200\max(\frac{\sqrt{V_a}}{F(t-t_0)}, 11.5) \tag{8.73}$$

Required in evaluating R_t within a computational solution is the current value of $\omega(t-t_0)$, the dimensionless time since the previous flow reversal. This is not as easy as it may first sound because times of past events such as flow reversal are not usually available at any given time. The proposed method is one that works well enough whenever the velocity u is close to sinusoidal. The method is best illustrated by considering $u = \sin\omega t$, for which the times of flow reversal are $\omega t_0 = \{0, \pi \ldots\}$. Acceleration is then $\dot{u} = \omega\cos\omega t$ and the ratio $\omega u/\dot{u}$ is just $\tan\omega t$. A formula for flow-reversal time that always works is

$$\omega(t-t_0) = \begin{cases} \arctan(\omega u/\dot{u}) & \text{if } \omega u/\dot{u} \geq 0 \\ \pi - \arctan(-\omega u/\dot{u}) & \text{if } \omega u/\dot{u} < 0 \end{cases} \tag{8.74}$$

The advantage of this formulation is that both u and $\dot{u}$ are generally computable. For a coarser but faster approximation the arctan function may be approximated for non-negative arguments (the only arguments required) by

$$\arctan x \approx \frac{\pi}{2}\left(1 - \frac{1}{x+1}\right) \text{ if } x \geq 0 \tag{8.75}$$

the approximation being within about 0.08 rad (5 degrees) for any positive x.

Criterion (8.72) establishes the time when turbulence generation begins, but more useful in a computational model is the criterion for the entire time turbulence generation is active. Presumably, this is from the time of transition until the Reynolds number once again drops below the steady-flow relaminarization value $R_e = R_l$. The value R_l makes sense because, once turbulence is present, the velocity profile becomes similar to steady turbulent flow, no matter what the Valensi number, due to the mixing effects of the turbulent eddies. But for flows with large peak Reynolds numbers, the time at which R_e drops below R_l may be relatively close to the time of flow reversal, and entirely missed in a coarse computational grid! This problem can be remedied by slightly modifying the transitional Reynolds number to

$$R_t^* = \max(R_t, R_c) \tag{8.76}$$

where R_c is the value of Reynolds number at some *cutoff time* $\omega(t_c - t_0)$. The condition of turbulence duration may now be stated

$$\text{turbulent while } R_e > R_t^* \tag{8.77}$$

A cutoff time of $\omega(t_c - t_0) = 3/4\pi$, for example, would leave the last $\pi/4$ radians (45 degrees) prior to flow reversal available for relaminarization. (This follows because $R_e < R_c$ after the cutoff time and $R_c \leq R_t^*$.) The cost is a possible slight delay in turbulence onset and slight advance in decay onset.

There is, of course, the problem of estimating the cutoff Reynolds number R_c at a current time, not the cutoff time. There is no need to get too fancy here. Essentially, what is required is that the current value of R_e exceed R_c before the cutoff time, equal R_c at the cutoff time and be less than R_c after the cutoff time. So, presuming R_e varies as $\sin\omega(t - t_0)$ and using the approximation

$\sin(\pi-\tau) \approx \tau$, a reasonable approximation for cutoff Reynolds number in terms of the current Reynolds number is

$$R_c = R_e \frac{\pi - \omega(t_c - t_0)}{\pi - \omega(t - t_0)} \tag{8.78}$$

The numerator on the right side is simply a constant, like $\pi/4$, than need be calculated only once. The value $\omega(t-t_0)$ in the denominator is already available in the calculation of R_t. This approximation also has the advantage of producing $R_e > R_c$ for any time prior to the cutoff time so turbulence onset will always be governed by R_t, as it should.

But what is the rate of turbulence growth, during the transition period? Theoretical turbulence modelers (*see* Patankar [37]) suggest that this rate should be proportional to $\frac{\partial u}{\partial r}$, which scales as u/d_h, or in dimensionless terms $\omega R_e/V_a$. A pragmatic approach can be used to decide the constant of proportionality. It should be large enough so that, for relatively high values of R_e/V_a, turbulence will be generated quickly, but not too quickly lest the solution become unstable or too difficult to properly resolve in the computational grid. The following steady generation term will produce an initial growth of turbulence $\frac{\partial \tau}{\partial \omega t}$ at $\tau = 0$ of about 4 (same growth rate as the maximum rate of change of the 4th harmonic) for a value $R_e/V_a = 500$, which is near the high end of known stirling practice according to the survey published in [44]:

$$G = \begin{cases} 0.008 \frac{\omega A R_e}{V_a} \mathcal{S}(1-\mathcal{T}) & \text{if } R_e > R_t^* \\ 0 & \text{otherwise} \end{cases} \tag{8.79}$$

Smoothing function $\mathcal{S}$, defined to be $W(R_e/R_t^* - 1)$, where W is the weight function (8.55), makes the generation term come in smoothly at $R_e = R_t^*$. The factor $1 - \mathcal{T}$ serves to limit growth to $\mathcal{T} = 1$.

Decay

The simplest formulation is to have the decay term active whenever the generation term is inactive — whenever $R_e < R_t^*$. For calculating the actual rate of turbulence decay we can turn to a two-dimensional κ transport equation such as the one recommended in Launder and Spaulding ([28], p. 77). Simplified to its bare essentials by neglecting the diffusion and production terms, taking the Lagrangian point of view where $D\kappa/Dt = d\kappa/dt$ and dividing out density, the resulting equation is

$$\frac{d\kappa}{dt} \propto -\frac{1}{\ell} \kappa^{\frac{3}{2}} \tag{8.80}$$

where the constant of proportionality is to be determined from pragmatic reasoning, as usual, and ℓ is a length scale proportional to the Prandtl mixing length ℓ_m. (Prandtl mixing length is the length that makes turbulent shear stress come out right in the expression $\tau = \rho \ell_m^2 \frac{\partial u}{\partial y}^2$.) Assuming that mixing length scales with hydraulic diameter we have

$$\frac{d\kappa}{dt} \propto -\frac{1}{d_h} \kappa^{\frac{3}{2}} \tag{8.81}$$

We can write this in terms of $\mathcal{T}$ by substituting $0.0032u^2\mathcal{T}$ for κ (from equation (8.59)) and taking normalization velocity u to be the constant value u_l corresponding to $R_e = R_l$. Then replacing u_l/d_h by $\omega R_l/(4V_a)$, we have

$$\frac{d\mathcal{T}}{dt} \approx -c_d \frac{\omega R_l}{V_a} \mathcal{T}^{\frac{3}{2}} \tag{8.82}$$

Even though decay is active whenever R_e is below R_t^* (which may be much higher than the physically-correct value R_l), the rate of decay is taken as that appropriate for $R_e = R_l$ to avoid artificially increasing the rate of decay due to a computational artifact. The value of c_d should be chosen high enough that relaminarization takes place for all but the very highest values of R_{em}/V_a. A good calibration case is case b of [46] where relaminarization occurs for $R_{em} = 47,000$ and $V_a = 29$. The time for of relaminarization in this case is $\omega\delta t = \arcsin(R_l/R_{em}) \approx 0.05$. Under these conditions, a value of $c_d \approx 0.25$ will produce an initial decay rate $\frac{\partial\tau}{\partial\omega t}$ at $\tau = 1$ of about 20, which is the desired value. To avoid numerical difficulty with a discontinuous decay term, it is a good idea to smooth the above rate by a transition function $\mathcal{S}$ which smoothly varies from zero to one as R_e drops below R_t^*. A convenient smoothing function is $W(1 - R_e/R_t^*)$, where W is the weight function (8.55). The final decay term in equation (8.60) is then

$$D = \begin{cases} 0.25\frac{\omega A R_l}{V_a} \mathcal{S}\mathcal{T}^{\frac{3}{2}} & \text{if } R_e < R_t^* \\ 0 & \text{otherwise} \end{cases} \tag{8.83}$$

Of some concern is how long it takes incoming turbulence to decay to zero from the worst-case inlet value of $\mathcal{T}_{in} = 1$, for flow with low Reynolds number ($R_e < R_l$). A long delay would be unrealistic. The time for the mean flow to travel from the inlet to $x = d_h$ is d_h/u and the change in turbulence over this time is $\Delta\mathcal{T} \approx \frac{d\mathcal{T}}{dt}\frac{d_h}{u} \approx -R_l/R_e$, from equation (8.82) with $c_d = 0.25$. Even considering smoothing function $\mathcal{S}$, It follows that relaminarization occurs within about 1 pipe diameter of the inlet, at $R_e = R_l/2$, and proportionately less than this for lower R_e. This seems fast enough.

8.8.3 Variable Volume Gas Domains

In variable-volume gas domains, incoming flow separation produces turbulence which mixes throughout the cylinder space uniformly and dominates all other sources of turbulence. At least, that is the assumption.

In addition to the main $x-t$ solution grid, variable volume gas domains have a special t-only grid that maintains volume V and turbulence kinetic energy $\rho\kappa$ as state variables. Volume is just

$$V(t) = A(t)L \tag{8.84}$$

κ is again the time-averaged turbulence kinetic energy per unit mass defined by equation (8.58).

According to Gedeon [12] and [13], correlations for Nusselt number N_u and enhanced axial conduction ratio N_k may be formulated directly in terms of the turbulent Reynolds number R_t defined as

$$R_t = \rho\sqrt{\kappa}d_h/\mu \tag{8.85}$$

Volume-specific turbulence kinetic energy is just $\rho\kappa$. Because of the uniform mixing assumption there is no spatial variation in $\rho\kappa$. Therefore, a time ring (special time grid for periodic solutions where the first index is logically identified as the successor to the last index) rather than a space-time grid suffices for its storage and an ordinary time-differential equation suffices for its solution

$$\frac{\partial \rho\kappa V}{\partial t} + \underbrace{[\rho u A \kappa]_{\text{in}}^{\text{out}}}_{\text{inflow}} + \underbrace{D}_{\text{decay}} = 0 \tag{8.86}$$

Compared to the duct turbulence equation, we have replaced the convection term by the net turbulence kinetic energy entering through the flow inlet(s) and neglected the generation term.

Inlet Boundary Conditions

We assume that for incoming flow, the inlet mean-flow kinetic energy is converted entirely to turbulence kinetic energy — that is, $\kappa_i = 1/2u_i^2$, where the i subscript denotes an inlet value. Velocity u_i is just the mean-flow velocity in the upstream gas duct. For outgoing flow we naturally assume κ_i takes the interior value κ. We do not need to worry about the discontinuity in κ_i at the time of flow reversal because we are interested only in the product $\rho u A \kappa_i$, which is always zero at flow reversal. For inlets connected to multiple gas ducts, we calculate κ_i at each connection according to the previous equation. Then we sum $\rho u A \kappa_i$ over all connectors to obtain the net flow of turbulence kinetic energy through the inlet.

Decay

For turbulence decay we assume a similar form to the decay rate for duct flow, as given by equation (8.81), except we keep decay active at all times during the cycle. The following decay rate contains a calibrated leading factor that was fit to data for cylinder heat transfer with inflow-produced turbulence, as reported in [5].

$$D = 5.8\frac{\sqrt{\kappa}}{d_h}\rho\kappa V \tag{8.87}$$

Chapter 9

Canisters

Canisters are the containers into which you normally put a matrix-type heat-exchanger. But you can fill them with more than that. You can drop in a radiation transport path for modeling radiant heat transport from one end to the other, which makes sense if the contents are sufficiently transparent. Or, you can drop in a bar or distributed conductor for modeling axial conduction in the canister walls. Some canisters even allow you to nest other canisters within them, which can be useful for modeling pistons or displacers within cylinders (*see* chapter 12). All of these attributes arise as toolbox-created child components, at your discretion. The main purpose of canisters is to pass on critical geometry-dependent information to these child components when you create them. The child components within canisters and they way they are connected to the outside world are what really matters.

All canisters are high level components that appear in the toolbox of the root-level stirling model component. They are born without connectors of any sort. To get connectors at this level you must move them up from the various child components you create within. Variables common to all canisters are:

NCell : (dimensionless) The number of spatial nodes in the computational grids of all underlying model components having such grids. Changing this variable causes these computational grids to re-initialize themselves. NCell will affect the solution time and memory requirements, not to mention solution accuracy. Treat it gingerly by making only gradual, small changes.

Length : (real, m) Canister length L in x direction.

Avoid : (real or cubic spline, m^2) Canister void cross sectional (x-normal) area A_c.

Asolid : (real or cubic spline, m^2) Canister solid (wall) cross sectional area A_s.

Mass : (real, kg) Canister mass m, provided for use in constraints in case it is part of a reciprocating mass system.

Solid : (enumerated) Solid (wall) material.

Tinit : (cubic spline, W) Axial temperature distribution $T(x)$ where $x = 0$ is the negative endpoint and $x = 1$ is the positive endpoint.

The working gas type (properties) comes from the parent model component. All canister types calculate mass as

$$m = \rho_s \int_{dx} A_s \tag{9.1}$$

where ρ_s is the solid density. Dependent variables Avoid and Asolid are calculated differently for each of the descendant canister types below, in terms of geometrical inputs. When they do not vary with x (axial coordinate) they are single-valued real variables. When x variation is allowed they are cubic splines.

9.1 Tubular Canister



A tubular canister is a right circular cylinder, just as its icon suggests. Its additional variables are:

Din : (real, m) Canister internal diameter d_i.

Wcan : (real, m) Wall thickness w.

It calculates void cross-section area as

$$A_c = \frac{\pi}{4} d_i^2 \tag{9.2}$$

and solid cross-section area as

$$A_s = \frac{\pi}{4}(d_o^2 - d_i^2) \tag{9.3}$$

where $d_o = d_i + 2w$ is outer diameter.

There is a close descendant of a tubular canister known as a *nested* tubular canister. It shares the same icon as a tubular canister but is intended for nesting within another canister so that variables NCell, Length, and Tinit come from the parent canister and do not appear in its variable list. Its inner diameter Din becomes a dependent variable based on the assumption that its outer diameter equals the inner diameter of the parent canister. In other words, a nested canister always fits snugly within its parent canister.

9.2 Annular Canister



An annular canister is the volume enclosed by two concentric right circular cylinders. Its additional variables are:

Din : (real, m) Inner-wall internal diameter d_i.

Dout : (real, m) Outer-wall outside diameter d_o.

Win : (real, m) Inner-wall thickness w_i.

Win : (real, m) Outer-wall thickness w_o.

It calculates void cross-section area as

$$A_c = \frac{\pi}{4}\left((d_o - 2w_o)^2 - (d_i + 2w_i)^2\right) \tag{9.4}$$

and solid cross-section area as

$$A_s = \frac{\pi}{4}\left(d_o^2 - (d_o - 2w_o)^2 + (d_i + 2w_i)^2 - d_i^2\right) \tag{9.5}$$

There is a close descendant of an annular canister known as a *nested* annular canister. It shares the same icon as an annular canister but is intended for nesting within another canister so that variables NCell, Length, Dout and Tinit come from the parent canister and do not appear in its variable list. A nested canister always fits snugly within its parent canister

9.3 Tubular-Cone Canister



A tubular-cone canister is similar to an ordinary tubular canister except its internal diameter and wall thickness are specified by cubic splines. In other words, both may vary with axial position. The variation may be a good deal more complex than linear, depending on the number of interpolation pairs you specify for the variables:

Din : (cubic spline, m) Canister internal diameter d_i.

Wcan : (cubic spline, m) Wall thickness w.

A powerful option now presents itself: Din and Wcan, may be optimized! At least any data value in any of their defining interpolation pairs may be optimized. Say, for example, you want to find the optimum diameter profile for a tubular-cone regenerator canister specified by three interpolation pairs. Then you would select as optimized variables Din.FData.1, Din.FData.2 and Din.FData.3. Likewise, you may also map either variable.

Tubular-cone canisters calculate void cross-section area A_c and solid cross-section area A_s using equations (9.2) and (9.3), similar to ordinary tube canisters, except d_i and d_o are taken as functions of axial position. What actually happens is that tubular cones override variables Avoid and Asolid as dependent cubic splines. These splines are each defined by n data pairs $(x, A(x))$ equal spaced in x within the interval $[0, 1]$, where n is the larger of the number of data pairs in the specification of Din or Wcan. This is self-evident in the display form or listing.

There is a subtlety here worth noticing: The resolution of Avoid or Asolid are only as good as the resolution of Din or Wcan. If you specify a linear diameter

profile (a true cone) by the interpolation pairs $(0, 0.05)$ and $(1, 0.10)$ for Din, for example, you might think that A_c would then have a quadratic profile since it depends on d_i^2. You might be wrong. If the Avoid cubic spline is also defined by only two interpolation pairs, it will also have a linear profile. This is of some concern because the solutions of child model components within the canister (gas and solid domains) generally regard area as the fundamental quantity, not diameter. So your linear diameter profile is actually modeled as if it were a square-root profile. To increase accuracy you might want to specify Din with three, or more interpolation points, even though it may be linear.

9.4 Annular-Cone Canister



An annular-cone canister is an extension of an ordinary annular canister in the same way a tubular cone was an extension of an ordinary cone. Now all of the following inputs are specified as cubic splines:

Din : (cubic spline, m) Inner-wall internal diameter d_i.

Dout : (cubic spline, m) Outer-wall outside diameter d_o.

Win : (cubic spline, m) Inner-wall thickness w_i.

Win : (cubic spline, m) Outer-wall thickness w_o.

Any or all of these may be mapped or optimized by specifying the appropriate FData.n qualifier as discussed above.

Annular cones calculate void cross-section area A_c and solid cross-section area A_s using equations (9.4) and (9.5), similar to ordinary annular canisters, except all the d's and w's are taken as functions of axial position. This takes the form of overriding Avoid and Asolid as dependent cubic splines using the same logic as for tubular cones, discussed above. And the same cautionary remarks apply as well. Namely, that two-point linear diameter profiles may produce linear, not quadratic, area profiles.

Chapter 10

Heat Exchangers

Heat exchangers are the containers — or sub-containers if they are themselves found within a canister — into which you normally put a gas-domain and one or more thermal-solid model components. Their main purpose is to pass on critical geometry-dependent information to these child components when you create them. Stand-alone heat exchangers appear in the toolbox of the root-level stirling model component, while those needing containers appear in canister toolboxes. They are born without connectors of any sort. To get connectors you must move them up from the various child components you create within. Variables common to all heat exchangers are:

NCell : (dimensionless) The number of spatial nodes in the computational grids of all underlying model components having such grids. Changing this variable causes these computational grids to re-initialize themselves. NCell will affect the solution time and memory requirements, not to mention solution accuracy. Treat it gingerly by making only gradual, small changes.

Length : (real, m) Heat exchanger length L in x direction. For a variable-volume heat exchanger this is a *rough* estimate of the mean flow-path length of a typical fluid particle — the path length from the inlet to a wall boundary. No need to be overly exact because thermal performance is not very sensitive to L in variable-volume heat exchangers.

Roughness : (dimensionless) Relative roughness of the surface in contact with the gas, defined as the average height ϵ of surface irregularities divided by the hydraulic diameter d_h (i.e. tube diameter or twice the gap for parallel plates). Required as an input only for heat-exchangers having uniform flow passages, where it is important for determining turbulent flow resistance. According to [25], the height ϵ is the average distance between low and high points on the surface, rather than the average distance from high points to the mean surface. Multiplying Machinery's Handbook [35] peak-to-mean roughness values by two suggests the following absolute roughness values for various manufacturing processes:

Process	ϵ (microns)
cold-rolling	2 to 6
drawing	2 to 6
extruding	2 to 6
drilling	3 to 13

So for example, a drawn seamless tube with 3 mm ID could be expected to have a relative roughness in the range of 0.001 to 0.002. A drilled hole of the same diameter as much as 0.004.

Aflow : (real, m^2) Mean flow cross sectional (x-normal) area A_f. Void volume divided by length.

Asec : (real, m^2) Mean solid cross sectional area A_s. (*see* equation (7.15) in chapter 7)

Pwet : (real, m^2) Wetted perimeter S_x. The surface area per unit length for gas-to-solid heat transfer.

Tinit : (cubic spline, W) Axial temperature distribution $T(x)$ where $x = 0$ is the negative endpoint and $x = 1$ is the positive endpoint.

Matrix type heat exchangers have the additional variable

Porosity : (real, dimensionless) Porosity β. Void volume divided by total volume. Total volume is void plus solid volume.

Matrix heat exchangers calculate mean-flow cross section area A_f as the product of porosity and canister cross-sectional area A_c.

$$A_f = \beta A_c \tag{10.1}$$

Duct and variable-volume type heat exchangers have the additional variable:

Twall : (real, m) Wall thickness t_w. Usually a non-critical variable used in determining solid cross-sectional area for some thermal-solid child component.

Every heat exchanger component comes with a unique gas-domain component in its toolbox. This domain is a descendant of either the matrix, duct or variable-volume type of gas domain and shares an icon with its ancestor object. The gas-domain is unique because it knows the specific correlations for friction factor, Nusselt number and axial conductivity ratio appropriate for the parent heat exchanger type. These correlations are documented below as part of the heat-exchanger documentation. Correlations are in terms of the following variables:

d_h	hydraulic diameter defined by equation (8.5)
β	porosity; void / total volume
ϵ	average height of surface irregularities (*see* Roughness above)
V_a	Valensi number defined by equation (8.6)
R_e	Reynolds number defined by equation (8.4)
P_r	$c_p\mu/k$; Prandtl number
P_e	R_eP_r; Peclet number
R_t	turbulent Reynolds number defined by equation (8.85)

10.0.1 Solid Tortuosity

Matrix type heat exchangers implement a tortuosity function for solid-mode axial thermal conduction. This function accounts for the possibility of a complicated solid conduction path which typically zig-zags through the matrix and bridges a number of contact points between particles or wires. The tortuosity function is inherited by the wall-surface child component within the porous matrix. The child component calls the tortuosity function from within its computational grid and uses the result to correctly model solid-mode axial conduction. The wall-surface child component displays the average tortuosity as an output (*see* sections 7.6.3 and 7.7). The discussion below is a condensed version of a more detailed account in memorandum [19].

Definitions

Static conduction down a porous matrix (total inside the canister, not including the canister) can be represented in terms of an average matrix conductivity k_{av}

$$Q_t = k_{av}A_c\frac{dT}{dx} \tag{10.2}$$

where A_c is the matrix canister frontal area. Compare this to the static conductions down gas and solid uniform bars of cross-section areas βA_c and $(1-\beta)A_c$, corresponding to the void and filled parts of the matrix in the simplest of models

$$Q_g = k_g\beta A_c\frac{dT}{dx} \tag{10.3}$$

and

$$Q_s = k_s(1-\beta)A_c\frac{dT}{dx} \tag{10.4}$$

k_g and k_s are the gas and solid conductivities and β is matrix porosity (void fraction). Generally the total matrix conduction is less than the sum $Q_g + Q_s$. Sage models the static gas conduction according to the first equation (10.3) but discounts solid conduction by a tortuosity factor to make the total conduction come out right. The tortuosity factor is defined as

$$f_s \equiv \frac{Q_t - Q_g}{Q_s} \equiv \frac{k_{av} - k_g\beta}{k_s(1-\beta)} \tag{10.5}$$

Calibrated Formulation

The difficult thing is figuring out the average matrix conductivity k_{av}. Sage starts with a basic formula for the average thermal conduction of a matrix consisting of spheres embedded in a medium of different conductivity, based on an example in Carslaw and Jaeger ([6], pp. 426–428). Victorian physicist James Clerk Maxwell (of Maxwell's equations) actually derived the formula for induced magnetization in a sphere placed in a uniform external field but the governing equations are the same for heat flow. When reduced to the form of a tortuosity factor according to the above definition (10.5) and multiplied by a calibration factor of the form $(k_s/k_g)^m$ the final tortuosity formulation is

$$f_s = \left(\frac{k_s}{k_g}\right)^{m-1} \left[\frac{3(k_s/k_g - \beta) + (2 + k_s/k_g)\beta}{3(1-\beta) + (2 + k_s/k_g)\beta}\right] \tag{10.6}$$

The above equation with $m = 0$ corresponds to the original Maxwell conduction model. For particular matrices Sage uses calibration exponents fit to NIST data in [31] (*see* below under particular matrix heat exchanger types). The Maxwell conduction model assumes high contact resistance between individual solid elements of a matrix. So it is not suitable for wrapped-foil matrices or any matrix with continuous solid elements in the axial direction.

10.0.2 Gas Axial-Conduction Enhancement

For complicated gas flows with microscopic details smaller than a computational grid can resolve Sage resorts to the mechanism of enhanced gas conduction to model at least some of the macroscopic thermal energy transport (the remainder being modeled as bulk enthalpy transport, based on the section-mean gas temperature and mass flow rate). Enhanced gas conduction is represented in terms of a dimensionless ratio

$$N_k \equiv k_e/k_g \tag{10.7}$$

where k_e is the effective gas conductivity and k_g is the molecular conductivity. Sage resorts to enhanced gas conduction for turbulent flows and also for flows through porous material. The former is typically known as *turbulent conduction* and the latter as *thermal dispersion*. Of the two, thermal dispersion is probably the more unfamiliar concept.

The limiting value of N_k for zero flow velocity is $N_{k0} = 1.0$. This is consistent with the above tortuosity formulation for solid-mode conduction and results in the correct overall matrix static conduction (gas + solid).

Thermal Dispersion

One way to think about thermal dispersion is in terms of a experiment for an analagous phenomenon in mass diffusion. Take a screen matrix filled with fluid (say water) initially at rest and with a segment near the middle (bounded by

two planes) colored differently from the surrounding fluid. If you were to swim through the matrix in the axial direction you would swim through clear water, then suddenly enter colored water, then emerge a short time later into clear water again. Now you can do two things. Thing one is to just let the experiment sit there with the fluid at rest. The colored and clear water will gradually diffuse into each other, smoothing out the sharp-edged color boundary. This is the mass-transfer analog of molecular conduction. Thing two is to start oscillating the fluid back and forth through the matrix in the axial direction. In this case the color boundary diffuses much faster than before due to the interaction of high-velocity flow channels between the wires and low-velocity wakes behind the wires. This is the analog of thermal dispersion.

10.1 Woven Screen Matrix



The woven screen matrix component adds input variable:

Dwire : (real, m) Wire diameter d_w.

It calculates wetted perimeter S_x as

$$S_x = 4(1 - \beta)A_c/d_w \tag{10.8}$$

where A_c is canister cross-section area. The following correlations are from NASA Contractor Report [16] which documents oscillating-flow regenerator tests made at Ohio University for the following stainless-steel woven-screen samples, over a range of peak Reynolds numbers from 1 to 3400:

200 mesh per inch wire diameter 53.3 microns (0.0021 in), porosity 0.6232

100 mesh per inch wire diameter 55.9 microns (0.0022 in), porosity 0.7810

80 mesh per inch wire diameter 94.0 microns (0.0037 in), porosity 0.7102

10.1.1 Friction Factor

The screen friction factor is a variation of the Ergun equation [33] with the constant term replace by a term proportional to Reynolds number raised to a negative but small exponent to better track observed reality at high R_e.

$$f = 129/R_e + 2.91R_e^{-0.103} \tag{10.9}$$

10.1.2 Nusselt Number

$$N_u = (1 + 0.99P_e^{0.66})\beta^{1.79} \tag{10.10}$$

10.1.3 Axial-Conduction Enhancement

$$N_k = 1.0 + 0.50 P_e^{0.66} \beta^{-2.91} \tag{10.11}$$

10.1.4 Tortuosity

Solid-mode tortuosity is calculated from equation (10.6) with calibration exponent $m = 0.165$, based on fitting to data for stainless-steel and phosphor bronze screens in [31].

10.2 Random Fiber Matrix



The random fiber matrix component adds input variable:

Dfiber : (real, m) Fiber diameter d_w.

It calculates wetted perimeter using the same formula (10.8) as for woven screens, presuming circular cross-section fibers. For non-circular fibers the wetted perimeter and the following correlations will probably be wrong. As they will if the fibers are not oriented perpendicular to the axial flow direction. The correlations are based on oscillating-flow regenerator tests and data-reduction procedures reported in NASA Contractor Report [16] along with more recent tests for higher porosity matrices reported in memorandum [20]. The correlations are based porosity-dependence curve fits to the best-fit modeling parameters for the following round-wire test samples as documented in [21]:

2 mil Brunswick inconel, wire diameter 52.5 micron, porosity 0.688

1 mil Brunswick stainless steel, wire diameter 27.4 micron, porosity 0.820

12 micron Bekaert stainless steel, wire diameter 13.4 micron, porosity 0.897

30 micron Bekaert stainless steel, wire diameter 31.0 micron, porosities 0.85, 0.90, 0.93, 0.96 (samples cut from same sheet of material and pressed to different porosities)

Wire diameters are mean effective values based on electron microscope image measurements. The two Brunswick samples were tested in 1992–03 as documented in NASA Contractor report [16]. The 12 micron Bekaert sample was tested under DOE funding in 2003. "Bekaert" refers to Bekaert Corporation, the company who made the random fibers and continues to do so as of 2006. (Brunswick Corporation no longer is in the random fiber business.) The 30 micron Bekaert samples were tested in 2006 under funding provided by the NASA Headquarters Science Mission Directorate (via a NASA Glenn Research Center Grant to Cleveland State University). The peak Reynolds numbers for the

combined test ranged from about 1 to 3500. In the correlations below porosity dependence is correlated in terms of

$$\alpha \equiv \frac{\beta}{1-\beta}$$

10.2.1 Friction Factor

$$f = a_1/R_e + a_2 R_e^{a_3} \tag{10.12}$$

where

$$\begin{aligned} a_1 &= 22.7\alpha + 92.3 \\ a_2 &= 0.168\alpha + 4.05 \\ a_3 &= -0.00406\alpha - 0.0759 \end{aligned}$$

10.2.2 Nusselt Number

$$N_u = 1 + b_1 P_e^{b_2} \tag{10.13}$$

where

$$\begin{aligned} b_1 &= (0.00288\alpha + 0.310)\alpha \\ b_2 &= -0.00875\alpha + 0.631 \end{aligned}$$

10.2.3 Axial-Conduction Enhancement

$$N_k = 1.0 + b_3 P_e^{b_2} \tag{10.14}$$

where

$$\begin{aligned} b_2 &= \text{same as above} \\ b_3 &= 1.9 \end{aligned}$$

10.2.4 Tortuosity

Solid-mode tortuosity is calculated the same as for screens, namely from equation (10.6) with calibration exponent $m = 0.165$. There is no data to back this up but the wires in random-fibers matrices are typically oriented perpendicular to the axial flow direction, similar to screens.

10.3 Packed Sphere Matrix



The packed sphere matrix component adds input variable:

Dsphere : (real, m) Sphere or particle diameter d_s.

It calculates wetted perimeter S_x as

$$S_x = 6(1-\beta)A_c/d_s \tag{10.15}$$

where A_c is canister cross-section area, presuming spherical particles. For non-spherical particles the wetted perimeter and the following correlations will probably be wrong.

The following correlations are based on unpublished oscillating-flow regenerator tests using essentially the same hardware and procedures as documented in NASA Contractor Report [16]. The material tested was 173 micron diameter spherical lead particles (173 micron mean diameter ± 14 micron standard deviation) packed to porosities in the range 0.38 to 0.43.

10.3.1 Friction Factor

$$f = \left(157/R_e + 5.15R_e^{-0.137}\right)\left(\beta/0.39\right)^{3.48} \tag{10.16}$$

10.3.2 Nusselt Number

$$N_u = 1 + 0.48P_e^{0.65} \tag{10.17}$$

10.3.3 Axial-Conduction Enhancement

$$N_k = 1.0 + 3.00P_e^{0.65} \tag{10.18}$$

10.3.4 Tortuosity

Solid-mode tortuosity is calculated from equation (10.6) with calibration exponent $m = 0.145$, based on fitting to data for lead, stainless-steel and copper spheres in [31].

10.4 Wrapped Foil Matrix



The wrapped foil matrix component adds input variables:

Gap : (real, m) Gap g between foil layers.

Thk : (real, m) Foil thickness b.

It calculates wetted perimeter S_x as

$$S_x = 2A_f/g \tag{10.19}$$

where A_f is flow cross-section area according to equation (10.1). It calculates porosity as

$$\beta = \frac{1}{1 + b/g} \tag{10.20}$$

10.4.1 Friction Factor

The laminar case is based on exact theory for fully-developed flow and the turbulent case is the Altshul approximation to the Colebrook formula with a shape correction factor of 1.1, as recommended in [25] (pp. 8, 22).

- Case laminar

$$f = 96R_e^{-1} \tag{10.21}$$

- Case turbulent

$$f = 0.121\left(\epsilon/d_h + 68/R_e\right)^{0.25} \tag{10.22}$$

10.4.2 Nusselt Number

The laminar case is based on exact theory for fully-developed flow, uniform heat flux, and the turbulent case is from figure 7.4 p. 146 of reference [27].

- Case laminar

$$N_u = 8.23 \tag{10.23}$$

- Case turbulent

$$N_u = 0.025R_e^{0.79}P_r^{0.33} \tag{10.24}$$

10.4.3 Axial-Conduction Enhancement

Same as equations (10.42) and (10.43) for tubular ducts since the flow microstructure is likely to be similar.

10.4.4 Tortuosity

Solid-mode tortuosity is taken as $f_s = 1$ (no tortuosity at all) which corresponds to an uninterrupted solid conduction path.

10.5 Generic Matrix



A generic matrix component allows you to specify your own coefficients and exponents within generic formulations for friction factor, Nusselt number, axial conduction enhancement and tortuosity. The generic matrix component adds input variables:

Dhyd : (real, m) Hydraulic diameter d_h as in equation (8.5).

FsC1 : (real, dimensionless) c_1 in tortuosity formulation (10.29).

FsC2 : (real, dimensionless) c_2 in tortuosity factor formulation (10.29).

FdM : (real, dimensionless) m in tortuosity formulation (10.29).

It calculates wetted perimeter S_x as

$$S_x = 4A_f/d_h \tag{10.25}$$

where A_f is mean flow area βA_c (porosity $\times$ canister area). Its associated gas domain adds input variables:

FdC1 : (real, dimensionless) c_1 in friction factor formulation (10.26).

FdC2 : (real, dimensionless) c_2 in friction factor formulation (10.26).

FdC3 : (real, dimensionless) c_3 in friction factor formulation (10.26).

FdM : (real, dimensionless) m in friction factor formulation (10.26).

NuC1 : (real, dimensionless) c_1 in Nusselt number formulation (10.27).

NuC2 : (real, dimensionless) c_2 in Nusselt number formulation (10.27).

NuM : (real, dimensionless) m in Nusselt number formulation (10.27).

NuN : (real, dimensionless) n in Nusselt number formulation (10.27).

KrC1 : (real, dimensionless) c_1 in axial conduction enhancement formulation (10.28).

KrC2 : (real, dimensionless) c_2 in axial conduction enhancement formulation (10.28).

KrM : (real, dimensionless) m in axial conduction enhancement formulation (10.28).

KrN : (real, dimensionless) n in axial conduction enhancement formulation (10.28).

10.5.1 Friction Factor

The generic matrix friction factor is

$$f = c_1 + c_2 R_e^m + c_3/R_e \tag{10.26}$$

where c_1, c_2, c_3 and m are input variables.

10.5.2 Nusselt Number

The generic matrix Nusselt number is

$$N_u = c_1 + c_2 R_e^m P_r^n \tag{10.27}$$

where c_1, c_2, m, n are input variables.

10.5.3 Axial-Conduction Enhancement

The generic matrix axial conductivity enhancement ratio is

$$N_k = c_1 + c_2 R_e^m P_r^n \tag{10.28}$$

where c_1, c_2, m, n are input variables.

10.5.4 Tortuosity

The generic matrix tortuosity factor is

$$f_s = c_1 + c_2 \left(\frac{k_s}{k_g}\right)^{m-1} \left[\frac{3(k_s/k_g - \beta) + (2 + k_s/k_g)\beta}{3(1-\beta) + (2 + k_s/k_g)\beta}\right] \tag{10.29}$$

where c_1, c_2 and m are input variables. This reduces to the Maxwell tortuosity factor of equation (10.6) for $c_1 = 0$ and $c_2 = 1$. If that is not appropriate you can specify specify a constant tortuosity with $c_1 > 0$ and $c_2 = 0$.

10.6 Tube Bundle



The tubular heat exchanger adds input variables:

Dtube : (real, m) Tube internal diameter d_i.

Ntube : (real, m) Tube number n. This can take on non-integer values so that it can be optimized.

It calculates flow area as

$$A_f = n\frac{\pi}{4} d_i^2 \tag{10.30}$$

It calculates solid cross-section area in terms of outside diameter d_o ($d_o = d_i + 2t_w$ where t_w is wall thickness) as

$$A_s = n\frac{\pi}{4}(d_o^2 - d_i^2) \tag{10.31}$$

And it calculates wetted perimeter as

$$S_x = n\pi d_i \tag{10.32}$$

10.6.1 Friction Factor

The laminar case is formulated in terms of the real and imaginary components of the oscillating-flow wall-shear-stress function as discussed in section (8.6.1) with $\boldsymbol{s}$ derived from the thermoacoustic function $\boldsymbol{f}_\nu$ for circular tubes in reference [50]. The approximations below are defined piecewise from low-V_a and high-V_a asymptotic limits, with the Valensi number dividing one branch from the other chosen to make the two branches continuous. These approximations are much easier to compute than the exact functions (complex Bessel functions) and are accurate to within a worst-case relative error on the order of 25%. The low V_a laminar equivalent friction factor is $f = 64/R_e$, according to equation (8.18). The turbulent case is the Altshul approximation to the Colebrook formula as recommended in [25] (p. 8).

- Case laminar

$$s_r = \begin{cases} 4 & \text{if } V_a \leq 32 \\ \sqrt{V_a/2} & \text{if } V_a > 32 \end{cases} \tag{10.33}$$

$$s_i = \begin{cases} V_a/6 & \text{if } V_a \leq 18 \\ \sqrt{V_a/2} & \text{if } V_a > 18 \end{cases} \tag{10.34}$$

- Case turbulent

$$f = 0.11\left(\epsilon/d_h + 68/R_e\right)^{0.25} \tag{10.35}$$

10.6.2 Nusselt Number

The laminar case is broken into separate parts N_{u0}, $\boldsymbol{N}_{uc}$ and $\boldsymbol{N}_{ua}$ — real and complex Nusselt numbers that apply respectively to the steady, compression-driven and advection-driven components of the gas-to-wall temperature difference, as discussed in section (8.6.2). These are based on piecewise-continuous blended averages of low and high Valensi-number solutions presented in [14] and also derived from [49]. The real-valued turbulent case is based on reference [22] and there is no distinction made for the various components of temperature difference.

- Case laminar

$$N_{u0} = 6.0 \tag{10.36}$$

$$\Re(\boldsymbol{N}_{uc}) = \begin{cases} 6.0 & \text{if } \sqrt{2V_aP_r} < 6.0 \\ \sqrt{2V_aP_r} & \text{otherwise} \end{cases} \tag{10.37}$$

$$\Im(\boldsymbol{N}_{uc}) = \begin{cases} \frac{1}{5}V_aP_r & \text{if } V_aP_r < 5\sqrt{2V_aP_r} \\ \sqrt{2V_aP_r} & \text{otherwise} \end{cases} \tag{10.38}$$

$$\Re(\boldsymbol{N}_{ua}) = \begin{cases} 4.2 & \text{if } \sqrt{2V_aP_r} < 8.4 \\ \frac{1}{2}\sqrt{2V_aP_r} & \text{otherwise} \end{cases} \tag{10.39}$$

$$\Im(\boldsymbol{N}_{ua}) = \begin{cases} \frac{1}{10}V_aP_r & \text{if } V_aP_r < 5\sqrt{2V_aP_r} \\ \frac{1}{2}\sqrt{2V_aP_r} & \text{otherwise} \end{cases} \tag{10.40}$$

- Case turbulent

$$N_u = 0.036\, R_e^{0.8}(L/d_h)^{-0.055}P_r^{0.33} \tag{10.41}$$

10.6.3 Axial-Conduction Enhancement

The laminar case assumes no enhancement beyond molecular conduction. The turbulent case is based on a theoretical analysis that goes roughly as follows: In turbulent flow the total fluid shear stress can be represented as $\tau = (\mu + \mu_t)\frac{\partial u}{\partial y}$ where μ is the molecular viscosity, μ_t is the so-called turbulent viscosity and y is the coordinate normal to the wall. For turbulent tube flow, μ_t can be determined from the Blasius friction factor ($f = 0.316R_e^{-0.25}$) and the known velocity profile. Then, using a *Reynolds-analogy* type of argument, the enhanced (turbulent) axial conductivity ratio (N_k) can be equated with $P_r\mu_t/\mu$. After the details have been worked out the resulting correlation is

- Case laminar

$$N_k = 1 \tag{10.42}$$

- Case turbulent

$$N_k = 0.022R_e^{0.75}P_r \tag{10.43}$$

10.7 Rectangular Channels



The rectangular channel heat exchanger adds input variables:

Wchan : (real, m) Channel inner width w_i.

Hchan : (real, m) Channel inner height h_i.

Nchan : (real, m) Channel number n. This can take on non-integer values so that it can be optimized.

It calculates flow area as

$$A_f = nw_i h_i \tag{10.44}$$

It calculates solid cross-section area in terms of outer channel width w_o and height h_o ($w_o = w_i + 2b$, $h_o = h_i + 2t_w$, where t_w is wall thickness) as

$$A_s = n(w_o h_o - w_i h_i) \tag{10.45}$$

And it calculates wetted perimeter as

$$S_x = 2n(w_i + h_i) \tag{10.46}$$

Aspect ratio a is defined to be the smaller of w_i/h_i or h_i/w_i. In terms of a, there are two geometrical parameters used in the following correlations

$$\begin{aligned} b &= 1.47 - 1.48a + 0.92a^2 \\ c &= 0.438 + 0.562(1-a)^3 \end{aligned}$$

10.7.1 Friction Factor

The laminar case is similar to the case for tubes, with the factor b representing the continuum between parallel plates and rectangular channels. The value of b is about 1.5 (1.47 actually) for an aspect ratio of zero (parallel plates) and 0.91 for an aspect ratio of one (square channels). The approximations below reduce to the above *tube* approximations for $b = 1.0$ and to the approximations for parallel plates for $b = 1.5$ (except for a 10% error in the low-V_a branch for S_i). The turbulent case is the Altshul equation, the same as for tubes.

- Case laminar

$$s_r = \begin{cases} 4b & \text{if } V_a \le 32b^2 \\ \sqrt{V_a/2} & \text{if } V_a > 32b^2 \end{cases} \tag{10.47}$$

$$s_i = \begin{cases} V_a/(6b) & \text{if } V_a \le 18b^2 \\ \sqrt{V_a/2} & \text{if } V_a > 18b^2 \end{cases} \tag{10.48}$$

- Case turbulent

$$f = 0.11\left(\epsilon/d_h + 68/R_e\right)^{0.25} \tag{10.49}$$

Factor b is the result of a curve-fitting process for laminar friction factors, traceable to Terry Heames and associates at the Argonne National Laboratory [22]. The fact that b is not quite 1.0 for square channels is, perhaps, a reasonable accounting of the differences between square channels and circular tubes. The fact that b is not exactly 1.5 in the parallel-plate limit is likely a minor curve-fitting error.

10.7.2 Nusselt Number

The laminar case is similar to that for tubes with different low-Valensi limits. The turbulent case comes from reference [22].

- Case laminar

$$N_{u0} = 10.0\,c \tag{10.50}$$

$$\Re(\boldsymbol{N}_{uc}) = \begin{cases} 10.0\,c & \text{if } \sqrt{2V_aP_r} < 10.0\,c \\ \sqrt{2V_aP_r} & \text{otherwise} \end{cases} \tag{10.51}$$

$$\Im(\boldsymbol{N}_{uc}) = \begin{cases} \frac{1}{5}V_aP_r & \text{if } V_aP_r < 5\sqrt{2V_aP_r} \\ \sqrt{2V_aP_r} & \text{otherwise} \end{cases} \tag{10.52}$$

$$\Re(\boldsymbol{N}_{ua}) = \begin{cases} 8.1\,c & \text{if } \sqrt{2V_aP_r} < 16.2\,c \\ \frac{1}{2}\sqrt{2V_aP_r} & \text{otherwise} \end{cases} \tag{10.53}$$

$$\Im(\boldsymbol{N}_{ua}) = \begin{cases} \frac{1}{10}V_aP_r & \text{if } V_aP_r < 5\sqrt{2V_aP_r} \\ \frac{1}{2}\sqrt{2V_aP_r} & \text{otherwise} \end{cases} \tag{10.54}$$

- Case turbulent

$$N_u = 0.035R_e^{0.75}P_r^{0.33} \tag{10.55}$$

In the laminar case, 10.0 and 8.1 are the recommended steady-flow compression-driven and advection-driven values for parallel plates in [14] and scale factor c is the above function of aspect ratio, which was obtained by curve fitting to data in Kays and London [27] (fig 6-1, p. 120) for laminar rectangular-tube Nusselt numbers, expressed as a fraction of the parallel-plate value, under constant heat-flux boundary conditions.

10.7.3 Axial-Conduction Enhancement

Same as equations (10.42) and (10.43) for tubular ducts since the flow microstructure is likely to be similar.

10.8 Rectangular Fins



The rectangular fin heat exchanger is a minor variation of the rectangular duct heat exchanger that uses the same type of gas domain but alters the solid variables passed to any child thermal-solid components. It presumes the rectangular passages are formed by alternating rectangular fins. Accordingly, it adds the input variable:

Tfin : (real, m) Fin thickness t_f.

and overrides wall thickness t_w (Twall) to be a dependent variable, calculated as $t_f/2$. That is, half the fin thickness is presumed to belong to the side-wall of the adjacent duct. It calculates solid cross-section area as the fin-only cross section

$$A_s = nt_fh_i \tag{10.56}$$

where h_i (Hchan) is the fin height. This gives any distributed-conductor child model components the appropriate cross-section area for modeling fin conduction (see chapter 7). Note that solid cross section area does not include end walls and fin height is presumed equal to channel height.

The correlations for f, N_u and N_k are the same as for rectangular channel heat exchangers.

10.9 Generic Cylinder



The generic-cylinder variable-volume component adds variables:

Swet : (real, m^2) Time-mean wetted surface S_0. Not extremely critical since it has only a secondary effect on the cycle pressure through the heat transfer between the gas and wall. But there is also a thermodynamic irreversibility associated with this heat transfer which affects overall efficiency somewhat. As a rule of thumb, a value of Swet within fifty percent is good enough. Too much accuracy is not justified because of the relatively large error band for the Nusselt number correlation.

Volume : (real, m^3) Baseline volume V_0 when all volume-displacement attachments are zero. Each volume displacement adds to the baseline volume. If all volume displacements are phasors (attached to sinusoidal moving part with zero mean) then V_0 is the same as the time-mean cylinder volume. Of primary importance since it directly effects the pressure ratio of the machine. Relative accuracy within a few percent is recommended.

Sratio : (real, m) An output variable which expresses the time-mean surface area S_0 as a fraction of the surface area of a minimal-surface right-circular cylinder with the same V_0 ($S_{min} = 5.54 V_0^{2/3}$). Useful for seeing if the Swet you have entered is reasonable compared to the Volume, or for constraining when optimizing Swet and Volume. If Sratio $\ll 1$, then your Swet is too small. If Sratio $\gg 1$, then either your Swet is too large or your volume has a lot of extra wetted surface area within it. Sratio has no effect on the numerical solution so you may safely ignore it if you want.

Generic cylinders calculate time-mean flow area as

$$A_f = V_0 / L \tag{10.57}$$

They calculate solid cross-section area for use in thermal-solid child model components as

$$A_s = t_w S_0 / L \tag{10.58}$$

where t_w is the wall thickness input and L is the length input.

The time-varying part of volume comes from volume-displacement child model components of the associated variable-volume gas domain. The assumption is that the length remains fixed but the flow area varies as a function of time to produce the required volume variation. This makes the equations simpler.

10.9.1 Length

So what is the *length* anyway? There are two ways to look at it.

The first is in terms of the underlying gas domain model which consists of a one dimensional passage of some length and cross-sectional flow area, as shown in section 8.2. So the problem is how to represent a variable volume space as a one dimensional passage? Here is one way: You specify the length as an input variable — say the average length of a streamline between the entrance to the space and its stagnation point on the opposite wall or moving piston face. Sage then computes the cross-sectional area from the above formula. Although this may seem somewhat arbitrary, it does not matter much. The length may vary by an order of magnitude or so without producing much effect on the simulation outputs. This is because nothing much in the variable-volume gas domain model depends on length or flow area. Volume and wetted surface are the main things that matter. So the recommendation is to pick a rough value of Length scaled to the machine you are modeling and then forget about it.

The second way to look at length is in terms of the underlying solid wall model. In the event the wall models an external conduction path where axial conduction is an important feature, then this length might override the gas domain length in modeling importance. In most cases, though, the solid wall is thermally isolated from everything except the gas, in which case the solid conduction length is irrelevant.

10.9.2 Friction Factor

The friction factor is set to $f = 0$ as a reasonable approximation for all likely applications.

10.9.3 Nusselt Number

This correlation is taken from personal memoranda [12, 13, 15] which derived a formulation for Nusselt number in stirling cylinder spaces, based on an analytic expression for wall heat flux in a gas undergoing compression and expansion between parallel walls. In place of molecular conductivity, the derivation used a turbulent conductivity, assumed to increase linearly with distance from the wall. Flow into the cylinder was the source of turbulence.

The correlation is broken into real and complex parts N_{u0}, and $\boldsymbol{N}_u$ that apply to the steady and fluctuating components of the gas-to-wall temperature difference, as discussed in section (8.6.2). The steady part is, for lack of any compelling evidence to the contrary, just the steady laminar Nusselt number for tubes scaled by the spatial-mean ratio of turbulent to molecular conductivity N_k. The complex part reflects a calibration of the analytic expression for fluctuating wall heat flux on the basis of experimental data for cylinders with inflow produced turbulence reported in [5].

$$N_{u0} = 6.0 N_k \tag{10.59}$$

$$\Re(\boldsymbol{N}_u) = \begin{cases} \sqrt{2V_aP_r} & \text{if } R_t \le 7.7\sqrt{2V_aP_r} \\ \dfrac{0.13R_t}{\ln\left(\dfrac{0.35\,R_t}{\sqrt{2V_aP_r}}\right)} & \text{otherwise} \end{cases} \tag{10.60}$$

$$\Im(\boldsymbol{N}_u) = \frac{\tanh(0.40/\eta)}{1+3\,\eta}\Re(\boldsymbol{N}_u) \tag{10.61}$$

where

$$\eta = \sqrt{\frac{R_t}{7.8V_aP_r}} \tag{10.62}$$

10.9.4 Axial-Conduction Enhancement

According to a variation of the Prandtl mixing length hypothesis, turbulent viscosity may be expressed in terms of turbulent Reynolds number as

$$\frac{\mu_t}{\mu} = R_t\frac{\ell}{d_h} \tag{10.63}$$

where ℓ is a mixing length scale. Based on personal memorandum [15], a mean representative value for ℓ/d_h in cylinder spaces is

$$\frac{<\ell>}{d_h} \approx 0.014 \tag{10.64}$$

Assuming that the turbulent Prandtl number is unity, the equation for enhanced (turbulent) conductivity ratio becomes simply

$$N_k = 1 + 0.014R_t \tag{10.65}$$

The leading constant covers the limiting case of $R_t = 0$.

Chapter 11

Miscellaneous Parasitics

Parasitic components model some of the stirling-cycle loss mechanisms that do not fall into the class of thermal solids or gas components.

11.1 Annulus (Shuttle/Seal/Appendix)



The annulus component is intended for modeling shuttle heat transfer and clearance-seal or appendix-gap gas flow. Shuttle heat transfer is built-in while annular-gap gas flow is optional by way of toolbox-created child model components. It is actually a descendant of the basic heat-exchanger model component (chapter 10) and inherits several variables from that type. It is born as a child to a parent model component which supplies a geometrical context in terms of two tubular canisters (piston and cylinder), including their common length, diameter, initial temperature distribution, relative displacement as a phasor variable and thermal properties. An annulus adds the variables:

XNeg : (real, dimensionless) The dimensionless x position of the annulus negative endpoint in the parent cylinder coordinate frame. $x = 0$ is the parent-cylinder negative endpoint and $x = 1$ is the positive endpoint. Default is XNeg = 0 for the case where the annulus negative endpoint coincides with the parent cylinder negative endpoint.

XPos : (real, dimensionless) Same as XNeg except for the positive annulus endpoint. Default is XPos = 1 for the case where the annulus positive endpoint coincides with the parent cylinder positive endpoint.

Gap : (real, m) Radial clearance gap g between the parent cylinders.

Length : (real, m) Annulus length. Not necessarily the same as the parent cylinder length, reflecting inputs XNeg and XPos.

TsNeg : (real, K) Temperature at annulus negative endpoint.

TsPos : (real, K) Temperature at annulus positive endpoint.

QNeg : (real, W) Shuttle heat flow at annulus negative endpoint.

QPos : (real, W) Shuttle heat flow at annulus positive endpoint.

AEQ : (real, W) Available energy loss to shuttle heat flow.

Within the annulus toolbox are tools for creating an optional foil-matrix gas-domain with one of three wall models: isothermal, conductive surface or floating. The isothermal wall is for modeling thermally-anchored seals (such as piston seals), although you must remember to account for any wall heat flow in your overall energy balance. The conductive surface is for thermally-isolated walls, such as appendix-gap walls. It inherits solid material and cross-sectional area information from the parent canisters and models the solid conduction path corresponding to their combined cross sections over that portion of their length specified by the XNeg and XPos input variables. The floating model is also an option for thermally-isolated walls, except that it does not model axial conduction. It can be useful on occasion to avoid for double-accounting for solid axial heat flows when they are separately considered in the parent canister models. However, floating walls with NCell > 2 often converge to unrealistic temperature distributions due to poor thermal anchoring.

You connect the shuttle-heat-transfer part of an annulus into the grand scheme of your stirling model much like you would a simple bar conductor — typically, at both ends to a temperature source (sink) objects via built-in steady-heat-flow boundary connectors. If you create a child gas domain, its flow connectors are connected to other gas domains as is appropriate in your model scheme. Two annulus instances may be ganged together in series to model a seal-appendix combination.

11.1.1 Theory

Shuttle heat transfer is a somewhat complicated process whereby, in the presence of a temperature gradient, the moving wall of an annulus alternately picks up from and deposits heat to the stationary wall, with the net effect of transporting heat in the axial direction. This picking up and depositing of heat is governed by the time-varying heat conduction in the transverse direction across the gas-filled annular gap, between and into the two walls. The shuttle heat transfer loss q_s may be presented in the form of an enhancement to molecular gas conduction q_m within the clearance gap, although it is implemented separately from the gas in Sage. The following formula is due to Rios [41]:

$$\frac{q_s}{q_m} = \frac{1}{2}\left(x/g\right)^2 \frac{1+\lambda}{1+\lambda^2} \tag{11.1}$$

where

g	=	radial clearance gap
k	=	molecular gas conductivity
k_s	=	wall solid conductivity
q_m	=	$-k\frac{\partial T}{\partial x}$ molecular conduction heat flux
x	=	relative displacement amplitude
λ	=	$1+(k/g)[(\delta_s/k_s)_1+(\delta_s/k_s)_2]$; the subscripts refer to the inner and outer walls
δ_s	=	$\sqrt{2k_s/(\omega\rho_s c_s)}$; thermal penetration depth
c_s	=	wall solid specific heat
ρ_s	=	wall solid density
ω	=	angular frequency

The above formula applies to the case of *thick* walls, where the wall thickness w is larger than the thermal penetration depth δ_s, for each wall. When this is not the case, shuttle heat transfer is reduced, because a thin wall cannot hold as much heat as a thick wall in the transverse heat-flow part of the shuttle mechanism. In fact a thin wall can hold only about w/δ_s as much heat. So, whenever one of the walls is thin ($w/\delta_s < 1$), Sage scales the above formula by the ratio of the smaller of w/δ_s for the two walls.

11.2 Radiation Transport Path



The radiation transport path embodies a simple model of radiant heat transfer between two gray bodies. It is plug-compatible with a simple bar conductor and comes with two built-in steady heat flow connectors at either x end for connection to a point temperature source and sink. It is born as a child to a parent model component which supplies the radiation frontal area. A radiation transport path has variables:

Emmis : (real, dimensionless) Gray-body emissivity ϵ. A number between 0 and 1. $\epsilon = 1$ for an ideal black body. $\epsilon \approx 0$ for a silvered surface.

TsNeg : (real, K) Temperature T_n of negative x surface.

TsPos : (real, K) Temperature T_p of positive x surface.

QNeg : (real, W) Radiation heat flow at negative x surface.

QPos : (real, W) Radiation heat flow at positive x surface.

AEQ : (real, W) Available energy loss to radiation heat flow.

11.2.1 Theory

A radiation transport path models in-vacuum radiation energy transport between two gray body surfaces with same area and emissivity, presuming all the

radiation leaving one surface reaches the other and vice-versa. This may be formulated as

$$q_r = \sigma \frac{\epsilon}{2 - \epsilon} \left(T_n^4 - T_p^4 \right) \tag{11.2}$$

where

q_r	=	radiation heat flux
T_n	=	temperature of negative x surface
T_p	=	temperature of positive x surface
ϵ	=	gray-body emissivity
σ	=	5.669E–8 W/(m^2 K^4); Boltzmann constant

Chapter 12

Composite Model Components

Composite model components are high level components that work like other model components except that they have built in child models that you cannot destroy. They automatically take care of some of the geometrical relationships among these built-in components that would otherwise prove bothersome to you.

12.1 Piston-Cylinder Composites

Piston-cylinder composites appear in the root-level stirling model toolbox. They all represent some form of piston within a cylinder, with capability to model the interaction between the two. Built-in are two nested tubular canister child components serving as the moving piston (shell) and the fixed cylinder (liner). The descendants below also build-in various types of moving-part child components.

You treat these built-in components just as if you had created them yourself. For example, you may put a regenerator matrix and gas domain within the moving shell, to serve as a regenerative displacer (chapter 9). Or you may add area attachments to the moving part (chapter 6) for connection to variable-volume gas domains (chapter 8).

Piston-cylinder composites also come with a toolbox with which you may create your own custom child model components. Available are springs and dampers for attachment to the moving part and a shuttle-heat-transfer annulus for modeling the interaction between the shell and liner, including possible gas flow.

Piston-cylinder composites are born without connectors of any sort. Any and all connectors come from the child components within and must be moved up to the top level if you wish to connect them to other root-level model components.

Variables common to all piston-cylinder composites are:

NCell : (dimensionless) The number of spatial nodes in the computational grids of all underlying model components having such grids. Changing this variable causes these computational grids to re-initialize themselves. NCell will affect the solution time and memory requirements, not to mention solution accuracy. Treat it gingerly by making only gradual, small changes.

Length : (real, m) Shell and liner length in x direction.

Dliner : (real, m) Cylinder liner outside diameter, reflecting wall thickness of liner

Dshell : (real, m) Moving shell outside diameter.

Tinit : (cubic spline, W) Axial temperature distribution $T(x)$ where $x = 0$ is the negative endpoint and $x = 1$ is the positive endpoint.

The working gas type (properties) comes from the root model component.

12.1.1 Phasor Free-Piston and Cylinder



This component adds a built-in phasor reciprocating mass intended as the basis for free-piston or free-displacer modeling. Choose this composite model when you want to solve for the limit-cycle sinusoidal response according to Newton's equation of motion. Since overstroking is a distinct possibility, with potential negative volumes in variable-volume gas domains, you will want to make sure you know the proper combination of area attachments, springs, etc., before choosing this model component.

12.1.2 Time-Ring Free-Piston and Cylinder



Similar to the above except with a time-ring reciprocating mass. This allows you to resolve higher harmonics in the resulting motion which may be of interest in certain cases. The number of nodes in the ring is according to the root-model input variable NTnode.

12.1.3 Phasor Constrained Piston and Cylinder



This component adds a built-in phasor constrained piston (or displacer) intended for first-approximation kinematic modeling or free-piston design work. The sinusoidal piston motion is now specified as input. Overstroking is no longer a problem with this option. And by proper use of constraints you can set up an optimization problem to solve for the proper combination of area attachments, springs, etc., to make the piston run properly when you switch to free-piston mode.

For example, you could attach a spring to the piston and set up one area attachment to represent a drive rod. You could then optimize the spring stiffness and rod area subject to the constraints that the real and imaginary parts of the required boundary force (F.real and F.imag) are both zero. You could chose to

optimize other combinations of *free-piston* variables as well, provided you wind up with two independent degrees of freedom.

12.1.4 Time-Ring Constrained Piston and Cylinder



Similar to the above except with a time-ring constrained piston. This model component is mainly intended for modeling the higher harmonics of kinematic motion, provided you know the correct Fourier-series coefficients. You could also use it for free-piston design work, as above, except there is no easy way to balance the higher harmonics within the equation of motion.

12.1.5 Simple-Crank Piston and Cylinder



This component adds a built-in simple-crank kinematic piston intended for modeling a kinematically driven piston using geometrical mechanism inputs. With these inputs you have a handle on optimizing the mechanism or introducing constraints in terms of them.

12.1.6 Phasor Free-Piston and Free-Cylinder



This component is similar to the phasor free-piston component except it adds another built-in phasor reciprocating mass representing the moving cylinder or casing of a so-called "free cylinder" machine. The main reason for including the moving cylinder in this component is so it can pass the relative motion between the piston and cylinder to any annulus child components (*see* chapter 11) for purposes of calculating shuttle heat transfer. In free-cylinder machines the cylinder and piston can both have large amplitudes with respect to a fixed reference frame yet be moving almost in parallel so that the relative amplitude between them is small. So it is important to calculate shuttle heat transfer based on the relative piston motion, not absolute motion.

The mass of the cylinder reciprocator should include all the mass rigidly attached to the cylinder. If you want to allocate some of the mass to an external reciprocating mass you can do so provided you connect the two reciprocating masses together with a force connection, which will force them to move together with the same amplitude and phase. Generally the cylinder reciprocator will have area attachments representing volume displacements to other gas spaces in your model and be connected to ground with a spring and damper, representing the suspension and load.

Keep in mind that in a free-cylinder model, both piston and cylinder motions are understood as being relative to a fixed coordinate frame. This is different than the usual mind-set when thinking about stirling-cycle thermodynamics where the displacer amplitude, say, really refers to the amplitude of the displacer relative to the cylinder. When working with a free-cylinder model you must make all such implicit understandings explicit in the model by providing appropriate volume displacements to any affected variable-volume gas spaces via area attachments to the moving cylinder.

There is no corresponding constrained-cylinder variation of this component. If you want to constrain the cylinder motion, or the piston motion for that matter, you can just connect it to an external constrained piston component using a force connection. The two components will then move together with the same motion. During the solution process, the force between them will automatically accommodate to produce whatever force is necessary to drive the reciprocating mass. You can always break the connection later to implement "free" motion.

12.1.7 Time-Ring Free-Piston and Free-Cylinder



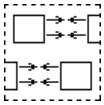
Similar to the above except with time-ring reciprocating masses representing both the piston and cylinder motion. This allows you to resolve higher harmonics in the resulting motion which may be of interest in certain cases. The number of nodes in the ring is according to the root-model input variable NTnode.

Chapter 13

Submodels and Containers

The components in this section allow you to group other model components together under a separate window.

13.1 Submodels



A submodel allows you to create all the child model components available in the root model, except organized one level deeper in the model tree and displayed in a separate window. This is handy for organizing large models, multi-stage models, managing display screen clutter and copy/pasting of groups of components simultaneously. You may promote connector arrows from within a submodel up to the root model level for connection there to other submodels or model components at the root level.

All of the input variables of the root model are inherited by a submodel and passed to any child model components created within the submodel. There are no new input or output variables introduced by a submodel. So whether or not a model component resides at the root level or in a submodel does not affect the model solution.

Using the cut and paste tools it is possible to move model components from the root model to a submodel and vice-versa. So long as the model component is not externally connected.

13.2 Containers

Container components are similar to submodel components except not all child components available in the root model are available in the container and new input variables are introduced.

The main reason for the existance of containers is to support heat exchanger modeling. For example, you might want to model the transverse heat flow (distributed between y-faces) between separate heat-exchanger components representing the two legs of a counterflow heat exchanger. Sage normally does not

allow this because the components might have different lengths, which would require the heat flux value (W/m) to change discontinuously across the boundary.

There are two ways to escape Sage's normal restrictions against such inter-component heat-flow connections. The first is create the components together within a *parallel container*. The second is to create them within a *multi-length container*.

Section 2.8 gives specific examples of interconnected equal-length and different-length heat exchangers.

13.2.1 Parallel Container



A parallel container defines a common length, number of computational cells (NCell) and initial temperature distribution (Tinit) inherited by all child components. Any canister or heat-exchanger child components created within the parallel container inherits these same values. This permits you to make transverse heat-transfer connections from the thermal solid components within the canisters or heat exchangers by promoting their connector arrows to a common level and connected them.

The only variables in parallel containers are the following inputs:

NCell : (dimensionless) The number of spatial nodes in the computational grids of all underlying model components having such grids.

Length : (real, m) Common length of all underlying model components.

Tinit : (cubic spline, W) Axial temperature distribution $T(x)$ where $x = 0$ is the negative endpoint and $x = 1$ is the positive endpoint. Underlying model components use Tinit as a fixed boundary condition or just an initial value, depending on whether the temperature of the component is fixed or solved.

The available model components in the parallel-container child-model creation palette are essentially all those available at the root-model level. So in addition to creating parallel canisters and heat exchangers you can also create moving parts and composite (piston/cylinder) components that might be associated with those components. In other words, a parallel container may be used as an organizational convenience to group together related components of large models, thereby simplifying the top-level edit window.

Also in the parallel-container child-model creation palette are the line heat source and distributed conductor components. A line heat source may be used to provide temperature boundary conditions to other components defined within the parallel container. The temperature so provided is that defined by the Tinit input variable of the parallel container. A distributed conductor may be used to provide a solid conduction path between other components. In either case the thermal connections are made via Gx heat-flow connections of negative or positive heat flow faces available on the Thermal Attachments page of the components to be connected.

13.2.2 Multi-Length Container



A multi-length container permits a different approach to transverse heat-flow connections, the result of which is to allow different-length components to be connected together. You might want to do this, for example, to connect together the walls of a tube spiral-wrapped around a straight central tube, as the iconic representation suggests. A special heat-flux transformer component, available in the basic page of the multi-length container child-model creation palette makes this possible (see below). Any components created as child models of a multi-length container can be interconnected via heat-flux transformers.

A multi-length container defines a number of computational cells (NCell) and initial temperature distribution (Tinit) that are inherited by all child components. It is similar to the above parallel container except that a common length is not defined. The only variables in parallel containers are the following inputs:

NCell : (dimensionless) As above.

Tinit : (cubic spline, W) As above.

Heat-Flux Transformer



A heat-flux transformer is the essential component available within a multi-length container for making transverse heat flow connections between components of different length. It overrides Sage's usual connection compatibility test and permits you to inter-connect thermal solids within any two heat exchangers (conductors or quasi-adiabatic surfaces) by connecting their negative or positive heat flow arrows to one of its two built-in connector arrows, via Gx or Gxt y-face heat-flow connections.

A heat-flux transformer works by changing the heat flux value (W/m) from one face to the other by the ratio of lengths for the two interconnected thermal solids. This conserves the total heat energy flow (W) across the boundary. The actual implimentation does this by way of a temperature grid whose values are defined implicitly by the following energy equation.

$$Q_n L_n = Q_p L_p \tag{13.1}$$

where Q_n and Q_p denote the heat fluxes at the negative and positive y-face boundaries and L_n and L_p are the lengths of the components connected across the two boundaries.

Warning about Convergence and Energy Conservation

A heat-flux transformer is designed to be interconnected only to components created within a multi-length container. But there are no software restrictions enforcing this rule. You can, in principle, connect it between higher-level components by promoting connection arrows to a higher level in the model. But if

you do so there is no guarantee that the solution will converge or that Sage's global energy conservation principle will hold.

If the NCell value for the heat flux transformer (inherited from multi-length container) is greater than for both higher level models then the solution will not converge. This is because a heat-flux transformer does not correspond to a physical object. Its defining energy equation only matches up boundary heat flows. If there are more T's in the heat-flux transformer grid than Q's in the conjoined model grids, there will be too many implicit function elements to satisfy simultaneously, given number of Q's available to do so. (This would not be a problem if there were a real solid energy equation present because then the internal temperature solution could adapt to any boundary condition by way of appropriate internal energy flows.)

A more subtle problem occurs if the NCell values differ for the two interconnected components: Global energy conservation will not hold. Grid interpolation methods, which are used to exchange solution information across connections, produce numerical truncation errors in the case when NCell values differ. This leads to slightly differ values for the total heat flow leaving one component domain and entering the other ($\int_n Qdx \neq \int_p Qdx$). Small energy conservation errors might not seem so bad but there is always the danger that the optimizer could exploit them and find a way to extract a large amount of "free"energy in order to boost power or efficiency.

Chapter 14

Material Property Data Bases

Some stirling model components use Gas or Solid enumerated-selection variables to define the thermal properties for the working gas or some solid material. These enumerated variables each contain a number of defining data fields. When you change the value of such an enumerated variable, you do not directly enter the values for its individual data fields. Rather, you select the appropriate material, by name, from a list box. While there are enough named materials available for many needs, there is the distinct possibility that you will want to specify some material not in the list. Anticipating this, Sage fills its property-selection list boxes from customizable property data-base files.

14.1 Changing Properties with PropBase

The default property files are gas.dta and solid.dta, located in your program directory (default c:\Program Files\Sage 4\Apps\Stirling\Bin). You can create a custom property file using the PropBase program distributed with the stirling model class. This program is labeled "property database editor" in your Window's program group. With PropBase running, File|Open the property file you want to start from, modify the data using the self-evident program dialogs, then File|Save As under a new name. Saving under a new name is important to avoid future conflicts with an upgrade distribution file of the same name. With PropBase it is easy to change properties of an existing material or create an entirely new one. **Note:** All property values must be entered in SI units.

To use your custom property file in Sage simply change the corresponding file name in the dialog that opens under the Options|Model Class menu command. The values you set are stored in the program initialization file (e.g. stirling.ini) located in the program directory, so they will be in effect for the current session as well as future sessions.

Sage reads data from property files whenever you change the value of a Gas

or Solid enumerated variable. This data becomes part of your model while Sage is executing and is stored in its input data file when your model is inactive. The next time you open the model, Sage automatically compares the data of any Gas or Solid enumerated variables with data by the same name, if any, in the current property files. You are ordinarily not aware of this unless there is a discrepancy, in which case Sage will put up a dialog giving you the option to update your model's data according to the property files or ignore the discrepancy.

14.2 Log Splines

Many properties are temperature dependent and represented by a discrete set of samples. When non-sampled values are needed, they are interpolated using standard cubic spline techniques — with one small twist. Instead of operating directly on a table of temperature-property values, the data is first transformed into the logarithm domain, interpolated there, then un-transformed back to the real domain. This is equivalent to curve fitting on log-log graph paper. The reason for this approach is that property values range over orders of magnitude in some cases and most property data is presented in the form of log-log plots. Cubic-spline interpolation, itself, is a way of fitting a piecewise-defined cubic polynomial to data so that first and second derivatives are continuous throughout the interpolation domain (*see* chapter 3 of reference [39]).

14.3 Solid Properties

The enumerated solid-material data base file is named solid.dta. The source for default solid data was mainly reference [51] and to a lesser degree reference [26].

In critical applications you should check over the conductivity and specific heat data pairs that appear in the display windows or output listing. In many cases, solid conductivity is not very predictable below 100K. Data from different sources for the same *nominal* material can differ by orders of magnitude presumably due to small variations in alloy composition. Solid specific heats, like conductivities, vary dramatically with temperature. However, there is much less scatter in the data as reported from several sources. Apparently, specific heat is not quite as sensitive to variations in material composition as is conductivity. The moral is that if you have good data available for your material, check it against the Sage values.

14.4 Gas Properties

The enumerated gas material data base file is named gas.dta. The sources for default gas data were thermophysical properties and P-v-T tabulations in references [51], [9], [52] and [34]. The value for R is simply the universal gas constant $\mathbf{R} = 8.314\text{E}3$ J/(kmol K) divided by the molecular weight of the gas.

14.4.1 Redlich-Kwong Fitting

For Redlich-Kwong gasses, theoretical values for parameters c_1 and c_2 are $c_1 = 0.42748$ and $c_2 = 0.08664$ (*see* chapter 5). The default gas-property data base does not use these values because they are not especially accurate for helium or hydrogen. Rather, it attempts to best-fit c_1 and c_2 to the second virial coefficient published in [8]. The following tables show an independent check of the accuracy of the fit by comparing compressibility calculated from tabulated data against values calculated by the best-fit Redlich-Kwong equation. The data source for this comparison is the compilation of Vargaftic [52]. Specific volume or density is presented there for various gasses over a wide range of temperatures and pressures. This can be most lucidly cast in terms of compressibility $Z = Pv/(RT)$ as a function of temperature and pressure. In general, agreement is good. The most demanding conditions, and the largest errors, occur at the lowest temperatures and highest pressures.

Helium

This gas is likely to be used over the widest range of conditions of any stirling working gas because it is suitable for both engines and cryocoolers. Data for 18K and 300K are summarized in the following tables.

T (K)	P (bar)	Z_{exper}	Z_{calc}
18	0.1	0.999	0.999
18	1	0.996	0.997
18	2	0.993	0.994
18	4	0.989	0.990
18	8	0.986	0.984
18	15	0.995	0.985
18	30	1.058	1.026
18	60	1.285	1.206
18	100	1.661	1.506

T (K)	P (bar)	Z_{exper}	Z_{calc}
300	1	1.000	1.000
300	2	1.001	1.001
300	4	1.001	1.002
300	8	1.004	1.003
300	15	1.007	1.007
300	30	1.015	1.013
300	60	1.029	1.026
300	100	1.049	1.044
303	200	1.090	1.087

Data for helium: Redlich-Kwong $c_1 = 0.5338$, $c_2 = 0.05886$

Hydrogen

With a critical temperature of 33K (versus 5.2K for helium) hydrogen is less likely to be a cryocooler working gas than it is an engine gas. Nevertheless, the attached data at 50K and that at 200K should cover all bases.

T (K)	P (bar)	Z_{exper}	Z_{calc}	T (K)	P (bar)	Z_{exper}	Z_{calc}
50	1	0.992	0.992	200	1	1.001	1.001
50	4	0.968	0.968	200	4	1.003	1.003
50	10	0.919	0.920	200	10	1.006	1.007
50	20	0.841	0.846	200	20	1.013	1.014
50	40	0.726	0.753	200	40	1.027	1.028
50	100	0.918	0.920	200	100	1.074	1.075
50	200	1.451	1.399	200	200	1.162	1.164
50	400	2.437	2.332	200	400	1.357	1.360

Data for hydrogen: Redlich-Kwong $c_1 = 0.4442$, $c_2 = 0.08207$

Nitrogen and Air

With critical temperatures about 130K, these gases apply exclusively to stirling engines or, perhaps, low-lift heat pumps. The tabulated data at 200K show good agreement at pressures below 100 bar but moderate error at higher pressures, notably 200 bar and above. This seems to be due to a dip in experimental Z vs P near 100 bar. Evidently the Redlich-Kwong fails to track this dip, which accounts for the rather significant discrepancy. However, at 400K and above, agreement is much better.

The data for nitrogen and air at 200K and 400K are summarized in the following tables. The c_1 and c_2 values for air are a weighed average of those for nitrogen and oxygen, based on their respective volumetric fractions in air.

T (K)	P (bar)	Z_{exper}	Z_{calc}	T (K)	P (bar)	Z_{exper}	Z_{calc}
200	1	0.998	0.998	400	1	1.000	1.000
200	2	0.996	0.996	400	2	1.001	1.001
200	4	0.991	0.992	400	4	1.001	1.001
200	10	0.979	0.980	400	10	1.002	1.003
200	20	0.959	0.962	400	20	1.006	1.006
200	40	0.921	0.933	400	40	1.012	1.014
200	100	0.845	0.918	400	100	1.038	1.045
200	200	0.907	1.064	400	200	1.093	1.114
200	400	1.268	1.496	400	400	1.238	1.292

Data for nitrogen: Redlich-Kwong $c_1 = 0.4456$, $c_2 = 0.1102$

T (K)	P (bar)	Z_{exper}	Z_{calc}	T (K)	P (bar)	Z_{exper}	Z_{calc}
200	1	0.998	0.998	400	1	1.000	1.000
200	2	0.995	0.995	400	2	1.000	1.000
200	4	0.991	0.991	400	4	1.001	1.001
200	10	0.977	0.978	400	10	1.002	1.002
200	20	0.954	0.957	400	20	1.004	1.005
200	40	0.910	0.922	400	40	1.010	1.010
200	100	0.814	0.881	400	100	1.031	1.034
200	200	0.855	0.996	400	200	1.079	1.092
200	400	1.205	1.389	400	400	1.212	1.247

Data for air: Redlich-Kwong $c_1 = 0.4372$, $c_2 = 0.1063$

14.4.2 Entering Tables of State

For gases defined in terms of a table of state, the data-entry process is complicated by the need to enter an entire table of compressibility values $Z(v_j, T_k)$. This compressibility table is visible in the scroll box titled *Properties of Selected Item* at the right of the form, when you highlight a tabular equation-of-state gas type. For example, the database file gas.dta contains a gas named Tabular Helium. Highlighting this gas shows its properties in the scroll box.

To change the compressibility table, click on the *Edit Properties* button below the scroll box, then on the *Compressibility* button of the sub-dialog box. You should now see a file-open dialog enabling you to load a file containing the appropriate data table. You do not enter values one by one as you do for single-valued data or cubic-spline interpolation pairs. You load an entire file of values all at once. This is to allow you to create the property table(s) using a spreadsheet program like Excel or by some other automatic means that might be easier and more reliable than entering each value by hand. For symmetry sake, there is also a provision for saving the a gas's property data to a separate file, although you will probably not need to do this very often.

Compressibility Data Format

The PropBase program expects to load a tab-delimited file of compressibility values. Excel normally produces such a format when saving as a text file (*.txt). The file must contain a rectangular table of data fields as follows:

first row a label such as "T \ v" identifying the nature of the first column and row of the table, followed by a number of specific volume v values in increasing order (important) with units (m^3/kg)

subsequent rows a temperature value T with units (K) followed by a number of property values under each v of the first row. The temperature values must be increasing in subsequent rows.

There is no limit to the number of v columns or T rows in the table. The easiest way to understand this is probably from the following, rather short, example — roughly what you might see on your Excel screen:

T \ v	0.005	0.006	0.007	0.008	0.009	0.01
3	5.8909	1.6582	9.11E-02	3.09E-02	3.47E-02	3.86E-02
3.5	5.2087	1.5932	0.26822	5.18E-02	5.82E-02	6.47E-02
4	4.7152	1.5693	0.42009	7.86E-02	8.84E-02	9.82E-02
4.5	4.3514	1.5679	0.54942	0.18123	0.12542	0.13935
5	4.0773	1.5788	0.66019	0.31901	0.20832	0.18872
5.5	3.8661	1.5964	0.75589	0.43549	0.32443	0.29551
6	3.7001	1.6174	0.8393	0.5353	0.42366	0.38945
6.5	3.5671	1.6397	0.91255	0.62182	0.50954	0.4708

You must adhere to this format closely, otherwise PropBase may squawk at you.

What Values?

As far as the property values themselves, you will probably want to produce them automatically using some sort of thermophysical-property software, such as the NIST database [34] or commercial HePak software (for helium). Obviously, the T-range must cover the entire temperature range you are interested in — with a bit of overlap margin for interpolation safety. The v range is determined from the desired pressure range and the available formulation for $v(T, P)$ (as a first approximation, $v = RT/P$ for an ideal gas). So the minimum and maximum v values would be:

$$v_{min} = v(T_{min}, P_{max}) \tag{14.1}$$
$$v_{max} = v(T_{max}, P_{min}) \tag{14.2}$$
$$\tag{14.3}$$

As far as computational time goes, the smaller the compressibility table, the less memory consumed and the faster the state-property interpolation routines will work. Tables on the order of 32×32 (rows $\times$ columns) are entirely reasonable. Tables on the order 100×100 would probably slow things down noticeably.

However, too small a table will reduce accuracy. So you will want to make sure there is enough resolution in your table to capture the non-ideal regions of $v{-}T$ space, where $Z \neq 1$. This is especially important at temperatures below the critical temperature where specific volume passes through the saturation curve (from gas to mixed liquid-vapor) and the liquid curve (from mixed liquid-vapor to pure liquid). There are sharp corners in $Z(v, T)$ at these phase-transition points, which leads to over and under-shoot of the interpolating polynomials on either side of the boundary. Figure 14.1 shows the compressibility values for helium in the mixed liquid-vapor region between 3 K and 5.25 K, which is just above the critical temperature.

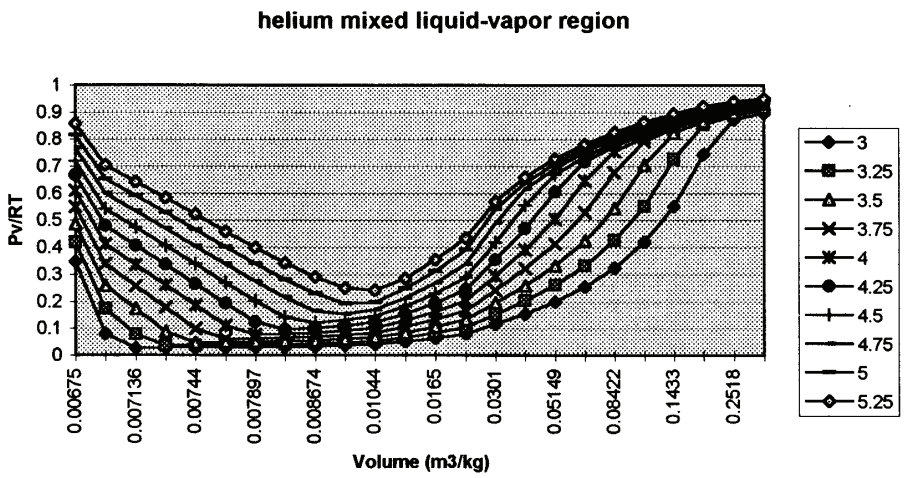


Figure 14.1: Compressibility of helium 4 in the mixed liquid-vapor region. The volume scale is nonlinear to better resolve the liquid and saturated-vapor curves. So the compressibility values between liquid and saturated vapor appear as smoothly increasing curves rather than straight lines, even though they actually are linear in v, due to pressure being constant during evaporation.

Choosing v_j and T_k Values Here is a strategy for choosing v_j and T_k values for any gas modeled after the successful strategy used for Tabular Helium in the gas.dta database.

First, pick a set of temperatures T_k in the range of interest and spaced close enough together to acceptably resolve the abrupt changes in compressibility when crossing into or out of the condensation region below T_{crit},. For Tabular Helium, these temperatures were 3.0, 3.25, 3.5, 3.75, 4.0, 4.25, 4.50, 4.75, 5.0, 5.25, 6.0, 6.5, 7.0, 7.5, 8, 10, 12, 15, 20, 25, 30, 40, 50, 70, 100, 150, 200, 250, 300, 400, 500. Above T_{crit} (5.2 K), the spacing increased gradually to large values because compressibility is better behaved.

Then, the choice of v_j's proceeds as follows. For each of the T_k temperatures below T_{crit}, calculate (with suitable property software) the liquid and saturated-vapor endpoint of the mixed liquid-vapor isobars — the specific volumes v_L and v_G where the fluid is completely condensed and completely evaporated. These values must be well resolved in your v_j values. A good way to do this is by listing the v_L and v_G in order and picking a v_j value midway between each. Choosing v_j values exactly equal to any v_L or v_G value is probably a bad idea because these are values where the derivatives of Z change discontinuously, which will lead to interpolating polynomial overshoot and undershoot. Having done this, you still have a number of gaps in the specific-volume range to be filled in. Specifically the region below the liquid curve, above the saturation curve and between the two above the highest v_L and the lowest v_G. Z is changing smoothly in these

regions so the choice of v_j is less critical. Spacing the v_j's by uniform increments or ratios with enough points that there are no large gaps in coverage seems to give good results.

The following table illustrates the entire process of choosing the v_j for Tabular Helium given the above set of T_k's:

v_j's (m^3/kg)	
0.00450	
0.00475	
0.00500	
0.00525	Equal spaced values below the liquid curve with the lowest value corresponding to 150 bar at 4 K.
0.00550	
0.00575	
0.00600	
0.00625	
0.00650	
0.00675	

T_k's	v_L's		
		.007012	
3.00	.0070744		
		.007136	
3.25	.0071982		
		.007274	
3.50	.0073488		
		.007440	
3.75	.0075317		
		.007644	Values midway between v_L values with linear extrapolation at the endpoints.
4.00	.0077562		
		.007897	
4.25	.0080385		
		.008224	
4.50	.0084094		
		.008674	
4.75	.0089390		
		.009440	
5.00	.0099403		
		.01044	
		.01313	Values in mixed liquid-vapor zone spaced in equal ratios.
		.01650	

T_k's	v_G's		
		.02075	
5.00	.025428		
		.03010	
4.75	.034774		
		.03999	
4.50	.045202		
		.05149	
4.25	.057781		
		.06575	Values midway between v_G values with linear extrapolation at the endpoints.
4.00	.073726		
		.08422	
3.75	.094706		
		.1090	
3.50	.12324		
		.1433	
3.25	.1634		
		.1929	
3.00	.2223		
		.2518	

0.316	
0.422	
0.562	
0.75	Values above the saturation curve in roughly equal ratios, with the highest value corresponding to 1 bar at 500 K.
1.0	
1.8	
3.2	
5.6	
10.0	

Low-Temperature guard row One other thing. Due to the way in which Sage builds its bicubic spline interpolating polynomials, second derivatives are assumed zero at the edges of the table. This affects the first derivatives calculated there and causes erroneous values in the derived internal energy values for the lowest temperature row, where the values of $\frac{\partial Z}{\partial T}$ are relatively large and a slight error is magnified. To limit the erroneous energy values you can add a *guard* temperature row just slightly colder than the previous coldest temperature. For example, in Tabular Helium there is a guard row with temperature of 2.99 K just below the 3.00 K row. Since the v_L and v_G values for 2.99 K are very close to those at 3.00 K, the above v_j values already bracket them well enough and there is no need to introduce more.

Interpolation Details

One way you can see if the resolution of your table is adequate is by creating a file of interpolated values by clicking the *interpolation details* button below the scroll box in the PropBase form. When you click this button, a dialog opens allowing you to specify a range of v and T and number of points. The result is a tab-delimited file of values, similar to your original compressibility table. You can inspect this file with Excel.

Note that the interpolation values produced are the values interpolated between your data points, which may or may not include the original data points. So, for example, by interpolating a bunch of points in a small region of $v - T$ space you can see exactly how the bicubic spline routines are interpolating between your Z values in a small region of your original table.

In addition to interpolated Z values, you also get interpolated values for mass-specific internal energy ε and entropy s. Because of arbitrary constants of integration these values may be offset somewhat from values you might be comparing them against. Constant offsets do not matter. Any other differences do matter. You should, at least, examine the internal energy values for accuracy because they are the basis for enthalpy flows and temperature calculations in subsequent gas-domain solutions. Entropy values are of lesser importance because they are only used for some available-energy outputs, not the basic solution variables.

Part II

Pulse-Tube Model-Class Supplement

Chapter 15

Overview

The pulse-tube model class is an extension of the stirling model class. It incorporates all of the stirling-cycle model components plus a few additions required for modeling so-called pulse-tube coolers. The principle new component is the pulse-tube itself, herein known as the *compliance tube*. This is the empty tube that serves as a compliant displacer of sorts. The naming reflects its functionality. Since Sage adopted this name the term *thermal buffer tube* seems to be gaining common acceptance and is also descriptive of functionality. However, no-one talks about "compliance-tube coolers" or "thermal-buffer-tube coolers" so the model class continues to be known as the pulse-tube model class. At any rate, the compliance tube component comes with a new gas model designed to incorporate some of the convective loss mechanisms peculiar to oscillating flow in the wall boundary layer. Other new components are flow restrictors, intended for representing orifi, valves and mass-flow pumps.

Sample pulse-tube input files are in the Apps\Ptube\Data sub-directory under the installation directory.

The input file extension for the pulse-tube model class is .ptb. The software is capable of reading and converting existing .stl (stirling) data files automatically to the pulse-tube format. Once converted a .ptb file cannot be demoted to a .stl file.

Chapter 16

Compliance Duct Gas Domains

The compliance-duct type of gas domain descends from the duct-type gas domain documented in the chapter 8. The major difference is the special treatment of the convective losses that arise in the wall boundary layer in the presence of an axial temperature gradient. These losses are critical because they carry thermal energy from the warm to the cold end of the compliance duct and degrade heat lift.

The gas axial conduction term q now comprises molecular conduction q_m, turbulent conduction q_t, free convection q_f, boundary convection q_b and streaming convection q_s, all considered in detail below. We can write this in dimensionless form as

$$\frac{q}{q_m} = 1 + \frac{q_t}{q_m} + \frac{q_f}{q_m} + \frac{q_b}{q_m} + \frac{q_s}{q_m} \tag{16.1}$$

where q/q_m on the left is just the axial conductivity enhancement ratio N_k and the terms on the right may be viewed as its component parts.

The variables of a compliance-duct gas domain are mostly the same as those in any gas domain documented in 8. New variables are:

Tmult : (real, dimensionless) Empirical multiplier for turbulent conduction q_t.

Gmult : (real, dimensionless) Adverse gravitational field multiplier for free convection q_f.

Bmult : (real, dimensionless) Empirical multiplier for boundary convection q_b.

Smult : (real, dimensionless) Empirical multiplier for streaming convection q_t.

QmolMean : (real, W) t and x averaged molecular conduction flow q_m.

QturbMean : (real, W) t and x averaged turbulent conduction flow q_t.

QfreeMean : (real, W) t and x averaged free convection q_f.

QoscMean : (real, W) t and x averaged boundary convection q_b.

QstrMean : (real, W) t and x averaged streaming convection q_s.

The empirical multiplier inputs give you free reign to scale any of the enhanced axial conduction mechanisms individually. They may still be scaled all-together by the inherited duct-gas input Kmult. Outputs QmolMean, ..., QstrMean are components of the inherited output QxMean, not additional losses. Their sum equals QxMean.

Just as for the ancestor duct gas domain, there is a compliance-duct gas domain for each individual type of compliance duct geometry.

16.1 Theory

The role of the compliance tube is to transfer the PV work of expansion from the cold end of the compliance tube to the warm end, thereby sucking heat out of the cold source, according to a first-law energy balance. Ideally this happens in the absence of any wall heat transfer or fluid mixing and the gas within the compliance tube oscillates back and forth with a uniform velocity profile across the tube cross section. Of course, in reality, there is wall heat transfer, fluid mixing and a non-uniform velocity profile. There may even be turbulence in ill-designed compliance tubes. All these real-world phenomena buck the ideal enthalpy flow with oppositely-directed convective losses. In any particular design instance, some may be small and some may be large, but prudent analysis demands we evaluate them all. That way none are liable to grow out of bounds during optimizations.

16.1.1 Turbulent Conduction

Turbulent conduction q_t is the same as that reported for axial-conduction enhancement in tube-bundle heat exchangers in chapter 10. Repeated here for convenience, the correlating expression is

$$\frac{q_t}{q_m} = 0.022 R_e^{0.75} P_r \tag{16.2}$$

16.1.2 Free Convection

The presence of an adverse gravitational field acting upon the temperature-induced density gradient within a compliance tube can produce buoyant instability that leads to free convection cells. Free convection is only a problem in compliance tubes oriented cold end up.

Free convection in vertically-oriented closed cylinders heated from below seems to lie in the unfashionable end of the free-convection universe. Most heat transmission textbooks ignore it entirely. However, Edwards and Catton in reference [10] are right on target, offering experimental data and a correlating expression for this very thing. Unfortunately, their correlating expression

involves a messy sum of tabulated coefficients. In another paper, considering convection across honeycomb panels comprising many convection ducts in parallel, Hollands [23] gives a correlating expression, still applicable to our case, but much easier to compute. The Hollands correlation in terms of a free-convection enhancement normalized by molecular conduction is:

$$\frac{q_f}{q_m} = 0.076X\left(1 - e^{1.19\frac{(X_c - X)}{X_c}}\right) \tag{16.3}$$

where

$$X = R_a^{1/3} \tag{16.4}$$

and X_c is the critical value of X, to be explained shortly. The enhancement is zero (molecular conduction only) for $X \leq X_c$. Rayleigh number R_a is the product of Grashof and Prandtl numbers

$$R_a = G_r P_r \tag{16.5}$$

A useful definition of Grashof number based on cylinder length L is

$$G_r = \frac{g\rho^2 L^4}{\mu^2}\frac{|T_x|}{T} \tag{16.6}$$

Here, $g = 9.8N/kg$ stands for Earth's gravitational field strength, T_x is the mean axial temperature gradient and T is the mean absolute temperature. In the above definition, the coefficient of fluid thermal expansion $\beta = -1/\rho\frac{\partial\rho}{\partial T}|_P$, which normally appears in the definition of Grashof number, has been replaced by its ideal-gas value $1/T$. The Grashof number measures the ratio of buoyant to viscous forces.

The critical Rayleigh number, used to calculate $X_c = {R_{a_c}}^{1/3}$, is computed as

$$R_{ac} = \frac{(a_1^2 + 15.9)^3}{a_1^2} \tag{16.7}$$

where

$$a_1^2 = a_0^2 + 1.99 \tag{16.8}$$

and

$$a_0 = 5.75\frac{L}{d} \tag{16.9}$$

The value a_1 is the so-called *horizontal wave number* for the first-mode convection cell within the tube. It is, essentially, a theoretical dimensionless wave number a_0 corrected for end effects by term 15.9. Theoretical wave number a_0 depends on the thermal boundary condition assumed at the wall. The above value applies to the case of an adiabatic wall boundary, which I felt made the most sense for us from the viewpoint of modeling free-convection as a decoupled loss mechanism.

The correlation (16.3) is not exactly the correlation Hollands recommends. His correlation contains a leading coefficient of 0.0585 instead of 0.076. I fudged

the leading coefficient to better agree with the experimental data of Edwards and Catton, published in [10], for heat transfer in a cylinder with adiabatic walls. My justification was that adiabatic-wall cylinders are more like compliance tubes than square-celled honeycombs — both because of the circular geometry and the lack of thermal interaction among adjacent cells. Correlation (16.3) agrees with the data to within 10% at $L/d = 2.5$, the highest aspect ratio considered.

In arguing for the form of his correlating expression, Hollands presents an informative and convincing case, worth repeating. It goes something like this: For very small L/d we have essentially convection in an infinite horizontal fluid layer heated from below. In such a geometry the top-heavy fluid layer becomes unstable at some critical Rayleigh number at which point convection cells appear, causing an enhancement to molecular-conduction heat transfer. As Rayleigh number increases, additional smaller-scale cells appear, further enhancing heat transfer. Finally, turbulence ensues at high-enough Rayleigh number. When vertical walls interrupt the horizontal layer, they suppress any convection cells smaller than the wall spacing, or tube diameter in our case. Thus the critical Rayleigh number at which the first convection cell occurs increases with reducing diameter (increasing L/d). Eventually though, at sufficiently high Rayleigh numbers, the conduction enhancement approaches that of an unobstructed horizontal layer, which is correlated by the Globe and Dropkin expression $q_f/q_m = 0.069 G_r^{1/3} P_r^{0.407}$. Hollands correlation captures both the critical Rayleigh number at which convection first appears and the high-Rayleigh-number limit.

16.1.3 Boundary Convection

Boundary convection is a sort of *shuttle* heat transfer produced by the convoluted near-wall velocity profile of oscillating flow. It is the analog of displacer shuttle heat transfer in a conventional stirling machine. At high Valensi numbers, the convoluted velocity and temperature profiles within the viscous and thermal boundary layer of oscillating flow interact to produce a convective thermal energy transport beyond that produced by the complex-valued Nusselt number model in Sage. Reference [11], which applies to laminar incompressible flow between parallel plates with an axial temperature gradient, shows that the effect is small so long as Valensi number remains below about 100. However, typical compliance tubes involve Valensi numbers significantly higher than 100.

Figure 6 of that paper shows a curve A, corresponding to the exact-solution enthalpy transport, and a curve B, corresponding to that of a solution based on complex-valued Nusselt number, like that of Sage's duct-gas model. The difference between these curves is the boundary convection q_b we are looking for. Taking the high Valensi number limits of the equations for these curves (equations (65) for $P_r = 0.7$, $\sigma = \infty$) and normalizing by molecular axial conduction q_m gives normalized boundary convection in the form

$$\frac{q_b}{q_m} = 0.159 \sqrt{V_a} (2\delta/d)^2 \tag{16.10}$$

where δ is the section-mean flow tidal amplitude. Rewriting $2\delta/d = 2(L/d)(\delta/L)$, the final boundary convection formula becomes

$$\frac{q_b}{q_m} = 0.64\sqrt{V_a}(L/d)^2(\delta/L)^2 \tag{16.11}$$

Although the above results were derived for parallel-plate flow, cast as they are in terms of hydraulic diameter, they apply to tubes as well as to narrow channels or annuli. This follows because boundary convection, in the high-Valensi-number limit, occurs entirely within a thin layer (thickness on the order $d/\sqrt{V_a}$) near the wall. As such, the phenomena is insensitive to the exact shape of the duct and amounts to a convective energy flow per unit wetted perimeter. The actual value of this energy flow per unit wetted perimeter could be obtained by multiplying either previous expression by the molecular conduction per unit area $q = -kT_x$, then by the ratio of flow area to wetted perimeter $d/4$. As expected, in the high-Valensi limit (neglecting the N_u term), the result would be seen to be independent of d or any other geometrical parameter. Assuming, in particular, it would be applicable to tubes, we could then convert it back to the original form by dividing by the area-to-perimeter ratio ($d/4$, same as for plates) and then again by q.

There remains the question of what happens in turbulent flow. Evidence suggest we might then base our boundary convection equation on an effective Valensi number where molecular viscosity is replaced by turbulent viscosity — which is generally very much higher. Thus, effective Valensi number is generally greatly reduced for turbulent flow and q_b would not be as large as predicted by equation (16.11). However, since enhanced axial conduction is likely to prohibit any reasonable compliance tube from operating in the turbulent flow regime in the first place, the issue is probably moot. We might as well ignore any turbulent correction to q_b, thereby giving added incentive for flow to remain laminar.

16.1.4 Streaming Convection

Streaming convection is a second-order steady flow circulation, within the whole tube interior, superimposed on the main oscillating-flow velocity field. It has no counterpart in displacer-type stirling machines. Unlike boundary convection, streaming convection occurs within the bulk of compliance tube, even in the high Valensi limit.

The second-order terms of an oscillating velocity field produce, in the high-Valensi limit, an effective wall-velocity boundary condition replacing the usual no slip condition. This effective wall velocity may be calculated as

$$u_w = \frac{3}{4}\left[\frac{|P-P_0||u|}{\gamma P_0}\left(\cos\theta + \sin\theta\right) + \frac{|u|^2}{\omega}\left(\frac{dA/dx}{A}\right)\right] \tag{16.12}$$

where $|P-P_0|$ is the pressure-fluctuation amplitude, P_0 is time-mean pressure, $|u|$ is the section-mean velocity amplitude, θ is the phase angle between them (velocity phase minus pressure phase) and A is the flow area. This formulation

is an approximation, leaving out some of the smaller terms, of a somewhat more complicated expression derived by Olson and Swift [36].

The fluid in the central part of the compliance tube, away from the wall boundary layer, attempting to satisfy this boundary condition while maintaining zero section-mean flow, assumes a zero-shifted parabolic velocity profile. In the presence of a temperature gradient, a central stream, brings fluid from one direction and exchanges heat with a near-wall stream which brings fluid from the opposite direction. This, results in a transverse temperature profile which interacts with the velocity profile to produce a net (section-average) thermal streaming convection [17].

For correlating the streaming convection loss there are two dimensionless groups of interest. The first is a Reynolds number based on effective wall velocity and hydraulic diameter

$$R_{ew} = \frac{\rho u_w d}{\mu} \tag{16.13}$$

The second is the dimensionless wall mass-velocity gradient

$$G_w = \frac{d^2}{\mu}\frac{\partial \rho u_w}{\partial x} \tag{16.14}$$

Tubes

In the case of a tubular geometry, streaming convection may be calculated from the approximate formula

$$\frac{q_s}{q_m} = \frac{1}{4}|R_{ew}P_r|L/d\left(1 - e^{-\frac{1}{48}\frac{d}{L}|R_{ew}P_r|}\right)F(P_rG_w) \tag{16.15}$$

where P_r is Prandtl number, L/d is length to hydraulic-diameter ratio and F is a function defined by

$$F(B) = \begin{cases} 1 + \frac{1}{64}B + \frac{1}{4096}B^2 & \text{if } B > -5 \\ \frac{6144}{B^3}\left(B\left(e^{B/32}+1\right) - 64\left(e^{B/32}-1\right)\right) & \text{if } B \le -5 \end{cases} \tag{16.16}$$

Narrow Slots or Annuli

In the case of a parallel-plate-like geometry the formula is

$$\frac{q_s}{q_m} = 0.193|R_{ew}P_r|L/d\left(1 - e^{-\frac{1}{162}\frac{d}{L}|R_{ew}P_r|}\right)F(P_rG_w/32) \tag{16.17}$$

where

$$F(B) = \begin{cases} 1 + \frac{7}{60}B + \frac{1}{96}B^2 + \frac{1}{1920}B^3 & \text{if } B > -10 \\ \frac{64}{B^4}\left(B^2\left(e^{B/4}+2\right) + 24B - 96\left(e^{B/4}-1\right)\right) & \text{if } B \le -10 \end{cases} \tag{16.18}$$

The above are approximations derived in [17] and [18]. However, the exponential formulations for large positive values of P_rG_w, which are likely to be inaccurate anyway, are replaced with the small P_rG_w approximations so they do not blow-up to absurd values in some cases.

16.2 Numerical Diffusion

Properly modeling, of even an ideal compliance tube, requires quite a few control volumes (NCell) to reduce numerical truncation errors to an acceptable level — typically about 7 or 8 in Sage. The problem is that the temperature field is mostly carried with the flow (rather than fixed to the matrix as in the case of a regenerator) which is a non-optimal state of affairs for the spatially fixed (Eulerian) solution framework of Sage.

The problem grows with increasing flow amplitude so that the error in predicted net enthalpy flux generally becomes quite significant when the tidal amplitude of the mean flow is more than about 25% of the duct length, or whenever the flow amplitude is high enough that a fluid particle at the inlet at the time of flow reversal travels more than about half way down the duct. The error tends to reduce the net refrigeration power, much as an increased gas thermal diffusion would, hence the name *numerical diffusion*.

Experience suggests that numerical diffusion is something to keep in mind but not worry about too much. It is generally not a problem during optimizations because there is usually no compelling reason for the optimizer to drive a compliance duct into the region of significant numerical diffusion. So, the error tends to be self limiting. The main problem is reduced simulation accuracy when validating experimental data at large compliance-duct tidal amplitudes.

16.2.1 Quantitative Estimates

The actual amount of numerical diffusion can be measured by comparing Sage simulated results against an exact analytic solution for which enthalpy transport is zero. Such a solution is that of an ideal adiabatic compliance tube filled with non-axially-conductive gas moving back and forth with uniform sinusoidal velocity, as would be produced by two parallel-moving pistons within cylinder spaces attached to either end. If the piston cylinders are each held at different temperatures (say by means of two heat exchangers positioned between them and the compliance tube) then an equilibrium temperature profile will establish itself within the compliance tube. Provided the tidal amplitude remains low enough, this temperature profile will consist of two isothermal legs from either inlet up to the maximum penetration depth (of a fluid particle that was at the inlet at the time of last flow reversal) with a continuous linear variation in between. The linear variation is not strictly-speaking necessary but it is one valid solution. This temperature profile will merely slosh back and forth with the mean flow, the net enthalpy flow down the tube will be zero and there will be no pressure variation. This happy state of affairs continues until the tidal amplitude increases to the point where gas from one end reaches the other. At this point warm gas is suddenly cooled, or vice-versa, with the result that net enthalpy transport is no longer zero.

When modeled by Sage, the idealized compliance-tube produces a net enthalpy transport h as given by table 16.1. The results are normalized by the

δ/L	NCell = 6 NTnode = 6	NCell = 8 NTnode = 8	NCell = 10 NTnode = 10	NCell = 12 NTnode = 12
0.1	0.0009	0.005	0.0004	0.0003
0.2	0.0061	0.0046	0.0036	0.0027
0.3	0.0184	0.0156	0.0127	0.0103
0.4	0.0387	0.0360	0.0315	0.0272

Table 16.1: Numerical diffusion error h/h_0 for different values of tidal amplitude ratio δ/L and grid cell numbers.

worst-case enthalpy transport h_0 that would occur in the case of zero tube length

$$h_0 = \frac{c_p}{\pi}\dot{m}_1(T_h - T_c)$$

where $\dot{m}_1$ is the average mass flow rate amplitude (it varies with temperature) and $(T_h - T_c)$ is the temperature difference. Theoretically h/h_0 should be zero up to the point where the tidal amplitude ratio δ/L reaches 1/2.

The results shown that h/h_0 grows quickly as tidal amplitude ratio approaches the critical value of 1/2. This is to be expected since the finite-difference solution for the pulse-tube tends to round out the corners of a temperature profile that is theoretically approaching a step function as tidal amplitude ratio approaches 1/2. But below a tidal amplitude ratio of about 0.25, erroneous enthalpy transport h/h_0 is less than 0.01.

16.2.2 Interpretation in terms of Refrigeration Power

So how significant is an erroneous enthalpy transport of $h/h_0 = 0.01$? This is hard to say since worst-case enthalpy transport h_0 is not something we usually think about. Instead we usually think about the ideal refrigeration power of a pulse tube which is

$$h_{pv} = \frac{1}{2}P_1\dot{V}_1$$

assuming pressure amplitude P_1 and volumetric flow rate amplitude $\dot{V}_1$ are in phase. But if we substitute $\dot{m}_1/\rho_0$ for $\dot{V}_1$, then $2P_0/(R(T_h + T_c))$ for ρ_0, we can formulate the ratio of ideal refrigeration power to worst-case enthalpy transport as

$$\frac{h_{pv}}{h_0} = \frac{\pi}{4}(1 - 1/\gamma)\frac{P_1}{P_0}\frac{(T_h + T_c)}{(T_h - T_c)}$$

So, for example, in a helium pulse tube with $(1 - 1/\gamma) = 0.4$, pressure ratio $P_1/P_0 = 0.15$ (somewhat on the high side), $T_h = 300$ and $T_c = 75$, we have $h_{pv}/h_0 = 0.079$. Therefore, an erroneous enthalpy transport of $h/h_0 = 0.01$ corresponds to $0.01/0.079 = 0.13$ of ideal refrigeration power — a significant amount. But the error quickly drops off as tidal amplitudes decreases. So, for tidal amplitude ratios on the order of 0.10, the error would only be of the order 1% of ideal refrigeration power.

Chapter 17

Compliance Duct Geometries

Compliance duct geometries are the containers that house compliance gas domains and provide to them certain geometrical information. They are analogous to the heat-exchanger geometries documented in chapter 10. They occur in the toolbox of the root-level pulse-tube model component and you use them for modeling that *empty-tube* part of a pulse-tube cooler, commonly known as the *pulse tube*.

17.1 Compliance Tube



The compliance tube is a descendant of the stirling-class tube-bundle heat exchanger. The main difference is that it substitutes a compliance-duct gas domain in its toolbox and adds a radiation-transport model component, in case you want to model radiation transport along the tube. Wall conduction is available in the thick-wall toolbox component already present.

From your point of view, you may think of a compliance tube as a special sort of tubular container. Its variables are the same as those the tube-bundle heat-exchanger model component. Most of the interesting stuff belongs to the compliance-duct gas domain child component (*see* chapter 16), which you drop in from the component palette.

17.2 Tapered Compliance Tube



A tapered compliance tube is similar to an ordinary compliance tube except its internal diameter and wall thickness are specified by cubic splines:

Dtube : (cubic spline, m) Tube internal diameter.

Twall : (cubic spline, m) Wall thickness.

The main reason for using a tapered compliance tube is to reduce or eliminate streaming convection by taking advantage of the dA/dx dependence of the effective wall-velocity u_w, according to equation (16.12). The proper choice of dA/dx will cancel the other term, thereby leading to zero streaming convection. Locally, at least. There may still be some variation of u_w along the tube length but it can generally be reduced compared to the case for uniform flow area. The demonstrated reduction of streaming convection by compiance-tube tapering has been reported by Olson and Swift in reference [36].

The implementation of the tapered compliance tube is similar to that of the tubular-cone canister reported in chapter 9. Dependent variables Aflow, Asec and Pwet (flow area, solid cross-sectional area and wetted perimeter also become cubic spline variables instead of uniform values. These splines are each defined by n data pairs $(x, A(x))$ equal spaced in x within the interval $[0, 1]$, where n is the larger of the number of data pairs in the specification of Dtube or Twall. So, the resolution of the dependent variables is only as good as the resolution of Dtube or Twall. In particular, if your intent is a nonlinear variation in flow area (as in a simple conical tube) you must specify more than the endpoint diameters in Dtube, even though it varies linearly with axial position. Specifying *Dtube* interpolation pairs at the relative x coordinates 0, 0.25, 0.5 and 1.0 would probably be sufficiently.

The turbulence transition model along with the heat-transfer and flow-friction correlations are unchanged from those of uniform area ducts. This is probably incorrect to some degree, with the result that heat transfer and flow friction will probably be off a bit. This should not pose a problem for compliance-tube modeling because these terms are generally small.

Chapter 18

Flow Restrictors

Flow restrictors are composite geometry and gas-domain components that model zero-volume adiabatic flow restrictions. They are intended for modeling the orifi found in pulse-tube coolers. They are found in the toolbox of the root-level pulse-tube model component and are born with gas flow connectors at either x end intended for connection to stirling-class gas domain model components. While you could also model orifi using a stirling-class heat exchanger geometries and child-model gas domains, the use of flow restrictors is easier and should improve solution speed and convergence stability. Keep in mind, though, that flow restrictors are not appropriate where the volume of the orifice is significant or where there might be a significant amount of heat transfer to or from the gas. Variables common to all flow restrictors are:

Length : (real, m) Flow length L.

Aflow : (real, m^2) mean-flow cross sectional area A.

Pwet : (real, m) Wetted perimeter w.

Fmult : (real, dimensionless) Empirical multiplier for viscous pressure drop.

Tinit : (real, K) Initial temperature. Used only for initializing or re-initializing solution variables ρ and ρe. As such, its value does not affect the final converged solution, but it may affect whether the solution converges at all. It should be set to a value consistent with the initial temperatures of adjacent components.

AEfric : (real, W) Available energy loss to viscous flow friction.

FT : (Fourier series, K) Gas temperature T.

FP : (Fourier series, Pa) Gas pressure P.

FDP : (Fourier series, Pa) Pressure difference across restrictor (positive − negative ends).

FH : (Fourier series, W) Stagnation enthalpy flow $uA(\rho e + P)$.

FRhoUA : (Fourier series, kg/s) Mass flow rate ρuA.

MachMean : (real, dimensionless) Time-averaged Mach number. Keep an eye on this. If it gets near unity Sage may not converge.

ReMean : (real, dimensionless) Time-average Reynolds number R_e.

The working gas (properties) comes from the parent model component.

All flow restrictors compute viscous pressure drop in terms of Darcy friction factor f and local loss coefficient K as

$$\Delta P = -\left(f\frac{L}{d_h} + K\right)\rho u|u|/2 \tag{18.1}$$

where hydraulic diameter is calculated in terms of flow area A and wetted perimeter w as

$$d_h = 4A/w \tag{18.2}$$

Individual flow restrictors calculate f and K differently, typically as a function of Reynolds number

$$R_e = \frac{\rho|u|d_h}{\mu} \tag{18.3}$$

18.1 Sintered Powder Plug



The sintered powder plug adds input variables:

Dcan : (real, m) Diameter D_c of plug or internal diameter of canister holding plug.

Dsphere : (real, m) Powder sphere or particle diameter d_s.

Porosity : (real, m) Plug porosity β (void volume / total volume).

It calculates flow area as

$$A = \beta\frac{\pi}{4}D_c^2 \tag{18.4}$$

and wetted perimeter as

$$w = 6(1-\beta)\frac{\pi}{4}D_c^2/d_s \tag{18.5}$$

presuming spherical particles. It calculates friction factor using the same correlation as for the stirling-class packed-sphere matrix heat exchanger geometry, namely:

$$f = (79/Re + 1.1)\beta^{-0.6} \tag{18.6}$$

Local loss coefficient K is zero.

18.2 Sharp-Edged Orifice



The sharp-edged orifice adds input variables:

Dorf : (real, m) Diameter D_o of orifice.

Cd : (real, dimensionless) Discharge coefficient C_d.

It calculates flow area as

$$A = \frac{\pi}{4} D_o^2 \tag{18.7}$$

and wetted perimeter as

$$w = \pi D_o \tag{18.8}$$

According to the conventional formulation for flow through a sharp-edged orifice, the mass flow rate $\dot{m}$ is governed by

$$\dot{m} = -C_d A \sqrt{\frac{\rho \Delta P}{(1 - A/A_h)^2}} \tag{18.9}$$

where C_d is the discharge coefficient, A_h is the housing area (upstream and downstream flow area) and ρ is the average of upstream and downstream density. To put this in the above pressure-drop form we solve for ΔP, and write ρu^2 for $\dot{m}^2/(\rho A^2)$, obtaining

$$\Delta P = -\frac{2}{C_d^2} \left(1 - A_f/A_h\right)^2 \frac{\rho u |u|}{2} \tag{18.10}$$

From this we can identify $(1-A_f/A_h)^2$ with input Fmult and local loss coefficient K as $2/C_d^2$. According to Idelchik ([25], p. 90), K ranges from about 2.8 for a thin sharp-edged orifice to 1.55 for a drilled hole in a thick plate ($L/d \geq 2$, sharp edged). Corresponding discharge coefficients are $C_d = 0.84$ (orifice) and $C_d = 1.14$ (drilled hole).

Actually, the sharp-edged orifice calculates loss coefficient K slightly differently in an attempt to improve solution stability. Namely, as

$$K = 500/R_e + 2/C_d^2 \tag{18.11}$$

The first term is reminiscent of the Darcy-flow part of the Ergun friction factor for porous materials. It is significant only for small R_e and serves to linearize ΔP as a function of u. Friction factor f is zero.

18.3 Asymmetric Sharp-Edged Orifice



The asymmetric sharp-edged orifice is a variation of a sharp-edged orifice with a pressure recovery diffuser in one direction or the other defined by new input variable:

Recov : (real, dimensionless) Recovery area ratio R.

You might, for example, use such an orifice in a model having a closed flow loop where you need a way to cancel unwanted DC (time averaged) flow circulation. You would do this by optimizing Recov subject to a constraint like FRhoUA.mean = 0.

Asymmetric flow is implemented in terms of two areas A_+ and A_-, the former being the effective orifice flow area for positive flow and the later the effective area for negative flow. The two areas are related by the formula

$$\frac{A_+ - A_-}{A_{min}} = R \tag{18.12}$$

where R is the recovery area ratio and A_{min}, the smaller of the two areas, is the orifice flow area inherited from the parent object. In other words, for $R = 0$ there is no recovery and flow is that of the parent sharp-edged orifice. For $R > 0$ there is a recovery channel in the positive direction with exit area $A_+ = (1 + R)A_{min}$. And for $R < 0$ there is a recovery channel in the negative direction with exit area $A_- = (1 - R)A_{min}$. The succinct way to say all this is that the orifice flow area A_f is always given by

$$A_f = \begin{cases} (1 + \max(R, 0))A_{min} & \text{if } u > 0 \\ (1 - \min(R, 0))A_{min} & \text{otherwise} \end{cases} \tag{18.13}$$

At the root of the DC flow phenomenon are ill-phased density and velocity fluctuations in the gas which tend to produce DC pressure gradients. The issue is whether or not DC pressure gradients produced by various flow resistances in the closed loop completely cancel each other. If they do not, then DC flow is the consequence. Enthalpy flow in components like the regenerator and compliance tube can be extremely sensitive to DC flow. So, one symptom of DC flow is extreme sensitivity of heat lift to small changes in input variables or grid node counts.

18.4 Check Valve



The check valve is a variation of a sharp-edged orifice that behaves like an orifice in one direction but essentially blocks flow in the other direction. New input variables are:

Aratio : (real, dimensionless) Ratio of fully-closed to fully-open flow area R. Generally a positive number $\ll 1.0$. A value of 1.0E–2 is a reasonable value. Backflow leakage increases as Aratio increases and decreases as Aratio decreases. But too small a value may cause convergence problems.

Popen : (real, Pa) Pressure difference ΔP_c required to open the valve. A positive value means positive flow opens the valve. A negative value means negative flow opens the valve. At pressures above ΔP_c flow resistance changes smoothly but rapidly to the fully-open value. A zero value means the valve is always open.

Instead of modulating flow by changing the actual flow area, like the asymmetric orifice, a check valve simply adjusts the flow friction to have the same effect. This avoids instabilities due to high gas flow velocities when the area suddenly goes from large to small. In the open position, check valves calculate pressure drop as the orifice pressure drop from equation (18.10), for the full open area, plus the opening pressure difference ΔP_c. In the closed position, pressure drop is the full-open orifice pressure drop increased by a factor $1/R^2$, where R is the closed-to-open area ratio. So even though actual flow area does not change, the effect on pressure drop is the same as if it had.

Check valves decide whether they are fully closed, fully open or somewhere in between, based on the current mass flow rate $\dot{m}$ compared to the mass flow rate $\dot{m}_c$ when the valve begins to open, which is reckoned as the mass flow rate that produces a closed-position pressure drop of $-\Delta P_c$. At twice $\dot{m}_c$ the valve is presumed fully open. No valve inertia is modeled.

In mathematical terms, this is succinctly stated in the formula for calculating pressure drop, which for positive flow is

$$\Delta P = \begin{cases} \Delta P_0/R^2 & \text{if } \dot{m} < \dot{m}_c \\ \Delta P_0 - \Delta P_c & \text{if } \dot{m} > 2\dot{m}_c \\ \text{smooth transition} & \text{otherwise} \end{cases} \tag{18.14}$$

where ΔP_0 is the pressure drop calculated from orifice equation (18.10) as a function of current mass flow rate, at full open area. For negative flow a similar equation applies except the signs of the inequalities are reversed.

So why can't a check valve seal perfectly in the back direction? The reason has to do with the way gas domains are set up and connected together. Essentially, a check valve receives mass flow rate as an input and must calculate an appropriate pressure drop as an output. And a pressure drop of infinity is not a mathematically proper value. So all a check valve can do is supply a large pressure drop for a small amount of backflow. But too large and solution non-convergence may be the result, which is why the Aratio input cannot be too small.

18.5 Time-Dependent Valve



The time-dependent valve is a variation of a sharp-edged orifice that behaves as if the inerited flow area (that given by input Dorf) is reduced (or increased) by a time-varying multipler factor

$$F(t) = \max(R(t), R_{min}) \tag{18.15}$$

The time-varying factor $R(t)$ comes from the Fourier-series input variable FRestrict and the minimum allowed value R_{min} from the real input MinRestrict, which must always be positive. A time-dependent valve is intended for modeling such things as the pressurization valve in a Gifford-McMahon cooler. New input variables are:

FRestrict : (Fourier series, dimensionless) Flow area multiplier $R(t)$ above. Keep in mind that valves that open or close abruptly are difficult to resolve in the coarse compuational grid employed by Sage. Better to model a more smoothly varying equivalent valve, if possible. Either that or increase the number of time nodes in the computational grid (input variable NTnode). For more information on how you might specify FRestrict to accomplish your objectives, see the section on Fourier Series in the *Sage User's Guide*.

MinRestrict : (real, dimensionless) Minimum flow area multiplier R_{min} above. Generally a positive number $\ll 1.0$. A value of 1.0E–2 is reasonable. Too small a value may cause convergence problems.

Like a check valve, this object does not change the actual flow area, but only adjusts the pressure drop to have the same effect, by calculating it as

$$\Delta P = \Delta P_0 / F(t)^2 \tag{18.16}$$

where ΔP_0 is the pressure drop calculated from orifice equation (18.10) as a function of current mass flow rate, at inherited flow area. This prevents instabilities due to high gas flow velocities when the area is small.

18.6 DC Flow Blocking Restrictors





Another way to cancel DC flow is to use one of these flow-restrictor variations, which descend from the sintered powder plug or the sharp-edged orifice components, respectively. They automatically block DC flow by imposing on top of the pressure drop inherited from the parent component an implicitly-solved DC pressure-drop sufficient to exactly cancel any DC mass flow rate. This DC flow blocking behavior is built into the model solution and does not require an optimization constraint to solve, as in the case of the asymmetric sharp-edged orifice. The amount of DC pressure-drop required appears as output variable DPstdy.

In some cases you may want to regulate the DC flow to some desired set-point rather than canceling it entirely. You can do this by changing the value of input variable DCRhoUA from its default value of zero to the desired set-point.

The additional variables for this component are:

DCRhoUA : (real, Kg/s) set-point for DC (time-average) mass flow rate.

DPstdy : (real, Pa) pressure drop required to achieve the DC mass flow rate set-point.

Variable DPstdy is not the same as the time-mean value of the pressure-drop Fourier series FDP because FDP includes the pressure drop of the ancestor component, which may have its own DC component for reasons outlined above.

While very convenient from a modeling point of view, these components suffer from the disadvantage that they do not specify the mechanism by which

DPstdy is to be generated. This is partly a good thing, because the value of DPstdy is subject to considerable numerical error. Changing the number of time or spatial nodes in the computational grid, or even the time-grid orientation with respect to the solution (by changing compressor piston phase), can dramatically affect DPstdy. This is because the time-average pressure drop is the residual of some much larger time-varying pressure drops.

Orifice designers, then, have two choices. Either they ignore DPstdy, hoping that the any DC flow effect will be too small to matter, or they deal with it. Dealing with it might involve designing-in some sort of mechanism to produce the a DC pressure drop bias of the predicted magnitude (allowing time in the test cell to calibrate it). Or it might involve eliminating DC flow by some other means — by means of an impermeable membrane, for example. In fact, it is useful to think of these components as impermeable membranes in series with an orifice.

Another disadvantage of these components is that they must only be used in situations where DC flow is possible (closed loop flow paths, not otherwise DC-flow restricted) lest a solver singularity result. In other words, if DC flow is already implied to be zero, then any DC pressure-drop imposed by these components will have no affect — and may result in a singular solution. This fact places an added burden on the model designer to recognize when DC flow is a problem and to correct it, but not over-correct it. For example, placing two DC-flow canceling components in series, is taboo.

18.7 Mass-Flow Pump



The universe of modeling options admits to another possibility — one where DC flow is desirable. To model an open thermodynamic cycle, for example. Or to investigate the effect of a prescribed amount of DC flow. This component allows you to model a mass-flow pump by recasting the mass-flow-rate Fourier series FRhoUA as an independent input variable rather than an output. What happens is the pressure drop across the component is solved implicitly in order to make the mass flow rate come out right. You can see what this pressure drop is by inspecting output variable FDP.

You can specify the mass flow rate to be anything you desire — including sinusoidal terms and higher harmonics. This means you can model much more than merely a DC flow generator. You can model the perfect flow restrictor. One that gives any desired mass flow and phasing without regard to the physical means for achieving it. And the subfields of the mass flow rate can be optimized or referenced in constraints, as discussed under the topic of Fourier-series variables in the Sage User's Guide. One warning though: If you specify a nonzero DC part of the mass flow rate, the mass-flow pump should be located within a closed loop capable of supporting DC flow, otherwise the mean pressure drop will be indeterminate and the solution will not converge — if DC flow is impossible, then there is no amount of pressure drop that will force the required value.

The flow area Aflow is another input. It is present in order to define flow velocity in the pump, but normally its value does not matter much. However, it should be large enough that the mean Mach number (output MachMean) is substantially below one.

When starting from initial values or a re-initialized solution, a mass flow pump in your model can slow down convergence, depending on the magnitude of the flow specified. The DC component of FRhouUA seems especially troublesome in this regard. If this is a problem, try starting with a small mass flow rate initially and increase it gradually over the course of several solutions.

18.8 Mass-Flow Driver



This is a phasor version of the previous mass-flow pump. It specifies mass flow rate as a phasor input variable, having sinusoidal time variation. You can optimize the subfields of the mass flow rate, or reference them in constraints, as discussed under the topics of Complex and Phasor variables in the Sage User's Guide.

Why would you want to use a mass-flow driver since you can specify all of the Fourier-series components, not just the sinusoidal part, with the previous mass-flow pump? Because this component allows the DC part and all higher harmonics to pass unobstructed. So there is no requirement that it be placed in a closed flow loop, as is the case with the mass-flow pump. Mass-flow drivers are perfectly happy as components in either closed or dead-ended flow loops. The DC flow component cannot be over-constrained. You can even connect mass-flow drivers at both ends of a duct, for example, to achieve a desired pressurization within the duct, provided the physics is reasonable.

The way the mass-flow driver works is by defining the desired mass flow rate as an independent phasor input variable PhsrRhoUA and implicitly solving for the phasor pressure drop that will bring this about. The phasor pressure drop is actually implemented as two implicit real variables corresponding to the real and imaginary parts. Although these variables are invisible, they give their values to the solution grid and you can monitor them in the first harmonic of output variable FDP.

As with the mass-flow pump, the flow area Aflow does not affect the pressure drop but it does affect the gas velocity. So it should be large enough that the mean Mach number (output MachMean) is substantially below one.

18.9 Flow Restrictor Theory

To be connection-compatible with the gas-domains of the stirling model class requires two things: flow restrictors must maintain a time-ring that includes state variables $P, u, \rho, \rho e$ for export to flow connectors. And they must import mass flow rate $\rho u A$ and upwind mass-specific enthalpy h from flow connectors for use in their own internal solution. $\rho u A$ must be taken as the boundary mass

flow rate, at all times, and H as incoming energy, only when flow is into the restrictor domain. Other state variables may be included for convenience.

The solution grid for flow restrictors is a single time ring containing state variables $A, \rho, \rho uA, \rho e, u, T, P$. These are in the same order and have the same meanings as in the stirling-class gas domains, except there are a few omissions for state variables no longer required. Generally, the variables in the grid are understood as downwind values, except for ρe and P which are separated into negative- and positive-boundary values to allow for an entropy-generating pressure drop. Aside from that, the variables are designed to maintain mass and energy continuity across the flow restriction. Neither space nor time differencing is required anywhere. Exact meaning and method of calculation for all variables are explained in detail below.

A

Flow area A is computed explicitly from geometrical inputs appropriate to the specific type of flow restrictor. The reason for including it as a state variable is so time-dependent flow areas may be specified, as is convenient in the implementation of the time-dependent valve component.

ρ

Mass density ρ is computed implicitly from the zero-volume mass-continuity equation

$$(\rho uA)_+ - (\rho uA)_- = 0 \tag{18.17}$$

where $(\rho uA)_+$ and $(\rho uA)_-$ are the values imported from the positive and negative flow connectors. This forces mass flow rate in the adjoining flow connectors to be equal. Implicit functions used for solving ρe also help to determine ρ. The useage in these other implicit functions is consistent with ρ being the downwind value.

ρuA

Mass flow rate ρuA is computed explicitly as the average of the values in the adjoining flow connectors:

$$\rho uA = \frac{1}{2}\left[(\rho uA)_+ + (\rho uA)_-\right] \tag{18.18}$$

This fulfills the requirement that the flow restrictor use imported (ρuA)'s in its interior solution.

ρe

Downwind energy density ρe is computed implicitly by solving the energy continuity equation written in the form

$$\frac{\rho e + P}{\rho} = h_i \tag{18.19}$$

where ρe and P on the left are downwind state variables while h_i on the right is the mass-specific enthalpy evaluated using variables imported from the component across the flow connection in the upstream direction. For an ideal gas h reduces to $c_p T$ in the zero-velocity limit where kinetic energy may be neglected.

For numerical reasons it is convenient to separate ρe into two variables ρe_- and ρe_+, thought of as lying at the negative and positive boundaries of the flow restrictor. The downwind value is ρe_+ when flow is positive and ρe_- when flow is negative. ρe_- is exported to the negative flow connector and ρe_+ to the positive flow connector. The reason this makes sense is because the implicit function determining either can be formulated without a discontinuity at flow reversal, as would otherwise be the case. That is, the equation for determining ρe_- is always that local mass-specific enthalpy $\frac{\rho e_- + P_-}{\rho}$ equals the value imported across the positive flow boundary, which is valid when flow is in the negative direction and does no harm otherwise. A similar but opposite equation determines ρe_+. The meaning of P_- and P_+ is similar to the meaning of ρe_- and ρe_+, and is explained further below.

It might be tempting to use these continuity conditions instead to calculate ρe_- and ρe_+ explicitly. This will not work because of a circular reference when accessing P, which itself depends on ρe.

u

Velocity u is computed explicitly in terms of other state variables as

$$u = \frac{(\rho u A)}{\rho A} \tag{18.20}$$

The parenthesis in the numerator indicate that $(\rho u A)$ is a single state variable. Since ρ is understood to be a downwind value so u is a downwind value.

T

Temperature T is computed explicitly in terms of other state variables from the equation of state function

$$T = T(\rho, \rho e, u) \tag{18.21}$$

The value of ρe is taken as ρe_- for negative directed flow and ρe_+ for positive directed flow. In other words, the downwind value. Since ρ and u are also downwind values the value of T is a downwind value.

P

Pressure P is a bit tricky. The first thought that comes to mind is to calculate it explicitly in terms of other state variables according to the equation of state. But then how would pressure drop be imposed on the solution?

The answer is to break down pressure into two components P_- and P_+, similar to the treatment for ρe. P_- is exported to the negative flow connector and P_+ to the positive flow connector. The idea is to calculate them explicitly, in terms of the thermodynamic pressure, calculated from the equation of state $P(\rho, T)$, plus or minus the pressure drop. For the case of P_- the equation of state is valid for the negative flow direction because of the downwind interpretation already given to ρ and T. For the positive flow direction $P(\rho, T)$ may be offset by the required pressure drop to produce the correct pressure. If this is also done similarly but oppositely for P_+ then the components across the flow connectors at either end see the correct pressure drop at all times. This can be written mathematically as:

$$P_- = \begin{cases} P(\rho, T) & \text{if } \rho u A \leq 0 \\ P(\rho, T) - \Delta P & \text{otherwise} \end{cases} \tag{18.22}$$

and

$$P_+ = \begin{cases} P(\rho, T) & \text{if } \rho u A > 0 \\ P(\rho, T) + \Delta P & \text{otherwise} \end{cases} \tag{18.23}$$

Note that $P_+ - P_-$ is always the pressure drop ΔP.

Part III

Low-T Cooler Model-Class Supplement

Chapter 19

Overview

The low-T cooler model class is an extension of the stirling and pulse-tube model classes. It incorporates all of the model components in these predecessor classes plus additional features and components required for modeling coolers at extremely low temperatures, near or below the critical temperature of the working gas. The low-T cooler model class extends Sage modeling capacity to include Joule-Thomson (J-T) cooling loops.

Sample pulse-tube input files are in the Apps\LowTCooler\Data sub-directory under the installation directory.

The input file extension for the low-T cooler model class is .ltc. The software is capable of reading and converting existing .stl (stirling) and .ptb (pulse-tube) data files automatically to the low-T cooler format. Once converted a .ltc file cannot be demoted to a .stl or .ptb file.

Chapter 20

Working Gas

The low-T cooler model class supports two new working gas formulations, both with improved-accuracy tabular equation of state, known as TBSpline2Gas and TBSpline3Gas in the Sage source code. The "BSpline" in the names refers to bicubic-spline interpolation, which is a way of fitting a piecewise-defined cubic polynomial to two-dimensional data (specific volume v and temperature T being the two dimension) so that first and second derivatives are continuous throughout the interpolation domain (*see* chapter 3 of reference [39]). Both TBSpline2Gas and TBSpline3Gas are evolutionary advancements compared to the tabular gas formulation available in the stirling and pulse-tube model classes. In the case of TBSpline2Gas, the table of internal energy interpolation values is read as an input, rather than generated internally as it was in the original TBSplineGas, with the result that accuracy and numerical stability is increased. In the case of TBSpline3Gas viscosity and thermal conductivity are tabulated over a range of both pressures and temperature, instead of just temperature at some fixed pressure as they are in TBSpline2 gases.

Both gas types may be available in the gas-property data file you are currently using (Options|model class, properties tab). They are selected by name from a list when specifying the "gas" input at the root-model level. Generally TBSpline2 gases contain a reference pressure in the name while TBSpline3 gases do not. You can tell for sure which gas type is in your model by inspecting the inputs in the root-model display page with the "full detail" option set in the "enumerated variable display detail" selection box of the Options|Sage dialog.

TBSpline3 gases are the most recent addition to the Low-T Cooler model class and are expected to eventually supercede TBSpline2 gases because of their increase accuracy over a wide range of pressures. For backward compatibility, TBSpline2 gases continue to be supported and documented although they are being phased out of the default gas property file LowTGases distributed with the Low-T Cooler software.

20.1 TBSpline2Gas

A TBSpline2Gas is defined by the following values:

T0 : (K) Representative temperature T_0.[1]

Rgas : (J/(kg K)) Gas constant R.

specific heat : (J/(kg K)) A cubic spline variable with temperature vs zero-pressure specific heat $(T, c_p(T))$ data pairs covering a broad range of temperatures.

viscosity : (kg/(m s)) A cubic spline variable with temperature vs viscosity $(T, \mu(T))$ data pairs.

conductivity : (W/(m K)) A cubic spline variable with temperature vs conductivity $(T, k(T))$ data pairs.

Z(v, T) : (dimensionless) A rectangular table of $Z(v_j, T_k)$ values where $Z = Pv/RT$ is compressibility, v is specific volume and T is temperature. The first (left) column contains the T_k values and the first (top) row the v_j values. The corresponding Z values lie in the body of the table.

ε(v, T) : (J/kg) A rectangular table of mass-specific internal energy values $\varepsilon(v_j, T_k)$, structurally similar to the $Z(v_j, T_k)$ table.

The values of the individual properties, including cubic-spline interpolation pairs or state tables, appear after the enumerated identifier in display windows and the output listing. The choices available when specifying the Gas input variable in the root-level model component come from the default database file LowTGases.dta, or a data base file customized by you using the property database editor LTCProp.exe.

20.1.1 Benefits of TBSpline2Gas

The previous tabular-state gas class TBsplineGas derived state tables for mass-specific internal energy $\varepsilon(v, T)$ and mass-specific entropy $s(v, T)$ from the compressibility state table $Z(v_j, T_k)$, which was provided as input. This approach worked reasonably well above the critical temperature T_{crit}. At lower temperatures, however, integrating Z across phase-change boundaries (where derivatives are discontinuous) resulted in noise in the derived ε table. One effect of this noise was to create high-frequency wiggles in curves of $c_p(T)$ at constant pressure below T_{crit}. The resulting wiggles caused $d(c_pT)/dT < 0$ at some temperatures, leading to solution convergence instability. This can be understood by noting that $\dot{m}c_pT$ is the enthalpy flow rate (ignoring any pressure-dependant part). So $d(CpT)/dT < 0$ is equivalent to $d(\dot{H})/dT < 0$ for fixed $\dot{m}$. It is not to hard

[1] A vestige of the ancestor ideal gas class used only as the temperature at which c_p was evaluated in returning the ratio of specific heats $\gamma = c_p/c_v$. T_0 is of no consequence in tabular gas classes.

to convince oneself that $d(\dot{H})/dT < 0$ produces a destabilizing enthalpy flow for a perturbation in upstream T. An enthalpy flow that tends to amplify any perturbation in upstream T.

To increase accuracy below T_{crit} the TBSpline2Gas class reads the $\varepsilon(v_j, T_k)$ table as input, leaving $s(v_j, T_k)$ as the only derived table. The procedure for generating the $s(v_j, T_k)$ table from the $Z(v_j, T_k)$ and $\varepsilon(v_j, T_k)$ tables is the same as that documented previously under the Stirling-Cycle Model Class in chapter 14.

20.1.2 Entering Tables of State

For TBSpline2Gas gases the procedure for entering gas properties and state tables is similar to the procedure documented in chapter 14 except that there are two state tables to enter, $Z(v_j, T_k)$ and $\varepsilon(v_j, T_k)$. TBSpline2Gas gases are identified by the name BSpline2 Gas in the Add menu item of the Propbase program LTCProp.exe. The two tables are visible in the scroll box titled *Properties of Selected Item* at the right of the form, when you highlight a tabular equation-of-state gas type. When such a gas is selected, the property display window contains a $\varepsilon(v, T)$ table following the $Z(v, T)$ table, although the energy table is labeled $E(v, T)$ due to the lack of a "ε" character in the available windows font.

To change the compressibility or internal-energy table, click on the *Edit Properties* button below the scroll box, then on the appropriate button of the sub-dialog box. You should now see a file-open dialog enabling you to load a file containing the appropriate data table. You do not enter values one by one as you do for single-valued data or cubic-spline interpolation pairs. You load an entire file of values all at once. This is to allow you to create the property tables using spreadsheet program like Excel in conjunction with a third-party property database or a commercial software package like HePak (Horizon Technologies) or REFPROP (NIST).

The strategy for choosing (v_j, T_k) values is similar to the strategy reported in section 14.4.2. Except more widely spaced values are now preferred, since wide spacings promote smoothness in the derived $c_p(T, P)$ data and there is no need to use fine spacings to increase accuracy in the ε table. Nor is there any need for a temperature guard row (a row of temperatures slightly colder than the previous coldest temperatures). Essentially, choose enough v's and T's to adequately resolve the non-ideal part of the state space at or near the condensation region below T_{crit}. A reasonable set of T values below T_{crit} for helium-4 is something like (3.25, 3.75, 4.25, 4.75). Above T_{crit} something like (6, 7.5, 10, 14, 20, 30, 45, 75, 120, 200, 350, 700, 1500). For specific-volume values in the phase transition region, it is reasonable to use midpoints of the phase-transition specific-volume values corresponding to the T values below T_{crit}.

Once the data is loaded you can inspect it in detail by clicking the Interpolation Details button. Selecting this option will produce a ASCII text file of data tables similar to the actual data except focusing on any particular range of the data you wish, possibly with higher resolution (more data points) than the

actual data so that you can evaluate the bicubic spline interpolation accuracy. You can produce interpolated data files arranged in columns of constant specific volume or constant pressure, over any range you wish, so long as it falls within the range of the original data.

20.2 TBSpline3Gas

TBSpline3Gases are designed for use where pressure changes significantly within same model scope (J-T coolers) or to replace multiple instances of TBSpline2Gases at various pressures.

TBSpline3Gas is a descendant of TBSpline2Gas with specific heat c_p, viscosity μ and conductivity k now defined using (v, T) interpolation tables rather than T-only interpolation pairs. Previous cubic-spline data-pairs (T, Cp), (T, μ) and (T, k) are still present but are now all understood as zero-pressure limits. They are automatically generated on input from the high-v column of the corresponding data table without any additional user input required (in the LTCProp.exe property management software). Previously only Cp was a zero-pressure limit, the other two properties were tabulated for a representative pressure. The only reason c_p is included as a (v, T) table is for the calculation of Prandl number used in certain heat-transfer formulations.

In support of modeling within the two-phase saturation zone TBSpline3Gases contain two new cubic spline variables $v_v(T)$ and $v_l(T)$, representing the saturated vapor and liquid specific volumes (*see* below).

TBSpline3Gases suffer a slight performance penalty compared to TBSpline2Gases because of the increased computation overhead of two-dimensional interpolation over one-dimensional interpolation when accessing c_p, μ and k and an increase in the total number of interpolation points to better cover the non-ideal fluid conditions near the critical temperature and density. A TBSpline3Gas is defined by the following values:

T0 : (K) Representative temperature T_0.

Z(v, T) : (dimensionless) A rectangular table of $Z(v_j, T_k)$ values where $Z = Pv/RT$ is compressibility, v is specific volume and T is temperature. The first (left) column contains the T_k values and the first (top) row the v_j values. The corresponding Z values lie in the body of the table.

ε(v, T) : (J/kg) A rectangular table of mass-specific internal energy values $\varepsilon(v_j, T_k)$, structurally similar to the $Z(v_j, T_k)$ table.

Cp(v, T) : (J/(kg K)) A rectangular table of constant-pressure specific heat values $c_p(v_j, T_k)$, structurally similar to the $Z(v_j, T_k)$ table.

Mu(v, T) : (kg/(m s)) A rectangular table of viscosity values $\mu(v_j, T_k)$, structurally similar to the $Z(v_j, T_k)$ table.

K(v, T) : (W/(m K)) A rectangular table of thermal conductivity values $k(v_j, T_k)$, structurally similar to the $Z(v_j, T_k)$ table.

v_v(T) : (m^3/kg) Saturated vapor specific volume. A cubic spline variable with temperature vs specific volume (T, v_v) data pairs ranging from T_{min} to T_{crit}.

v_l(T) : (m^3/kg) Saturated liquid specific volume. A cubic spline variable with temperature vs specific volume (T, v_l) data pairs ranging from T_{min} to T_{crit}.

The values of the individual properties, including state tables, appear after the enumerated identifier in display windows and the output listing. The choices available when specifying the Gas input variable in the root-level model component come from the default database file LowTGases.dta, or a data base file customized by you using the property data-base editor LTCProp.exe.

20.2.1 Entering Tables of State

In principle you can enter state tables in TBSpline3Gases using the PropBase program LTCProp.exe the same as for TBSpline2Gases, except there are five state tables to enter. TBSpline3Gas gases are identified by the name BSpline3 Gas in the Add menu item of the Propbase program LTCProp.exe. The five tables are visible in the scroll box titled *Properties of Selected Item* at the right of the form, when you highlight a tabular equation-of-state gas type.

A Better Way: RefPropToBSpline3Gas

There is also an automatic way to generate TBSpline3Gases that bypasses all the tedium. Windows application RefPropToBSpline3Gas.exe allows you to read fluid property information in the form of fld property files distributed with the NIST REFPROP fluid property software[32] and directly translate that information to TBSpline3Gas properties. What makes this possible is a dynamic link library refprop.dll, distributed with the REFPROP software. The application RefPropToBSpline3Gas.exe is part of the Low-T Cooler distribution but you will have to obtain the refprop.dll from NIST (http://www.nist.gov/srd/nist23.htm).

RefPropToBSpline3Gas is very simple to use. Just click the "Convert REFPROP fluid property file" button and follow the prompts to select and open the appropriate fluid property file, convert its properties into TBSpline3Gas format and save them to a Sage-format dta file. You may then use the LTCProp.exe program to append the resulting file to the gas-property data base file you use with the Low-T Cooler application (the one selected in the Options|Model Class dialog, Properties tab. This is how the default property file LowTGases.dta was generated.

The implementation of the RefPropToBSpline3Gas program is straight forward but not completely trivial. Special considerations are required for some properties in the two-phase region and at certain low-volume (high density) interpolation points out of REFPROP's range.

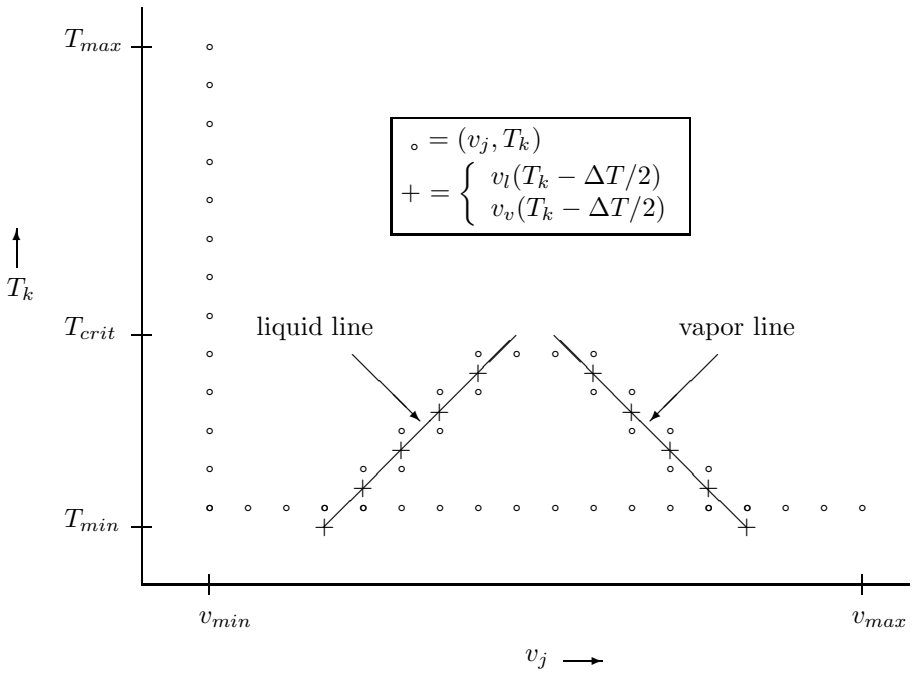


Figure 20.1: BSpline3Gas interpolation points.

Interpolation Points To avoid problems in or near the saturation zone is necessary to choose the (v_j, T_k) interpolation points with care near the boundaries of the two-phase region. These boundaries are defined by the values of saturated vapor and liquid volumes $v_v(T)$ and $v_l(T)$ available in REFPROP. The way this is done is illustrated somewhat abstractly in figure 20.1. v_j and T_k values are drawn in non-linear scale so the saturation curves $v_v(T)$ and $v_l(T)$ appear as straight lines. The temperature points T_k are chosen as follows:

- First a number of points spaced by ΔT from $T_{min} + \Delta T/2$ to just below the critical temperature $T_{crit} - \Delta T/2$. T_{min} is near the minimum value supported by REFPROP for the particular fluid.
- Followed by a number N points spaced by ΔT from $T_{crit} + \Delta T/2$ to $T_{crit} + (N - \frac{1}{2})\Delta T$.
- Followed by a number of points spaced in equal ratios from $T_{crit} + (N + \frac{1}{2})\Delta T$ up to T_{max} (the maximum value supported by REFPROP)

The specific volume points v_j are then chosen with reference to the saturated liquid and vapor densities v_l and v_v tabulated at points offset by $\Delta T/2$ from the above T_k values, at the points $T = T_{min}, \ldots, T_{crit} - \Delta T$:

- First a number of equal spaced points between a minimum value v_{min} and the lowest liquid volume $v_l(T_{min})$.

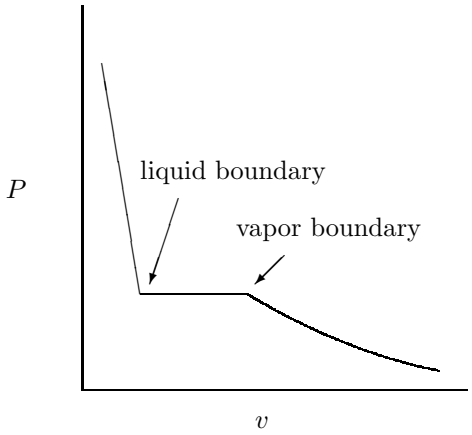


Figure 20.2: Typical pressure isotherm below T_{crit}.

- Followed by the $v_l(T)$ values, $T = T_{min} + \Delta T, \ldots, T_{crit} - \Delta T$
- Followed by a number of equal space points between $v_l(T_{crit} - \Delta T/4)$ to $v_v(T_{crit} - \Delta T/4)$
- Followed by the $v_v(T)$ values, $T = T_{crit} - \Delta T, \ldots, T_{min} + \Delta T$
- Followed by a number of points spaced in equal ratios from $v_v(T_{min})$ up to v_{max}, where v_{max} is the greater of twice $v_v(T_{min})$ and RT_{max}/P_{min}, for P_{min} a reasonable minimum pressure such as 1 atmosphere

Chosen in this way it is a simple matter to know which k correspond to temperatures below T_{crit} and which j, k indices correspond to points in the two-phase zone.

Saturation Pressure In theory and in REFPROP the saturation pressure in the two-phase region depends only on temperature. In other words, at a given temperature the pressure remains constant as specific volume goes from the saturated vapor value $v_v(T)$ (pure vapor) down to the saturated liquid value $v_l(T)$ (pure liquid), as illustrated in figure 20.2. From the point of view of Sage convergence this is bad — for two reasons. First because Sages cubic-spline interpolation functions tend to overshoot or undershoot the value $P(v)$ near the corners $P(v_v)$ and $P(v_l)$ where the slope changes discontinuously. Second because the sudden condensation or evaporation of fluid that results from a small change in pressure in the saturation zone tends to play havoc with the fluid momentum equation.

To remedy these problems the RefPropToBSpline3Gas program takes some liberties with pressure in the saturation zone. Instead of a constant pressure

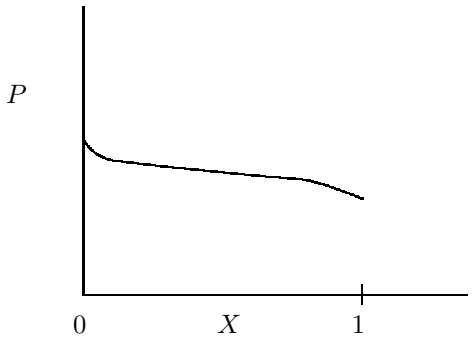


Figure 20.3: Smoothed pressure isotherm between liquid boundary $X = 0$ and vapor boundary $X = 1$. X is just the specific volume relative to the interval over which the smoothing occurs.

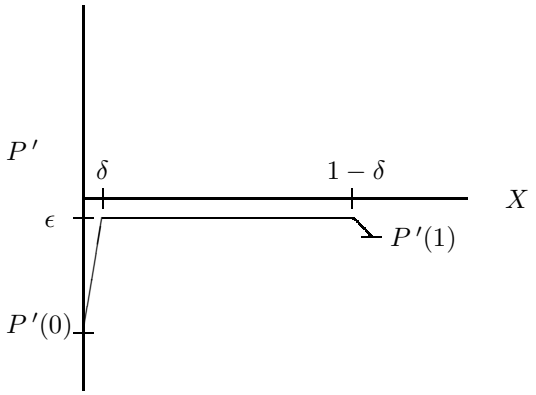


Figure 20.4: Slope of smoothed isotherm P' showing key parameters ϵ, δ, $P'(0)$ and $P'(1)$ used in its construction.

in the two-phase region it produces is monotonely decreasing pressure, first-derivative continuous at the liquid and vapor endpoints. This is illustrated in figure 20.3 which shows the part of the $P(v)$ isotherm between a dimensionless specific-volume $X = 0$ just below the liquid volume v_l and $X = 1$ just above the vapor volume v_v. For any given T_k isotherm, the pressure $P(0)$ corresponds to the pressure at the last v_j value before entering the saturation zone (in the pure liquid region) and the pressure $P(1)$ corresponds to the pressure at the first v_j after leaving the saturation zone (in the pure vapor region). The smoothing function is constructed from its slope $P'(v)$ which is piecewise defined in three parts as illustrated in figure 20.4. The first part between $X = 0$ and δ is a linear transition from the liquid-end slope $P'(0)$ to a value ϵ. The second part between $X = \delta$ and $1 - \delta$ is the constant value ϵ. The third part between $X = 1 - \delta$ and 1 is a linear transition from ϵ to the vapor-end slope $P'(1)$. The value ϵ is chosen to account for a fraction λ of the total change in pressure, namely

$$\epsilon = \lambda(P(0) - P(1)) \tag{20.1}$$

where λ tentatively has the value $1/3$ but is reduced if necessary so that the resulting slope ϵ is never steeper than half the vapor-end slope $P'(1)$. The value δ is chosen to account for the remainder of the pressure change. By considering the triangular areas in figure 20.4 the equation for δ is

$$\frac{\delta}{2}\left[(P'(0) - \epsilon) + (P'(1) - \epsilon)\right] = (1 - \lambda)(P(0) - P(1)) \tag{20.2}$$

The integral of this slope function is a pressure $P(v)$ that varies smoothly in the saturation zone but is not too far from the correct value. A small inaccuracy in the service of numerical stability. This pressure smoothing is embedded in the compressibility data table $Z(v_j, T_k)$ because pressure itself is not directly tabulated.

Other Properties in the Saturation Zone REFPROP does not define transport properties k and μ (conductivity and viscosity) in the saturation zone, presumably due to the confusion of the two-phase state. BSpline3Gases do not have this luxury — they must return something no matter what the fluid state. The obvious thing to do is to interpolate k and μ in the two-phase zone based on the values at the liquid and vapor boundaries (for each T_k isotherm). BSpline3Gases do this using simple linear interpolation because first-derivative continuity of k and μ is not critical as it is for pressure. Likewise BSpline3Gases linearly-interpolate C_p in the two-phase region.

The remaining tabulated property is fluid internal energy ε which is well defined in the saturation zone and BSpline3Gases just use the REFPROP value. Except shifted by a constant offset to make the value always positive, as required by Sage.

Out-of-Range Interpolations Near the extreme low end of the specific volume range (high density end) and at high temperatures the corresponding fluid

pressure is extremely high. Under these conditions REFPROP returns values for the required fluid properties in most cases, even though they may be out of the range of REFPROP interpolation functions. BSpline3Gases accept these property values on the grounds that there is virtually no chance that any reasonable simulation model will operate at such pressures.

The only time RefPropToBSpline3Gas overrides REFPROP returned values is in the rare event when REFPROP returns negative pressure $P(v_j, T_k)$ (sometimes at extremely low specific volumes). In that case RefPropToBSpline3Gas interpolates all properties at (v_j, T_k) by linear extrapolation from the first two in-range tabulated properties in the direction of higher specific volume at the same T_k.

Chapter 21

Low T Components

21.1 Adiabatic Compressor



This component descends from the mass-flow pump component documented in the chapter "Flow Restrictors" of the Pulse-Tube Model-Class Reference Guide. Like the mass-flow pump it provides a mass flow rate, usually in a closed flow-loop, with the pressure rise (or drop) across the component established by the flow characteristics of the overall flow loop. You can see what this pressure rise is by inspecting output variable FDP. As with the mass-flow pump you can uses Sage's optimizer to solve for the resistance of the flow loop to adjust FDP to a predetermined value.

Unlike the mass-flow pump the adiabatic compressor adds the compressor PV power input to the enthalpy flow stream. The ancestor mass-flow pump implemented an isenthalpic flow stream as if compressor PV power input was always balanced internally by an equal but opposite heat flow out of the pump. The present compressor requires an aftercooler or other type of heat exchanger in the flow loop to remove the heat of compression. However, the required PV power input is a virtual part of the model and is indicated only as an output variable (FWc). There is no actual PV power connection required to the compressor from an external Sage component.

It is also possible to run the compressor backwards (e.g. reverse the pressure change for the same mass flow rate) to model an adiabatic expander. In this case there is enthalpy removed from the flow stream by the amount of the expander PV power. In physical terms there must be something in the flow loop to boost the pressure upstream of the expander. This might be another adiabatic-compressor component acting as a true compressor upstream of the expander. In other words, two adiabatic compressor components may be arranged in series within a flow loop with the upstream component serving as the compressor and the downstream component serving as the expander. However, since the time-mean flow of both components is specified as input they must both be exactly equal unless there is someplace for the excess mean flow to go. Into a bypass

flow path for example. Otherwise the solution will not converge. If there is a bypass flow path in the model then the compressor and expander mass flow rates should be exactly equal lest the flow through the bypass be zero, resulting in convergence problems for another reason.

In addition to the variables inherited from the mass-flow-pump, the adiabatic compressor components adds the new output variables:

FWc : (Fourier series, W) compressor PV power output. The sign reflects the Sage convention that positive numbers indicate energy flows out of the gas. So a negative value indicates work flow into the compressor. A positive value indicates work flow out of an expander. Usually the mean value (time-mean power) is the only part of concern, but in some applications the time-varying components may also be of interest.

FHneg, FHpos : (Fourier series, W) Stagnation enthalpy flows $uA(\rho e + P)$ at the component neg and pos boundaries. These two replace output FH inherited from the mass-flow pump.

The formulation for adding the PV power input to the enthalpy flow stream is completely general and remains valid no matter what the mass-flow-rate looks like. The enthalpy flow increment is evaluated each node of the time grid, based on the continuity of entropy across the compressor. Entropy calculations use a general formulation that does not assume ideal gas properties.

A few items documented previously under the mass-flow pump component bear repeating here:

Because the set-point mass flow rate is a Fourier series input, it is possible to produce pulsatile flows — including sinusoidal terms and higher harmonics — as well as steady flows with the adiabatic compressor component. The subfields of the mass flow rate can be optimized or referenced in constraints, as discussed under the topic of Fourier-series variables in the Sage User's Guide. If you specify a nonzero DC part of the mass flow rate, the mass-flow pump should be located within a closed loop capable of supporting DC flow, otherwise the mean pressure drop will be indeterminate and the solution will not converge — if DC flow is impossible, then there is no amount of pressure drop that will force the required value.

The flow area Aflow is another input. It is present in order to define flow velocity in the pump and normally its value does not matter much. However, it should be large enough that the mean Mach number (output MachMean) is substantially below one.

When starting from initial values or a re-initialized solution, an adiabatic compressor in your model can slow down convergence, depending on the magnitude of the flow specified. If this is a problem, try starting with a small or zero DC component of mass flow rate initially and increase it gradually over the course of several solutions. Also be sure to check that the initial temperature value (Tinit) is close to the initial temperatures of the components to which it is connected.

21.1.1 Theory

The flow restrictor theory outlined in the Sage Pulse-Tube Model-Class Reference Guide remains the same except for two changes:

1. A new state variable is added to the solution grid, corresponding to the change of mass-specific enthalpy Δh across the component due to PV power output.

2. The governing equations determining downwind energy densities ρe_+ and ρe_- are revised to include the compressor PV work input to the enthalpy flow stream.

Energy densities ρe_+ and ρe_- at the negative and positive component boundaries are computed implicitly according to an energy balance equation involving incoming and outgoing enthalpy flows and compressor PV work input. This energy balance equation takes the form

$$H_+ + W_c - H_- = 0 \tag{21.1}$$

where the H's are the enthalpy flows through the positive and negative boundaries and W_c is the PV power flow (positive for outgoing). Energy conservation requires that the PV power output equals the product of mass flow rate and mass-specific enthalpy change, or

$$W_c = (\rho u A)\Delta h \tag{21.2}$$

Enthalpy flows may be represented in terms of state variables as

$$H = \frac{\rho u A}{\rho}(\rho e + P) \tag{21.3}$$

In the case of ρe_+, the enthalpy flow at the positive boundary H_+ is taken from local solution variables (including ρe_+ itself) and the enthalpy flow at the negative boundary H_- is imported from the component across from the flow connector at the negative boundary. The case for ρe_- is similar, except the internal and external enthalpy flows come from opposite boundaries. To avoid the energy balance equation becoming indeterminant at zero mass flow rate, the mass flow rate $\rho u A$ is divided out leaving the energy balance equation used for determining ρe_+ in the form

$$\frac{\rho e_+ + P_+}{\rho} + \Delta h - \left(\frac{\rho e + P}{\rho}\right)_- = 0 \tag{21.4}$$

The quantity Δh is a single implicit variable to be determined like any other implicit state variable in the solution grid. In an ideal-gas formulation this implicit variable would be equivalent to $c_p \Delta T$. The equation that determines this implicit variable is a statement of entropy continuity in the form

$$s_+ - s_- = 0 \tag{21.5}$$

where s is mass-specific entropy. The values of s_+ and s_- are calculated from solution variables ρ and T at the component boundaries. Downwind values of ρ and T are already part of the solution. Upwind values are imported from the components across from the flow connectors at each boundary. In the case of an ideal gas, entropy continuity leads to the well known adiabatic compressor power equation

$$W_c = c_p(\rho u A)T_{in}\left(1 - \left(\frac{P_{out}}{P_{in}}\right)^{\frac{\gamma-1}{\gamma}}\right) \tag{21.6}$$

where γ is the ratio of specific heats c_p/c_v.

21.2 Pressure-Regulated Compressor



This component descends from the previous adiabatic compressor component except that the roles of pressure rise FDP and mass flow rate FRhoUA are reversed. It specifies pressure rise as an input and produces whatever mass flow rate is required to achieve that pressure rise.

A pressure-regulated compressor is handy for those situations where the pressures at the intake and discharge of the compressor are known. One of these pressures must be established by a pressure source (pressure bottle icon). To do that just connect an appropriate gas domain near the inlet or discharge to the pressure source. Then the pressure rise FDP specified for the compressor determines the other pressure according to the convention that FDP is the difference $(P_+ - P_-)$, where P_+ is the pressure attached to the positive end of the compressor (right-pointing arrow) and P_- is the pressure attached to the negative end (left-pointing arrow).

For this component to work properly it must be located in a flow loop for which mass flow rate is free to vary and that can produce an equal but opposite pressure drop for some reasonable mass flow rate.

21.3 Flow Reversers





These components serve as special "gas domains" that connect together two negative-facing or positive-facing gas inlets using built in flow-connectors, both directed positively or negatively as the case may be. They provide a simple means to reverse flow direction which otherwise would require an ad-hoc dead-ended volume component. For example, in a J-T cooling loop they can be used to reverse the flow direction upstream and downstream of a counterflow heat-exchanger (*see* the sample input file JTloop.ltc). Essentially, these components just pass information across flow connectors so that the two gas inlets at the far ends of the connectors effectively see each other. Except that velocity is reversed in the process.

The only input variable for flow reversers is the temperature

Tinit : (real, K) Initial temperature. Used only for initializing or re-initializing solution variables ρ and ρe. As such, its value does not affect the final converged solution, but it may affect whether the solution converges at all. It should be set to a value consistent with the initial temperatures of adjacent components.

There are no output variables.

21.3.1 Theory

The job of a flow reverser is not completely trivial because it must adhere to the solution conventions imposed by Sage's flow connector scheme (*see* chapter 8). Basically there needs to be a way to force incoming mass flow rate (ρuA) to equal outgoing mass flow rate. So flow reversers must implement at-least one implicit solution grid variable for this purpose. Having done that they must take additional steps to ensure energy continuity (because local solution values are now decoupled from the incoming values). In the end, flow reversers implement a simplified version of the solution scheme used in flow restrictors (*see* chapter Flow Restrictors of the Pulse-Tube Model-Class Reference Guide). They contain a time grid with state variables ρ, ρe, and P, governed by equations of mass continuity and energy continuity.

ρ

Mass density ρ is computed implicitly from the zero-volume mass-continuity equation

$$(\rho uA)_1 + (\rho uA)_2 = 0 \tag{21.7}$$

where $(\rho uA)_1$ and $(\rho uA)_2$ are the values imported from flow connectors 1 and 2. This forces mass flow rate in two flow connectors to be equal but opposite. Implicit functions used for solving ρe also help to determine ρ.

ρe

Basically, energy density ρe is computed implicitly from the energy continuity equation written in the form

$$\frac{\rho e + P}{\rho} = h_i \tag{21.8}$$

where ρe and P on the left are internal state variables while h_i on the right is the mass-specific enthalpy evaluated using variables imported from the component across the *incoming* flow connection. For an ideal gas h reduces to c_pT in the zero-velocity limit where kinetic energy may be neglected.

For numerical reasons it is convenient to separate ρe into two variables ρe_1 and ρe_2. ρe_1 is exported to flow connector 1 and ρe_2 to flow connector 2. Each is guaranteed valid at the right time — ρe_1 whenever flow is incoming from connector 2 and vice-versa for ρe_2. The reason for this formulation is because the implicit function determining either can be formulated without a

discontinuity at flow reversal, as would otherwise be the case. That is, the equation for determining ρe_1 is always that local mass-specific enthalpy $\frac{\rho e_1 + P_1}{\rho}$ equals the h_i value imported across connector 2. this enforces energy continuity when flow is incoming from connector 2 and otherwise does no harm. A similar equation determines ρe_2. The meaning of P_1 and P_2 is similar to the meaning of ρe_1 and ρe_2, and is explained below.

It might be tempting to use these continuity conditions instead to calculate ρe_1 and ρe_2 explicitly. This will not work because of a circular reference when accessing P, which itself depends on ρe.

P

Pressure is also broken into two components P_1 and P_2, similar to the treatment for ρe. P_1 is exported to the flow connector 1 and P_2 to flow connector 2. Both are calculated explicitly from the equation of state $P(\rho, T)$. This requires temperature which is calculated from the zero-velocity equation of state. For evaluating P_1 the temperature is calculated as $T(\rho, \rho e_1, 0)$. For evaluating P_2 it is calculated as $T(\rho, \rho e_2, 0)$. (The 0's in the argument lists are velocity) Pressures is required in the solution grid so that the flow connectors can properly enforce Bernoulli's law across the connections.

Why zero velocity? Basically, velocity is irrelevant to mass or energy conservation. The only thing velocity determines is the pressure shift across the flow connectors, according to Bernoulli's law. But any pressure shift across the incoming-flow connector, produced by the velocity in the flow reverser, is canceled by an opposite pressure shift across the outgoing-flow connector. So flow reverser velocity does not matter and it is convenient to assume the simplest thing: zero velocity.

21.4 Flow Separator



This component separates a supersaturated incoming flow stream into distinct vapor-phase and liquid-phase streams. It is designed for positive flow only, with the supersaturated flow stream entering through the negative (left) boundary and the vapor and liquid streams exiting through two positive (right) boundaries. The upper right flow connector is reserved for the vapor stream and the lower right boundary is reserved for the liquid stream. The working gas type must be TBSpline3Gas (Chapter 20) for this component to work properly. Otherwise there will always be zero flow directed to the liquid stream. BSpline3Gases are available in the default gas property file LowTGases.dta.

A flow separator is useful for modeling J-T coolers operating as gas liquefiers, where some of the flow stream liquefies after expansion and is no longer available to return through the recuperative counterflow heat exchanger. A flow separator may be inserted into the Sage model just after the J-T expansion orifice. Since Sage can only model continuous closed flow loops that do not abruptly end or begin, the separated vapor and liquid streams must ultimately join together

again somewhere downstream of the flow separator. For example, the liquid stream might pass through a vaporizing heat exchanger then merge with the vapor stream at the warm end of the counterflow heat exchanger and re-enter the compressor intake.

The supersaturated stream condenses inside a flow separator and something must regulate the flow to the vapor and liquid exit streams consistent with the vapor mass fraction of the condensate. What does this is a pressure drop (or rise) ΔP introduced in the outgoing liquid stream, which regulates the flow leaving through the liquid line. This corresponds to the physical reality of a regulated pump of some sort. The flow through this pump is assumed to be incompressible, which means that the fluid density and internal energy do not change but there is an external pumping power input or output required equal to the pressure change ΔP multiplied by the liquid-line volumetric flow rate $((\rho UA)_{b+}/\rho_{b+})$. This pumping power appears as an output variable FWc.

A flow separator is designed to work with any incoming fluid state, not just supersaturated. In the event the incoming fluid is a pure vapor (quality $X = 1$) the flow just passes through unaltered from the incoming flow line (left) to the outgoing vapor line (upper right), with zero flow out the outgoing liquid line (lower left). Actually the liquid-line flow is never allowed to be exactly zero to prevent solution convergence problems in downstream components. In the event the incoming fluid is a pure liquid (unlikely) the flow passes through unaltered to the outgoing liquid line.

A flow separator will work with flow in the reverse direction although it is not clear what physical reality such operation corresponds to.

The input and output variables for a flow separator are:

Tinit : (real, K) Initial temperature. Used only for initializing or re-initializing solution variables ρ and ρe. As such, its value does not affect the final converged solution, but it may affect whether the solution converges at all. It should be set to a value consistent with the initial temperatures of adjacent components.

FWc : (FourierSeries, W) Pumping power done on the liquid flow stream by the control pressure drop ΔP. The sign reflects the Sage convention that positive numbers indicate energy flows out of the fluid. So a positive value indicates expansion work leaving the flow separator. A negative value indicates compression work entering the separator. No problem if small but beware the Sage optimizer relying on such things to boost efficiency.

FT : (Fourier series, K) Gas temperature T.

FP : (Fourier series, Pa) Gas pressure P.

FDP : (Fourier series, Pa) Liquid-line control pressure rise (positive) or drop (negative).

FH : (Fourier series, W) Stagnation enthalpy flow $uA(\rho e + P)$.

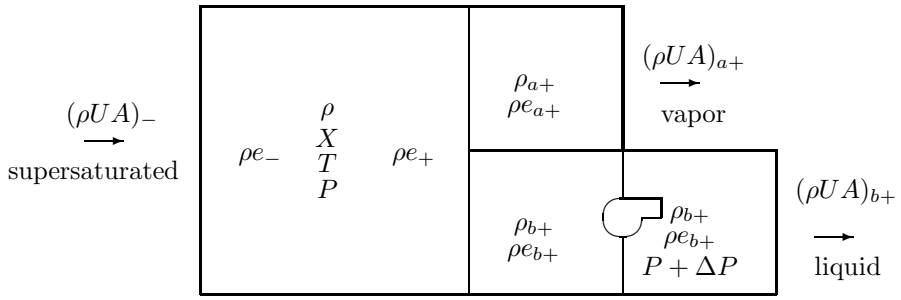


Figure 21.1: State variables for flow separator

FRhoUA : (Fourier series, kg/s) Mass flow rate ρuA.

FX : (Fourier series, dimensionless) Quality (vapor mass fraction) of internal state.

21.4.1 Theory

Flow separator analysis starts out similar to that of the flow reverser component (section 21.3) with the implementation of internal state variables ρ (mass density) and ρe (energy density) based on conservation of mass and energy for a zero-volume component. Separating the presumed supersaturated incoming fluid stream in to liquid and vapor parts is something new and requires a new state variable X (vapor mass fraction or *quality*) and some help from the gas equation of state as to the values of mass densities ρ_v and ρ_l and internal energies e_v and e_l appropriate for saturated vapor and liquid phases at the prevailing temperature and pressure. The diagram in figure 21.1 illustrates the key state variables within a flow separator.

ρ

Mass density ρ is computed implicitly from the zero-volume mass-continuity equation

$$(\rho uA)_{a+} + (\rho uA)_{b+} - (\rho uA)_- = 0 \tag{21.9}$$

where the ρuA's are the mass flow rates imported from flow connectors at the three flow boundaries. Subscript $a+$ denotes the positive vapor-phase exit (upper right), subscript $b+$ denotes the positive liquid-phase exit (lower right) and subscript $-$ denotes the negative inlet (left). Implicit functions used for solving ρe also help to determine ρ. The usage in these other implicit functions is

consistent with ρ being consistent with the incoming flow state no matter what the flow direction.

ρe

Energy density ρe is computed implicitly from an energy continuity equation. For numerical reasons, it is broken into two distinct parts ρe_+ and ρe_-. The first, used for the presumed case of positive flow, is implicitly solved by the equation

$$\frac{\rho e_+ + P}{\rho} = h_- \tag{21.10}$$

The second, used for the unanticipated but possible case of negative flow, is implicitly solved by the equation

$$\frac{\rho e_- + P}{\rho} = X h_{a+} + (1 - X) h_{b+} \tag{21.11}$$

Separating ρe into two parts like this avoids the difficulty of using a single mixed equation that changes right-hand sides on flow reversal. Pressures P is explicitly calculated as explained below. The h's in the above equations denote the incoming mass-specific enthalpies evaluated using variables imported from the associated flow connectors. The subscripts have the same meaning as above. The quality X used as a weight factor in equation (21.11) is appropriate because the mass flow rates through connectors $a+$ and $b+$ are always held in the proportions of X to $1 - X$.

Mass and energy densities ρ and ρe_- are exported directly to the negative flow connector while ρ and ρe_+ form the basis of the mass and energy densities exported to the two positive flow connectors, but are not directly used for that purpose. They must be first broken down into vapor and liquid values ρ_{a+}, ρ_{b+}, ρe_{a+}, ρe_{b+}.

P, T

The pressure P appearing in equations (21.10) and 21.11) is calculated explicitly from the equation of state $P(\rho, T)$. This requires temperature which is calculated from the zero-velocity equation of state, as $T(\rho, \rho e_-, 0)$ or $T(\rho, \rho e_+, 0)$. (The 0 in the argument list is velocity. See section 21.3 on flow reversers for a justification of the zero-velocity assumption.) The formulation $T(\rho, \rho e_+, 0)$ is used when flow is positive and the formulation $T(\rho, \rho e_-, 0)$ in the event flow is negative. The implication, based on the way ρe_+ and ρe_- are calculated, is that P and T are consistent with the incoming flow state no matter what the flow direction. Pressures is required in the solution grid so that the flow connectors can properly enforce Bernoulli's law across the connections. P is directly exported to the negative flow connector and the positive vapor-only flow connector $a+$, but the pressure exported to the liquid-only connector $b+$ is $P + \Delta P$, where the ΔP is used to control the mass flow rate ratios for the two streams, as noted above.

X (Quality)

For a quantity of mixed liquid and vapor at equilibrium having vapor mass m_v and liquid mass m_l, the vapor mass fraction or quality is defined as

$$X \equiv \frac{m_v}{m_v + m_l} \tag{21.12}$$

The total volume occupied by the two states is the sum $m_v/\rho_v + m_l/\rho_l$, where ρ_v and ρ_l are the vapor and liquid densities. This same total volume may also be represented in terms of a mixed-mean density ρ as $(m_v + m_l)/\rho$. Equating these two representations of volume, dividing through by $m_v + m_l$ and using the above definition of quality gives an equivalent definition of quality in terms of the three densities

$$X \equiv \frac{1/\rho - 1/\rho_L}{1/\rho_v - 1/\rho_L} \tag{21.13}$$

In the flow separator model ρ is understood as that of the density state variable as determined by equations (21.9), (21.10), (21.11), and ρ_v and ρ_l come directly from the equation state as a functions of T. Evidently one of the things a flow separator has to do maintain mass balance is pass X mass units of vapor at density ρ_v to the vapor flow connector $a+$ and $1 - X$ units of liquid at density ρ_l to the liquid flow connector $b+$. To avoid problems when ρ is not strictly between ρ_v and ρ_l flow separators actually calculate quality according to the conditional formula

$$X = \begin{cases} 1 & \text{if } \rho \leq \rho_v \\ \text{from equation (21.13)} & \text{if } \rho_v < \rho < \rho_l \\ 0 & \text{if } \rho_l \leq \rho \end{cases} \tag{21.14}$$

But even this is not always valid because not all equation-of-state types available in Sage models define ρ_v and ρ_l and those that do, do so only below the critical temperature. In the event ρ_v and ρ_l are not defined, flow separators assume $X = 1$ (all vapor), and all incoming flow passes directly from the negative connector to the vapor-phase flow connector $a+$.

ΔP

Flow separators need ensure that the ratio of mass flow rates through the vapor and liquid connectors $a+$ and $b+$ are consistent with the vapor mass fraction X. Within the solution scheme of Sage the only way to control mass flow rate is via pressure, as is the case in an actual physical system. So flow separators implement pressure drop ΔP as state variable solved implicitly by the equation:

$$X(\rho u A)_{b+} = (1 - X)(\rho u A)_{a+} \tag{21.15}$$

Flow separators add this ΔP to the pressure exported to the liquid flow connector $b+$ but do not modify exported density ρ_{b+} and energy density ρe_{b+}. This is a reasonable approximation for the liquid line because of the nearly incompressible nature of the liquid state. The assumption that fluid remains in

the liquid state after a pressure change is questionable in the case the pressure change reduces the pressure. Why would the liquid not then vaporize? The answer is that it will in the downstream component, as it would in reality. The physical interpretation of what happens in the flow separator is that the liquid remains in a superheated state. The degree of the pressure change required to distribute the liquid flow and its effect on the downstream state depends on the total system model.

Auxiliary Flow-Connector Variables

Variables ρ_{a+}, ρe_{a+} ρ_{b+} and ρe_{b+} appearing in figure (21.1) are the values of mass density and internal energy passed to the vapor and liquid flow connectors. They are not solved as part of the solution grid but rather just calculated when required as functions of the state variables and the gas equation of state, as follows:

$$\rho_{a+} = \begin{cases} \rho_v & \text{if } 0 < X < 1 \\ \rho & \text{otherwise} \end{cases} \tag{21.16}$$

$$\rho e_{a+} = \begin{cases} \rho_v e_v & \text{if } 0 < X < 1 \\ \rho e_+ & \text{otherwise} \end{cases} \tag{21.17}$$

$$\rho_{b+} = \begin{cases} \rho_l & \text{if } 0 < X < 1 \\ \rho & \text{otherwise} \end{cases} \tag{21.18}$$

$$\rho e_{b+} = \begin{cases} \rho_l e_l & \text{if } 0 < X < 1 \\ \rho e_+ & \text{otherwise} \end{cases} \tag{21.19}$$

The equation of state provides the equilibrium vapor and liquid mass densities and mass-specific energies ρ_v, ρ_l, e_v and e_l as functions of T. Actually, the equation of state supplies mass-specific *internal* energies ε_v and ε_l, but with the zero-velocity assumption these are equivalent to the mass-specific total energies (internal plus kinetic) e_v and e_l.

The abrupt transitions suggested by the above equations at $X = 0$ and $X = 1$ are bad for solution convergence so Sage replaces the abrupt transitions with smooth transitionsover the ranges $0 < X < \epsilon$ and $1 - \epsilon < X < 1$, where ϵ is a small number ($\epsilon = 0.1$ in the current implementation). Sage does the smoothing in such a way that the densities and specific energies ρ_v, ρ_l, e_v and e_l are always consistent with the vapor quality X. The smoothing does not affect the mass flow distribution into the vapor and liquid lines.

Bibliography

[1] M. M. Abbott, *Cubic Equations of State: An Interpretive Review*, in *Equations of State in Engineering and Research*, Advances in Chemistry Series 182, (1978)

[2] B.J. Abu-Ghannam and R. Shaw, *Natural Transition of Boundary Layers — The Effects of Turbulence, Pressure Gradient, and Flow History*, J. Mech. Engineering Science, V. 22, No. 5, pp. 213–228, (1980)

[3] R. Akhavan, R.D. Kamm and A. H. Shapiro, *An Investigation of Transition to Turbulence in Bounded Oscillatory Stokes Flows, Part 1. Experiments*, J. Fluid Mech. vol. 225, pp. 395-422, (1991)

[4] A. Bejan, *Entropy Generation Through Heat and Fluid Flow*, Wiley-Interscience, (1982)

[5] F. J. Cantelmi, *Measurement and Modeling of In-Cylinder Heat Transfer with Inflow-Produced Turbulence*, MS Thesis, Virginia Polytechnic Institute and State University, June (1995)

[6] H.S. Carslaw and J.C. Jaeger, *Conduction of Heat in Solids*, Oxford at the Clarendon Press, (1959)

[7] J.E. Coppage and A.L. London, *Heat Transfer and Flow Friction Characteristics of Porous Media*, Chemical Engineering Progress, Vol. 52, No. 2, pp. 57-63, Feb. (1956)

[8] J.H. Dymond and E.B. Smith, *The Virial Coefficients of Gasses*, Clarendon, (1969)

[9] E.R.G. Eckert and R.M Drake, *Analysis of Heat Mass Transfer*, McGraw-Hill, (1972)

[10] D.K. Edwards and I. Catton, *Prediction of Heat Transfer by Natural Convection in Closed Cylinders Heated from Below*, Int. J. Heat and Mass Transfer, Vol. 12, pp. 23–30, (1969).

[11] D. Gedeon, *Mean-Parameter Modeling of Oscillating Flow*, ASME Journal of Heat Transfer, Vol. 108, pp. 513–518, Aug. (1986)

[12] D. Gedeon, *A Cylinder Heat Transfer Model*, personal memorandum, (Tmodel2.pdf), December (1989)

[13] D. Gedeon, *An Approximate Cylinder Nusselt Number*, personal memorandum, (Tmodel3.pdf), December(1989)

[14] D. Gedeon, *Advection-Driven vs Compression-Driven Heat Transfer*, NASA report under PO C-23433-R, Task 2, (AdvectionVsCompressionSansFigures.pdf), May (1992)

[15] D. Gedeon, *Cylinder Heat Transfer Update*, personal memorandum, (TModel4.pdf), October (1995)

[16] D. Gedeon and G. Wood, *Oscillating-Flow Regenerator Test Rig: Hardware and Theory with Correlelations for Screens and Felts*, NASA-Lewis Contractor Report 198422, (1996)

[17] D. Gedeon, *Flow Streaming in Compliance Tubes*, personal memorandum, (FlowStreamingComplTubes.pdf) (1995)

[18] D. Gedeon, *Flow Streaming in Compliance Tubes: High Velocity Correction*, personal memorandum, (FlowStreamingHiVelCorrection.pdf) (1999)

[19] D. Gedeon, *Sage Tortuosity Reformulation*, personal memorandum (SageTortuosityFormulation.pdf), February (2006)

[20] D. Gedeon, *Random Fiber Data Reduction Results: porosities 85%, 90% 96%*, personal memorandum (NASARandomFiberDataReduction.pdf), January (2006)

[21] D. Gedeon, *Random Fiber Correlations with Porosity Dependent Parameters — Updated with New 96% Porosity data*, personal memorandum (NASARandomFiberMasterCorrelationUpdate.pdf), November (2006)

[22] Heames T.J., Uherka D.J., Zabel J.C., Daley J.G., *Stirling Engine Thermodynamic Analysis: A Users Guide to SEAM1*, Argonne National Laboratory, ANL-82-59, pp. 11–20, Sept. (1982)

[23] K.G.T. Hollands, *Natural Convection in Horizontal Thin-Walled Honeycomb Panels*, Trans. ASME J. Heat Transfer, Vol. 95, pp. 439–444, (1973).

[24] J.P. Holman, *Heat Transfer*, Fourth Edition, McGraw-Hill (1976)

[25] E. Fried, I.E. Idelchik, *Flow Resistance: A Design Guide for Engineers*, Hemisphere, (1989)

[26] V.J. Johnson, *Properties of Materials at Low Temperature (Phase 1)*, Pergamon Press, (1961)

[27] W.M. Kays, A.L. London, *Compact Heat Exchangers, 3rd Edition*, McGraw-Hill, (1984)

[28] B.E. Launder and D.B. Spaulding, *Mathematical Models of Turbulence*, Academic Press, (1972)

[29] K. Lee, *A Simplistic model of Cyclic Heat Transfer Penomena in Closed Spaces*, 18th IECEC, pp. 720–723, (1983)

[30] J.M. Lee, P. Kittel, K. D. Timmerhaus and R. Radebaugh, *Flow Patterns Intrinsic to the Pulse Tube Refrigerator*, International Cryocooler Conference, (1992)

[31] M.A. Lewis and R. Radebaugh, *Measurement of Heat Conduction through Metal Spheres*, in Cryocoolers 11, edited by R.G. Ross, Jr., Kluwer Academic/Plenum, pp. 419–425, (2001)

[32] Eric Lemmon, Mark McLinden, Marcia Huber, NIST Standard Reference Database 23, *NIST Reference Fluid Thermodynamic and Transport Properties — REFPROP*, U.S. Dept. of Commerce, (2002)

[33] I.F. Macdonald, M.S. El-Sayed, K. Mow and F.A.L. Dullien, *Flow through Porous Media – the Ergun Equation Revisited*, Ind. Eng. Chem. Fundam., Vol. 18, No. 3, pp. 199–208, (1979)

[34] NIST Standard Reference Database 12, *Thermophysical properties of pure fluids*, avaliable from http:\\www.nist.gov\srd, (1998)

[35] E. Oberg, F. D. Jones, *Machinery's Handbook*, Seventeenth Edition, The Industrial Press, (1964)

[36] J.R. Olson G.W. Swift, *Acoustic streaming in pulse tube refrigerators: Tapered pulse tubes*, LA-UR-96-3083, (1996)

[37] S. V. Pantakar, W. E. Ibele, W. J Koehler *Numerical Predictions of Turbulent Oscillating Flow — Initial Studies*, NASA-Lewis Progress Report, (1989)

[38] R.P. Peyret and T. D. Taylor, *Computational Methods for Fluid flow*, Springer-Verlag (1983)

[39] W.H. Press, B.P. Flannery, S.A. Teukolsky, W.T. Vetterling, *Numerical Recipes*, Cambridge, (1986)

[40] O. Redlich, J.N.S. Kwong, *On the Thermodynamics of Solutions. V. An Equation of State. Fugacities of Gaseous Soutions*, Chemical Reviews, V 44, p. 233, (1949)

[41] P.A. Rios, *An Approximate Solution to the Shuttle Heat-Transfer Losses in a Reciprocating Machine*, ASME J. Eng. Power, pp. 177–182, (1971)

[42] H. Schlichting, *Boundary-Layer Theory*, Seventh Edition, McGraw-Hill, (1979)

[43] E. Schmidt, *Thermodynamics*, Dover, (1966)

[44] J.R. Seume and T.W. Simon, *Oscillating Flow in Stirling Engine Heat Exchangers*, 21st IECEC, (1986)

[45] J.R. Seume, *An Experimental Investigation of Transition in Oscillating Pipe Flow*, Ph.D. Thesis, University of Minnesota, (1988)

[46] J. Seume, G. Friedman and T.W. Simon *Fluid Mechanics Experiments in Oscillatory Flow*, NASA CR-189127, Lewis Research Center, (1992)

[47] T.W. Simon and J.R. Seume, *A Survey of Oscillating Flow in Stirling Engine Heat Exchangers*, NASA 182108, (1988)

[48] T.W. Simon, M. Ibrahim, M. Kannapareddy, T. Johnson, G. Friedman, *Transition of Oscillatory Flow in Tubes: An Empirical Model for Application to Stirling Engines*, 27th IECEC, (1992)

[49] G.W. Swift, *Thermoacoustic Engines*, J. Acoust. Soc. Am., **84** (4), pp. 1145–1180, (1988)

[50] G.W. Swift, *Thermoacoustics: A Unifying Perspective for some Engines and Refrigerators*, Fourth draft, LA-UR-99-895, 1999

[51] Y.S. Touloukian and C.Y. Ho, *Thermophysical Properties of Matter*, Purdue Research Foundation, Plenum Publishing Corp., (1970)

[52] N. B. Vargaftik, *Tables on the Thermophysical Properties of Liquids and Gases*, Second Edition, Wiley, (1975)

[53] A.M. Wahl, *Mechanical Springs*, McGraw-Hill (1963)

Index

UpwindFrac, 84
ε(v, T), 178, 180
v_l(T), 181
v_v(T), 181
AEQw, 60, 64, 72
AEQx, 56, 60, 64, 72
AEQy, 56, 60
AEQ, 54, 126, 127
AEdiscr, 56, 60, 64, 72
AEfric, 72, 161
Aflow, 108, 161
Aratio, 164
Asec, 108
Asolid, 103
Avoid, 103
A, 54
BVirialErr, 27
Bmult, 151
C1, 26
C2, 26
Ca, Cr, Cs, Cw, Cm, 40
Cd, 163
Cf0, 47
Cf, 46
Cp(v, T), 180
DCRhoUA, 166
DPstdy, 166
Dcan, 162
Dfiber, 112
Dhyd, 116
Din, 104–106
Dliner, 130
Dorf, 163
Dout, 105, 106
Dshell, 130
Dskin, 63
Dsphere, 114, 162
Dtube, 117, 159
Dwire, 111
D, 40, 55, 59, 63
EOSErrMean, 73
Emmis, 127
E, 40
FDP, 72, 161, 193
FHmean, 73
FHneg, FHpos, 188
FH, 162, 193
FIcoil, 47
FM, 73
FPMean, 72
FPNeg, 72
FPPos, 72
FP, 161, 193
FQcombust, 71
FQwNet, 63, 71
FRestrict, 166
FRhoUANeg, 73
FRhoUAPos, 73
FRhoUAmean, 73
FRhoUA, 162, 194
FTMean, 72
FTsMean, 64
FT, 161, 193
FV, 72
FWc, 188, 193
FX, 194
FdC1, 116
FdC2, 116
FdC3, 116
FdM, 116
Fmult, 70, 161
FreqNorm, 25
Freq, 26
FsC1, 116

FsC2, 116
Gap, 114, 125
Gas, 26
Gmult, 151
G, 137
HNeg, 72
HPos, 72
Hchan, 119
Hmult, 70
Icoil, 46
Inorm, 46
K(v, T), 180
K0, 45
Ka, 41
Klocal, 71
KmultBnd, 71
Kmult, 63, 71
KrC1, 116
KrC2, 116
KrM, 116
KrN, 116
Kr, 40
K, 75, 92
Lambda, 63
Length, 103, 107, 125, 130, 134, 161
Lnorm, 25
Lratio, 39
L, 54
MachMean, 73, 162
Mass, 55, 59, 63, 103
Me, 41
MinRestrict, 166
Mu(v, T), 180
M, 40
NCell, 24, 103, 107, 130, 134, 135
NTnode, 24, 25
Nchan, 119
Ntube, 117
NuC1, 116
NuC2, 116
NuM, 116
NuN, 116
N, 40
Omega, 26
PV, 71
Pcharge, 70
Pcrit, 26
Phase, 39
Pnorm, 25
Popen, 164
Porosity, 108, 162
Pwet, 108, 161
P, 25
QNeg, 53, 54, 126, 127
QPos, 53, 54, 126, 127
QfreeMean, 151
QmolMean, 151
Qnorm, 26
QoscMean, 152
QstrMean, 152
QturbMean, 151
QwNeg, 72
QwNet, 53, 59
QwPos, 72
QxMean, 72
QxNeg, 56, 60, 64
QxPos, 56, 60, 64
QyNeg, 53, 56, 59
QyPos, 53, 56, 60
Rcoil, 46
Rcrank, 39
ReMean, 73, 162
Recov, 163
Rgas, 26, 178
Rho, 40
Rn, 45, 47
Roughness, 107
Rp, 45, 47
Sc, 41
Smult, 151
Sm, 41
Solid, 54, 55, 59, 63, 104
Sratio, 122
Swet, 122
S, 137
T0, 26, 178, 180
TNeg, 72
TPos, 72
TbInNeg, 71
TbInPos, 71
TbMean, 73
Tcrit, 26

TdMean, 73
Tfin, 121
Thk, 114
Tinit, 104, 108, 130, 134, 135, 161, 191, 193
Tmult, 151
Tnorm, 24, 26
Tn, 41
Tortuosity, 59, 63
TsNeg, 54, 56, 60, 64, 125, 127
TsPos, 54, 56, 60, 64, 125, 127
Ts, 53
Twall, 108, 159
T, 40, 95
UpwindFrac, 71
VaMean, 73
Vmean, 72
Volume, 122
Wcan, 104, 105
Wchan, 119
Wcoil, 46
Win, 105, 106
Ws, 41
W, 40, 55, 59
XNeg, 125
XPos, 125
Xm, 45, 47
Z(v, T), 27, 178, 180
ZMean, 73
conductivity, 26, 52, 178
density, 52
g, 138
specific heat, 26, 52, 178
s, 138
viscosity, 26, 178

accuracy, 24
adiabatic compressor
    as expander, 187
    convergence issues, 187, 188
    energy balance, 189
    entropy continuity, 189
    ideal-gas PV power, 190
    pulsatile flows, 188
    PV power, 187
    theory, 189
annulus shuttle/seal/appendix, 17, 18, 125
appendix gap flow, 125
available energy, 23, 52, 70
    balance principle, 24
axial conduction
    channels, 121
    cylinder, 124
    fins, 122
    foil, 115
    generic, 117
    random fibers, 113
    screens, 112
    spheres, 114
    tubes, 119
axial conductivity enhancement, 151
axial stiffness, 42

bar conductor, 7, 17, 54
Bernoulli's law, 87
bicubic spline, 177
boundary convection, 154
busbar, 13

canister, 103
    annular, 104
    annular-cone, 106
    tubular, 7, 10, 104
    tubular-cone, 105
charge line, 6
charge pressure, 85
clearance seal flow, 125
combustion heating, 83
combustion-space gas domains, 76
compliance tube, 149, 159
    tapered, 159
composite model components, 129
compressibility, 28, 32
compressor
    pressure regulated, 190
    adiabatic, 187
conduction
    enhanced, 110
conduction continuity, 88
conductive surface, 6, 9, 11, 19, 56
conductivity, 27, 28

constrained piston and cylinder, 17, 130, 131
container, 133
  multi-length, 135
  parallel, 14, 16, 134
continuity equation, 76, 78
control volumes, 84
  min resolving pressure drop, 85
control volumes and accuracy, 157
convective triggering, 93
coordinates
  gas domain, 68
  thermal solid, 50
cylinder, 122

DC flow, 164
density, 169, 191, 194
density connections, 21, 67
discharge coefficient, 163
displacer, 16
  constrained, 130
  free, 130, 131
  kinematic, 131
  leakage, 18
  regenerative, 18, 129
distributed conductor, 13, 55
duct gas domains, 75

energy density, 29, 30, 33, 34, 170, 191, 195
energy equation, 76, 79
energy flow continuity, 87
enhanced axial conduction, 83
enthalpy flow continuity, 88
entropy, 23, 29, 30, 34
  gas domain, 69
  thermal solid, 51
equation of state
  entering state tables, 179, 181
  error, 29, 35
  ideal, 27
  in gas domain, 83
  Redlich-Kwong, 29, 30, 139
  stability issues, 178
  Tabular, 35, 141
  TBSpline2Gas, 177

film heat transfer, 81
flexure
  springs, 40
  stack, 40
  stack couple, 41
  theory, 41
floating isothermal surface, 65
flow area, 162, 169
flow connections, 21, 67
flow inlets, 89
flow restrictor, 161
  asymmetric sharp-edged, 163
  check valve, 164
  DC flow blocking, 166
  mass-flow driver, 168
  mass-flow pump, 167
  sharp-edged, 163
  sintered powder plug, 162
  time-dependent valve, 165
flow reverser
  theory, 191
flow reversers, 190
flow separator, 192
  theory, 194
flow velocity, 170
force connections, 21, 67
free convection, 152
free-piston and cylinder, 130–132
friction factor, 162
  channels, 120
  cylinder, 123
  fins, 122
  foil, 115
  generic, 117
  random fibers, 113
  screens, 111
  spheres, 114
  tubes, 118

gas constant, 138
generic cylinder, 5, 122
generic matrix, 116

heat conductor
  bar, 54
  distributed, 55

surface, 56
heat exchanger
conductive matrix, 11
finned, 9
interconnected, 14
isothermal duct, 8
isothermal matrix, 10
heat flow connections, 21, 49
heat sink, *see* heat source
heat source, 53
line, 9, 11, 53
point, 53
surface, 53
heat transformer, 135
hydraulic diameter, 75, 162

ideal gas, 27
interpolation order, 84
isothermal surface, 6, 9, 11, 19

kinematic mechanism, 39

linear motors, 46
local loss coefficient, 162

mass flow rate, 169
matrix gas domains, 75
mean pressure, 85
molecular conduction, 151
momentum equation, 76, 79
momentum flow continuity, 87
motors
linear, 46

numerical diffusion, 157
Nusselt number
channels, 121
cylinder, 123
fins, 122
foil, 115
generic, 117
random fibers, 113
screens, 111
spheres, 114
tubes, 118

orifice, *see* flow restrictor

packed sphere matrix, 114
parasitics, 125
Peclet number, 109
piston
constrained, 130
free, 130, 131
kinematic, 131
piston-cylinder composites, 129
porosity, 109
Prandtl number, 28, 109
pressure connections, 21, 67
pressure drop resolving, 85
pressure source, 6, 70
properties
auto-update, 138
changing, 137
cubic-spline interpolation, 138
data base, 137
gas, 26, 138
solid, 52, 138

quasi-adiabatic surface, 60

radial stiffness, 43
radiation transport path, 17, 127
random fiber matrix, 11, 112
recovery area ratio, 164
rectangular channels, 14, 119
rectangular fins, 9, 121
Redlich-Kwong, *see* equation of state
regenerator, 7
Reynolds number, 75, 162
turbulent, 102
rigorous surface, 7, 64
root component, 25

sample files, 3, 149, 175
second-law balance, 24
shuttle heat transfer, 17, 125
simple crank piston, 39
simple-crank piston and cylinder, 131
sonic velocity, 28
specific heat, 28
springs
flexure, 40
nonlinear, 45

nonlinear relative, 46
staggered grid, 84
streaming convection, 155
submodels, 133

thermal dispersion, 110
thick surface, 6, 64
thin surface, 64
torsional stress, 43
tortuosity, 59, 61, 109
foil, 115
generic, 117
random fibers, 113
screens, 112
spheres, 114
tube bundle, 8, 14, 117
tube bundles, 16
turbulence intensity, 94
turbulence kinetic energy, 101
turbulence models, 92
turbulent conduction, 110, 152
turning angle, 42

Valensi number, 75
variable-volume gas domains, 75
variable-volume space, 5
virial equation, 31
viscosity, 27, 28
viscous pressure drop, 162
viscous pressure gradient, 79

wetted perimeter, 75, 162
woven screen matrix, 7, 18, 111
wrapped foil matrix, 114